

40 CONCEPTOS
CLAVE **IA**
SOBRE INTELIGENC
ARTIFICIAL

RAFAEL CABALLERO · ENRIQUE MARTÍN



40 CONCEPTOS
CLAVE **IA**
SOBRE INTELIGENC
ARTIFICIAL

RAFAEL CABALLERO · ENRIQUE MARTÍN



40 CONCEPTOS
CLAVE **IA**
SOBRE INTELIGENC
ARTIFICIAL

RAFAEL CABALLERO · ENRIQUE MARTÍN



Índice

[Prólogo](#)

- [1. ¿Qué es la inteligencia artificial?](#)
- [2. El sueño de las máquinas pensantes](#)
- [3. ¿Quién fue Alan Turing?](#)
- [4. Las primeras décadas de la inteligencia artificial](#)
- [5. ¿Qué es el test de Turing?](#)
- [6. Un minitest de Turing: los CAPTCHA](#)
- [7. ¿Qué es el aprendizaje automático?](#)
- [8. Aprendizaje supervisado](#)
- [9. Aprendizaje no supervisado](#)
- [10. Aprendizaje por refuerzo](#)
- [11. ¿Cómo saber si un modelo es realmente adecuado?](#)
- [12. Big data](#)
- [13. Hardware específico para IA](#)
- [14. Neuronas artificiales](#)
- [15. Redes neuronales artificiales](#)
- [16. Deep learning](#)

- [17. IA generativa](#)
- [18. Redes generativas adversativas](#)
- [19. Los transformadores](#)
- [20. Aprendizaje por transferencia y ajuste fino](#)
- [21. Procesamiento del lenguaje natural](#)
- [22. Representaciones vectoriales](#)
- [23. Visión artificial](#)
- [24. LLM: los grandes modelos de lenguaje](#)
- [25. Modelos de lenguaje que toman decisiones](#)
- [26. Ingeniería de instrucciones](#)
- [27. Alucinaciones y otras criaturas](#)
- [28. Generación aumentada por recuperación](#)
- [29. Modelos de IA propietarios y abiertos](#)
- [30. Sesgos en el aprendizaje automático](#)
- [31. Impacto de la IA en la vida de las personas](#)
- [32. La legislación de la IA](#)
- [33. Conjuntos de entrenamiento y derechos de autor](#)
- [34. De OpenAI a DeepSeek: el nuevo tablero geopolítico de la IA](#)
- [35. Ataques a los sistemas de IA](#)
- [36. Lenguajes de programación para inteligencia artificial](#)
- [37. El problema de la inercia del conocimiento](#)

[38. Otras formas de IA. Explicabilidad](#)

[39. IA bioinspirada](#)

[40. Superinteligencia](#)

[Bibliografía](#)

[Índice de Términos](#)

Hitos

[Cover](#)

[Bibliografía](#)

[Página de título](#)

[Página de copyright](#)

[Dedicatoria](#)

[Prólogo](#)

[Índice temático](#)

[Índice de contenido](#)

Rafael Caballero

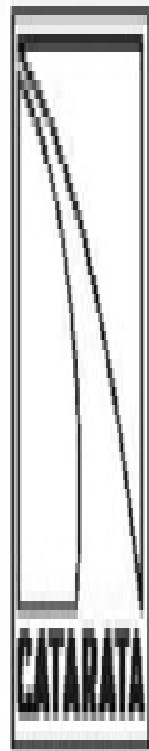
Ingeniero técnico en Informática y doctor en Ciencias Matemáticas. Profesor de la Facultad de Informática de la Universidad Complutense de Madrid con 25 años de experiencia en bases de datos y gestión de la información. También es autor de más de cien publicaciones científicas y de varios libros sobre lenguajes de programación. Aplica su interés por big data a los grandes catálogos astronómicos, habiendo descubierto mediante el análisis de estos catálogos más de 500 estrellas dobles nuevas.

Enrique Martín

Doctor en Ingeniería Informática y profesor en la Facultad de Informática de la Universidad Complutense de Madrid desde 2007, impartiendo cursos sobre bases de datos, gestión de la información y big data tanto en grados como en másteres. Su investigación principal gira en torno al uso de métodos formales para el análisis de programas en entornos distribuidos, incluyendo sistemas robóticos y contratos inteligentes en cadenas de bloques.

Rafael Caballero y Enrique Martín

**40 conceptos clave sobre
inteligencia artificial**



© Rafael Caballero y Enrique Martín, 2025

© Los libros de la Catarata, 2025

ZURBANO, 76

28010 MADRID

Tel. 91 532 20 77

www.catarata.org

40 conceptos clave sobre inteligencia artificial

isbn: 978-84-1067-454-7

ISBN: 978-84-1067-413-4

DEPÓSITO LEGAL: M-22.140-2025

thema: UYQ/UXJ/PDR

este libro ha sido editado para ser distribuido. La intención de los editores es que sea utilizado lo más ampliamente posible, que sean adquiridos originales para permitir la edición de otros nuevos y que, de reproducir partes, se haga constar el título y la autoría.

*Para Josele. Te fuiste demasiado pronto,
pero tu recuerdo siempre estará con nosotros*

Prólogo

Aún recordamos cómo, a finales de la década de los noventa del siglo pasado y principios de los 2000, aparecieron programas de reconocimiento de voz como Dragon NaturallySpeaking, que permitían hablar a través del micrófono del ordenador y transcribían el texto de manera automática. En aquellos momentos, parecía algo absolutamente mágico, que podía reemplazar completamente a la habilidad de escribir en el teclado del ordenador. No funcionaban particularmente mal, pero aun así aprender a escribir en el teclado con soltura era una decisión acertada (y creemos que lo sigue siendo). También recordamos los sistemas de reconocimiento óptico de caracteres que tenían los escáneres de la época, que prometían entender el texto de una hoja manuscrita y generar un fichero de Word totalmente preciso. De nuevo era algo que parecía mágico, solo que en este caso hemos de reconocer que a esa tecnología aún “le faltaba una vuelta” para ser útil. Vemos, por tanto, que la inteligencia artificial lleva entre nosotros bastante tiempo y, aunque sistemas como estos que hemos comentado en aquellos años no eran demasiado precisos, ya estaba plantada la semilla de la gran revolución que se desataría en pocos años.

En esa misma época, primera década de los 2000, empezaron a aparecer tecnologías big data destinadas a almacenar y procesar grandísimas cantidades de datos, lo que, entre otras cosas, dio un gran empujón a una técnica de inteligencia artificial conocida como aprendizaje automático mediante redes neuronales, un método que, como veremos, saca excelente partido de esta situación. Sin embargo, el gran punto de inflexión en lo que se refiere a la inteligencia artificial ocurrió el 30 de noviembre de 2022, cuando OpenAI presentó ChatGPT y lo puso a disposición de todos los usuarios. Ese día el público en general se dio cuenta de cuánto había avanzado la inteligencia artificial. Incluso los que trabajaban en este campo se sorprendieron al ver cómo se podía preguntar cualquier cosa y obtener del sistema frases perfectamente construidas y con mucho sentido (salvo algunas excepciones). La nueva herramienta nos permite, por ejemplo, hacer resúmenes de grandes textos con facilidad, traducir entre distintos idiomas teniendo en cuenta el contexto e incluso pedirle actividades consideradas artísticas como escribir cuentos o componer versos. Y lo hace, en general, con mucha solvencia.

Ya no se trata de promesas de ciencia ficción, sino que es algo tangible que usamos a diario en nuestros ordenadores, móviles y tablets, permitiéndonos, en muchos casos, automatizar gran cantidad de las tareas que realizamos en nuestro día a día. El resultado es un cambio radical en la estructura productiva en la que se basa nuestra sociedad, un cambio muy veloz que genera ilusión, pero también vértigo e incluso temor. Y es que, aunque la utilicemos a diario, a menudo desconocemos cómo funciona y esto nos impide no solo aprovechar correctamente sus posibilidades, sino ejercer un análisis crítico de esta tecnología disruptiva.

El objetivo de este libro es ayudar a lograr este conocimiento presentando las principales claves de la inteligencia artificial de una manera lo más accesible posible. Queremos dar una visión general de la evolución de la inteligencia artificial y explicar los principales fundamentos que subyacen a herramientas como ChatGPT. Consideramos que mostrar lo que se esconde tras las bambalinas es importante para desmitificar y contextualizar el funcionamiento de estas potentes herramientas. Si logramos nuestro propósito, la próxima vez que el lector haga una petición a un chatbot inteligente entenderá por qué contesta de la forma que lo hace, lo que le ayudará a utilizar mejor la herramienta y también a decidir si quiere emplearla o no.

La magnitud del cambio que supone la inteligencia artificial hace que no podamos limitarnos a verla únicamente como un avance tecnológico. Por eso, en el libro consideramos su impacto a nivel económico, social, cultural, científico, etc., pues esos elementos también son necesarios para evaluar el impacto de estos avances en su manera más amplia y poder convivir con ella de la manera más sana posible.

El público objetivo de este libro son los lectores que tengan un conocimiento básico de tecnología, pero sin ser especialistas en la materia. Al presentar las claves más técnicas de la inteligencia artificial y sus fundamentos, hemos buscado incluir explicaciones lo más accesibles posible, pero sin renunciar al rigor que permite acceder a un conocimiento suficientemente preciso. En ese sentido, hemos intentado omitir todas las ecuaciones y fórmulas matemáticas, aunque hemos de confesar que alguna nos ha quedado. A la hora de presentar algunos conceptos técnicos hemos evitado aspectos que no consideramos imprescindibles para una comprensión general, por lo que algunas explicaciones no serán totalmente precisas desde un punto de vista técnico. Un lector con conocimientos más avanzados de inteligencia artificial notará rápidamente estas

ausencias, pero esperamos que disculpe estas omisiones, necesarias para transmitir con facilidad las ideas principales a un público más amplio.

El campo de la inteligencia artificial avanza a mucha velocidad, apareciendo técnicas y modelos nuevos y más potentes prácticamente cada mes. Aunque en esta obra hemos intentado centrarnos en los fundamentos más estables de esta tecnología, en algunos casos hemos necesitado referirnos a aspectos punteros del campo actualizados hasta mediados de 2025. Sabemos que eso significa que si se lee este libro, digamos, en 2027, algunos de los comentarios estarán, por fuerza, desactualizados. Esperamos que con la información fundamental presentada el lector pueda detectar estos aspectos obsoletos, acepte nuestras disculpas y pueda buscar información actualizada en aquellos aspectos que le interesen.

Este libro está organizado en 40 capítulos, cada uno dedicado a un tema clave de la inteligencia artificial y ordenados de tal manera que una lectura de principio a fin los aborde en la secuencia que consideramos más adecuada para su comprensión en conjunto, por lo que los autores recomendamos este orden de lectura. Temáticamente, los capítulos se pueden dividir en cinco bloques. El primer bloque, formado por los cuatro primeros capítulos, repasa la breve pero apasionante historia de la inteligencia artificial, imprescindible para entender cómo hemos llegado al desarrollo actual. Un segundo bloque, algo más técnico, y que ocupa desde el capítulo 6 hasta el 23, describe las principales tecnologías involucradas, incluyendo términos fundamentales como deep learning, big data, transformadores, etc. Son conceptos fundamentales para entender los actuales modelos de lenguaje como ChatGPT o Gemini, de los que nos ocupamos en el tercer bloque, que incluye desde los capítulos 24 al 28. El cuarto bloque, del 29 al 37, reflexiona acerca del impacto de la inteligencia artificial en la sociedad desde diferentes puntos de vista. Nuestro viaje termina presentando algunas técnicas alternativas de IA y discutiendo el futuro de la inteligencia artificial.

Hemos escrito cada capítulo con la intención de que sea autocontenido e independiente de los demás. Por lo tanto, el lector interesado en algunos aspectos concretos podría acudir directamente a ellos y, en caso de necesitarlo, consultar el índice de términos para descubrir en qué capítulos se han presentado los distintos conceptos que desee explorar con más detalle.

Esperamos que el lector disfrute de la lectura de este libro tanto como nosotros al escribirlo y que, tras finalizarlo, tenga una comprensión más profunda tanto de los fundamentos técnicos que sostienen los sistemas de inteligencia artificial

como de su impacto en los diversos aspectos de nuestra vida. La inteligencia artificial ha llegado para quedarse y seguiremos empleando este tipo de sistemas a diario, pero esperamos que conocer sus claves permita a sus usuarios ser más eficientes, conscientes y exigentes en su utilización.

1. ¿Qué es la inteligencia artificial?

En un libro sobre inteligencia artificial, a la que nos referiremos a menudo por sus siglas, IA, debería quedar claro desde el principio de qué estamos hablando al usar este término. Por desgracia, empezamos mal: no hay un consenso acerca de este asunto, y sí, en cambio, interminables debates acalorados. Asociar el término inteligencia a un ordenador resulta controvertido y para muchos es un simple término propagandístico que falsea lo que la máquina hace realmente.

La postura extrema en favor de la inteligencia “no humana” vendría representada por el pansiquismo, que plantea que todo objeto tiene alguna forma de mente o inteligencia. Es una postura un tanto radical, así que normalmente se utiliza su versión débil que indica que solo ciertas máquinas alcanzan la condición de inteligentes. Así, John McCarthy, el científico que acuñó el término IA allá por 1955, decía que la inteligencia artificial es “la ciencia y la ingeniería de fabricar máquinas inteligentes, especialmente programas de ordenador inteligentes”. Esta definición no aclara mucho, al dejar abierto el problema que subyace a esta discusión: qué es realmente la inteligencia. Lo que sí podemos apuntar es que los defensores de aplicar el término inteligencia a ciertas máquinas, como Yuval Noah Harari, acostumbran a distinguir entre los términos inteligencia y conciencia, defendiendo que puede darse la primera sin la presencia de la segunda, como de hecho ejemplifica la propia IA.

En el extremo opuesto se encontraría la visión que podemos llamar biológica, que defiende que la inteligencia es una característica inherente a ciertos animales superiores o incluso solo humana (en este caso podríamos hablar de una perspectiva antropocéntrica), por lo que la inteligencia artificial simplemente no existiría. Para los defensores de este punto de vista, como el físico Roger Penrose, los ordenadores pueden darnos información útil, pero eso no requiere inteligencia, porque realmente no saben qué significa lo que nos están mostrando, simplemente procesan símbolos pero no los comprenden. El argumento clave es la necesidad de comprensión o conciencia como paso previo a poder hablar de inteligencia. De esta forma, un ordenador no podría ser inteligente porque aunque “sepa cosas”, no sabe que las sabe.

Por supuesto hay otras formas de enfocar el problema, como el materialismo versus dualismo que mencionaremos más adelante, pero incluso en esta primera aproximación ya podemos anticipar la complejidad del debate.

En un intento de encontrar una definición más humilde, pero que pueda adecuarse a lo que se entiende por IA en nuestros días, hablaremos aquí de inteligencia artificial como el intento de reproducir mediante procesos computacionales tareas complejas típicamente humanas. Sabemos que esta definición no escapa al debate y que incluso tendrá el dudoso honor de ser criticada por unos y otros. Los defensores del pansiquismo nos acusarán de antropocentrismo por proponer una definición ligada a las características de la especie humana. ¿Es que acaso un ser para ser inteligente necesita hacer “cosas de humanos”? ¿Somos la medida de la inteligencia? Los antropocentristas, por su parte, nos acusarán de dar una definición operacional o funcionalista, es decir, de trocar torticeramente la idea de inteligencia por la de simulación o apariencia de inteligencia. Debemos aceptar ambas críticas, pero escudándonos en que no es nuestro propósito definir la inteligencia ni tampoco estamos planteando una postura filosófica determinada, sino que buscamos una noción de lo que se entiende comúnmente por este término. Quizás podríamos excluir la palabra humanas de nuestra definición y dejarlo en tareas complejas, pero la replicación de tareas humanas es lo que más llama la atención de la IA reciente y creemos que merece ser enfatizado.

Por desgracia, incluso nuestra humilde definición es problemática: ¿cuáles son esas tareas complejas típicamente humanas? Como veremos al hablar de la historia de la IA, este concepto ha ido evolucionando, quizás porque parece que las tareas a las que se refiere deben ser tan complejas que un ordenador no pueda resolverlas y, cuando lo hace bien, es que eso no era inteligencia artificial. Larry Tesler, conocido por ser el informático que ideó el tan aplicado concepto de copiar y pegar en un ordenador, resumió esta idea mediante la siguiente frase: “La inteligencia artificial es aquello que todavía no se ha hecho”.

Quejas de informáticos aparte, en nuestro caso sí podemos al menos aproximar lo que queremos decir por tareas complejas típicamente humanas. Nos referimos a tareas que no se pueden describir con precisión mediante un conjunto pequeño de reglas sencillas. Por ejemplo, hacer una suma de dos números positivos con idéntico número de dígitos no es una tarea compleja típicamente humana en nuestro sentido, porque admite un método o algoritmo simple, que nos enseñan en el colegio:

Empieza a sumar dígito a dígito desde la derecha. Hay una tabla que aprendemos para resolver este dato, aunque de pequeños lo hagamos a menudo contando con los dedos.

Si la suma es mayor que 10, digamos 16, y no estás sumando los últimos dígitos, pon debajo, en el resultado, solo el último dígito, el 6, y apunta que “te llevas 1”, es decir, recuerda incrementar en uno la siguiente suma.

Si la suma es mayor de 10 y era la última, entonces sí apunta sin más el resultado.

Por tanto, cuando un ordenador implementa este método no estamos ante inteligencia artificial según nuestra definición.

Para ver un ejemplo del caso contrario pensemos en describir la escritura humana mediante una serie de reglas. Simplificando mucho, consideremos solo el problema de describir una sencilla letra “o”. Podríamos decir, por ejemplo “una letra o se representa por un círculo”. Pero esto sería inútil, porque ningún humano hace un círculo perfecto al hacer esta letra. Podemos, por supuesto, ampliar el concepto diciendo que realmente es un óvalo, pero mucha gente no cierra totalmente el círculo o añade un adorno tras cruzarse el final y el inicio. Admitamos entonces que el óvalo no esté cerrado, sino que pueda estar abierto por arriba. Tras este añadido será fácil confundir una letra “u” con una “o”, ya que la “u” está abierta por arriba. Si lo pensamos bien, veremos que por más reglas que añadamos no llegaremos a precisar del todo la enorme variedad de letras “o” que escribimos los humanos. Seguramente, llegará un punto en el que incluso las reglas se contradigan. Resulta difícil especificar algo tan aparentemente simple, porque los humanos, cada uno con su cultura y personalidad, tendemos a hacer las cosas de muchas formas diferentes. Por eso escribir una “o” para una máquina es una tarea trivial, pero reconocer esa misma letra escrita por un humano cualquiera se convierte en una tarea compleja típicamente humana. Nosotros entendemos bastante bien (salvo que lo haya escrito nuestro médico) lo que escriben otros humanos, pero sin embargo no sabemos explicar con detalle cómo lo hacemos. Por eso diremos en este texto que el reconocimiento de letras escritas por humanos, junto con otras tareas

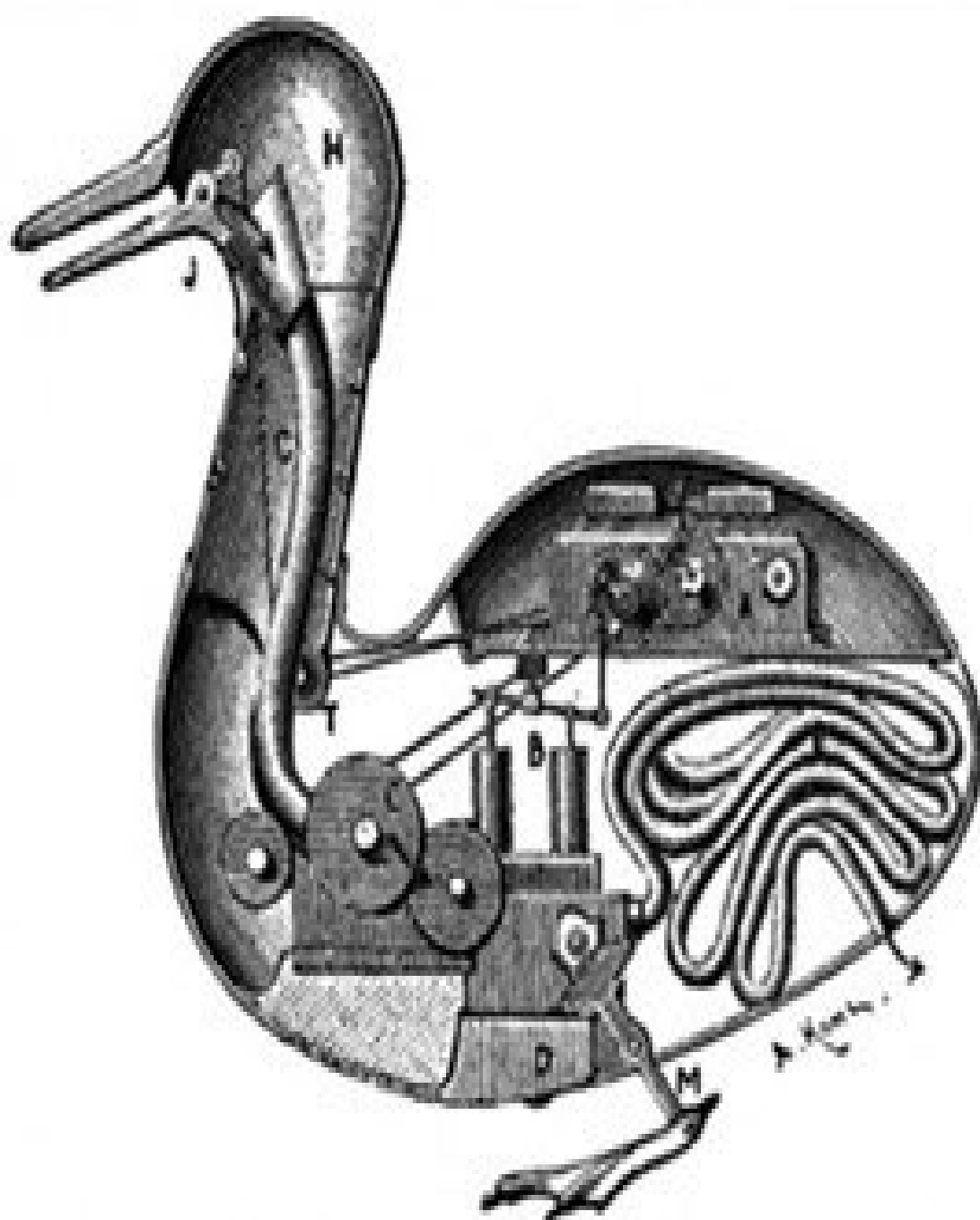
como reconocer la voz humana o ser capaz de conducir un coche en una carretera por la que circulan otros (imprevisibles) humanos son tareas que consideramos asociadas a la inteligencia artificial.

2. El sueño de las máquinas pensantes

La idea de crear artefactos complejos e inteligentes, criaturas “construidas y no nacidas”, parece que ha acompañado al hombre desde una época muy temprana. Cuando Homero escribe la *Ilíada* en el siglo VIII antes de nuestra era, poniendo por escrito un relato transmitido de forma oral durante siglos, incluye en su obra a “doncellas de oro, semejantes a muchachas vivas, que tenían juicio en el pecho, voz y fuerza, y sabían ejecutar obras de los inmortales dioses”. Estas criaturas habían sido creadas por Hefesto, dios del fuego y la forja, y hábil artesano, conocido posteriormente como Vulcano por los romanos. Hefesto también es el autor de Talos, el formidable gigante de bronce encargado de custodiar la isla de Creta para el rey Minos. Es interesante notar que, aunque, como vemos, estas prodigiosas criaturas mecánicas ya habitaban el imaginario humano, su creación, tanto en la época de Homero como durante muchos siglos, parecía requerir siempre de algún tipo de intervención divina.

En el siglo XVII, el punto de vista empieza a cambiar con la llegada del racionalismo, que sustituye la apelación a la intervención divina por un marco de causalidad mecánica y matematizable. Así, René Descartes opina que los humanos están compuestos por dos sustancias, la mente y el cuerpo, en lo que se conocerá como dualismo. Para Descartes los animales solo tienen una de estas dos sustancias, la corporal, y los representa como meras máquinas. Un siglo después, Julien Offray de La Mettrie, representante del materialismo defiende que el hombre también está formado por una única sustancia en su escandalosa obra *El hombre máquina*, de 1748, que le valió tener que huir de Holanda, adonde se había refugiado huyendo, a su vez, de su tierra natal, Francia. La comparación de animales y humanos con artefactos se vio posiblemente favorecida por el desarrollo de numerosos mecanismos en la época, desde los primeros relojes de bolsillo en tiempo de Descartes hasta el ingenioso pato de Vaucanson en vida de La Mettrie, un pato mecánico considerado la primera mascota robótica al ser capaz de simular la ingestión, la digestión y la excreción de granos de cereal.

**Imagen del Pato de Vaucanson en
la revista Scientific American (1899)**



INTERIOR OF VAUCANSON'S AUTOMATIC DUCK.

A, clockwork; *B*, pump; *C*, mill for grinding grain; *F*, intestinal tube;
J, bill; *H*, head; *M*, feet.

Fuente: Wikimedia Commons.

La mejora paulatina de estos ingenios mecánicos llevó a los más audaces pensadores a considerar si no se podría construir, esta vez sí por humanos, un autómatas que simulara en todo a una persona. A principios del siglo XIX, el escritor Ernst Theodor Amadeus Hoffmann plantea esta idea en sus inquietantes relatos *El hombre de arena* y *Los autómatas*, donde explota a la perfección la atracción y rechazo que causan entre los seres humanos los robots, término que acuñará el escritor checo Karel Čapek un siglo más tarde.

Una característica común a todas estas propuestas, ya fueran de ficción o artefactos reales, es que se centraban en lo que hoy llamaríamos hardware, es decir la construcción de los dispositivos y sus mecanismos asociados, y no en el software o cómo lograr que los autómatas tuvieran el comportamiento esperado. Y es natural, porque en la época todavía no existía el concepto de programación de un dispositivo. Un primer paso en este sentido lo dio Joseph Marie Jacquard en 1804 al proponer que el control del diseño de un telar se podía hacer mediante tarjetas perforadas, partiendo de los diseños de precursores como Vaucanson y su pato mecánico. Aunque se tratara todavía de controlar una máquina con un objetivo específico concreto, la patente de Jacquard mostró que una secuencia externa codificada podía regular el comportamiento de un dispositivo. La idea fue tan fructífera que los primeros ordenadores se basaron para su programación en tarjetas perforadas, donde cada una representaba una línea de código. El método se siguió utilizando hasta la década de 1980, donde fueron sustituidas por los actuales teclados.

Pero volvamos al siglo XIX. El primero en plantearse la posibilidad de una máquina general, al menos para la resolución de problemas matemáticos, fue Charles Babbage, un excéntrico caballero inglés que bien podría haber sido un personaje de novela de Julio Verne. En 1836 diseñó la máquina analítica, el primer computador mecánico de propósito general, que debía ser controlado por tarjetas perforadas inspiradas en las de Jacquard. Por desgracia, la máquina nunca llegó a construirse debido a las limitaciones técnicas de la época. De todas formas, pensando en la máquina propuesta por su amigo Babbage, Ada Byron, condesa de Lovelace e hija de lord Byron, planteó que quizá aquella máquina pudiera servir para algo más que hacer operaciones matemáticas. Anticipando

futuras aplicaciones de lo que hoy en día calificaríamos como inteligencia artificial, Ada sugirió que, por ejemplo, si se pudieran representar adecuadamente las notas musicales y los mecanismos de composición “la máquina podría componer piezas de música elaboradas”. Sin embargo, al mismo tiempo, esta visionaria también intuyó algunos de los puntos cruciales del debate actual al escribir que la máquina “no pretende originar nada [...] solo hace lo que sabemos ordenarle”, un argumento muy utilizado para distinguir entre la creatividad y el mero procesamiento de símbolos en lo que se denomina todavía hoy como la objeción de lady Lovelace. ¡Cómo les habría gustado a Charles y a Ada ver hasta dónde han llegado sus ideas!

En los años posteriores hubo algunos avances que merecen ser mencionados, como la idea de Herman Hollerith de utilizar tarjetas perforadas para tabular el censo de Estados Unidos de 1890. Ya a principios del siglo XX, el ingeniero e inventor español Leonardo Torres Quevedo presentó el Ajedrecista, el primer autómatas capaz de jugar al ajedrez sin intervención humana. Por supuesto, no era un juego de ajedrez general, sino el caso particular de dar jaque mate al rey negro con solo el rey blanco y una torre. Pero, a diferencia de famosos fraudes históricos, como el Turco mecánico de 1770, que realmente contenía un ser humano escondido en el interior, la máquina de Quevedo, posteriormente mejorada en su aspecto por su hijo Gonzalo, funcionaba a la perfección. Era totalmente autónoma y calculaba las jugadas y movía las piezas por sí sola, lo que la convierte en el primer “videojuego” (analógico) y uno de los hitos fundacionales de la automática y la IA, a pesar de que el algoritmo, una variante del método de la caja que usan los jugadores de ajedrez en esta finalización, era muy sencillo y estaría lejos de las tareas complejas típicamente humanas que pedimos que resuelva hoy en día un método para ser denominado de inteligencia artificial.

Pese a estos avances, ni las ideas teóricas ni la tecnología estaban todavía suficientemente maduras y hubo que esperar unos años más, hasta alrededor de la Segunda Guerra Mundial, para que estas ideas pudieran plasmarse en una máquina a la que pudiéramos llamar ordenador en un sentido moderno.

3. ¿Quién fue Alan Turing?

La primera descripción conceptual de un ordenador se debe al matemático inglés Alan Turing, quien en 1936 escribió un artículo donde presentaba una “máquina ideal”, la conocida como máquina de Turing. La máquina usaba como almacenamiento una cinta dividida en casillas. Una cabeza situada sobre la cinta leía y escribía símbolos siguiendo un conjunto de instrucciones sencillas, cosas del tipo “si en la casilla debajo de la cabeza lectora hay un 0, escribe un 1 y mueve la cinta a la derecha”. Aunque parezca un mecanismo sencillo en extremo, no se ha encontrado nada que pueda hacer un ordenador que no se pueda reproducir en esta máquina ideal, lo que se conoce como tesis Church-Turing (Alonzo Church fue otro matemático que, a la vez que Turing propuso otra forma diferente pero equivalente de describir lo que podía hacer un ordenador).

La máquina de Turing no fue concebida para llevarse a la realidad ni para servir de inspiración a un futuro ordenador real. Es un modelo matemático diseñado para explorar las posibilidades y limitaciones de la programación, cosa que hace perfectamente. Esto permitió, por ejemplo, fijar el concepto de algoritmo como cualquier programa que pudiera ser ejecutado en esta sencilla máquina ideal. La potencia del artefacto teórico se aprecia en el uso magistral que hizo Turing de él, por ejemplo, demostrando que ciertas funciones matemáticas no pueden ser programadas, las llamadas funciones no computables. No es de extrañar, por tanto, que Turing sea considerado el padre de la informática teórica, y que el premio más importante en esta área, el equivalente al Nobel de la informática, es el Premio Turing.

Turing no solo influyó en el desarrollo teórico de la informática, sino que participó de forma señalada en el desarrollo de máquinas reales que se pueden considerar los antecesores de los primeros ordenadores. En efecto, durante la Segunda Guerra Mundial se unió al centro británico para el descifrado de códigos con el propósito de decodificar los mensajes que producía la máquina alemana Enigma, que permitía al alto mando del ejército de Hitler comunicarse con sus submarinos de forma segura encriptando mensajes. Para decodificar estos mensajes, Turing colaboró en el diseño de la máquina conocida como

Bomba, un antecesor del ordenador que simulaba posibles configuraciones de la máquina alemana hasta acertar con las que producían los mensajes cifrados, lo que finalmente se logró, aunque ya avanzada la guerra. Para muchos expertos lograr decodificar estos mensajes fue un punto clave en la victoria aliada, ya que antes de conseguirlo el ejército nazi hundía los barcos de la armada británica más rápido de lo que podían ser contruidos, abocando a Gran Bretaña a una posible rendición.

Por si no fuera suficiente todo lo reseñado, Turing también dio los primeros pasos en relación con nuestro tema concreto: la IA. En su artículo de 1950 “Maquinaria computacional e inteligencia” plantea la pregunta “¿pueden pensar las máquinas?”, lo que da lugar al famoso test de Turing que veremos más adelante.

En el mismo artículo, Turing también planteó el problema del aprendizaje de una forma muy visionaria. A la objeción de lady Lovelace de que el ordenador solo manipula símbolos siguiendo unas reglas, contesta que las consecuencias de esa manipulación pueden ser difíciles de prever por el programador; por eso, las máquinas pueden sorprendernos. En este sentido, proponía que en lugar de reproducir un cerebro adulto de forma artificial sería más lógico intentar imitar el cerebro de un niño, que luego aprendería mediante un sistema de recompensas y castigos. Como veremos, esto anticipa la idea del aprendizaje automático, donde se parte de un sistema “en blanco” que va aprendiendo a partir de datos, mientras que el caso particular de las recompensas y castigos corresponde con bastante precisión con lo que hoy en día conocemos mediante el término de aprendizaje por refuerzo.

Por desgracia, Alan Turing no pudo ver los desarrollos posteriores de la inteligencia artificial porque se quitó la vida en 1954, tras haber sido condenado dos años antes por homosexualidad a castración química (tuvo que elegir entre esta opción o la cárcel), lo que le dejó minado física y moralmente, además de señalado en la sociedad de la época. Ha habido mucho debate sobre la influencia que tuvo en su condena el que fuera conocido debido a su trabajo en la Segunda Guerra Mundial y haber estado al tanto de secretos militares. Quizás, si en lugar de ayudar a su país a ganar la guerra se hubiera mantenido como un oscuro profesor de matemáticas, su juicio ni siquiera habría tenido lugar.

En 2013 recibió un indulto real de forma póstuma y desde 2021 su retrato figura en el billete de 50 libras como recuerdo de su legado y de la injusticia que sufrió.

4. Las primeras décadas de la inteligencia artificial

Tal y como hemos visto, el sueño de máquinas “construidas y no nacidas” acompaña al hombre desde tiempos muy remotos. No es por tanto extraño que nada más aparecer los primeros ordenadores, en la década de los cuarenta del siglo pasado, se dispararan las expectativas.

El pistoletazo de salida oficial para la inteligencia artificial moderna se produjo en 1956, en la Conferencia de Dartmouth, que tuvo lugar en la universidad Dartmouth College (New Hampshire, Estados Unidos), organizada inicialmente por John McCarthy, un joven profesor universitario que, como ya hemos anticipado, introdujo el término de inteligencia artificial. Además de McCarthy, entre los organizadores se encontraban Marvin Minsky, quien crearía el primer laboratorio de IA en el Massachusetts Institute of Technology; Nathaniel Rochester, arquitecto jefe del diseño del IBM 701, el primer ordenador producido masivamente, y Claude Shannon, padre de la teoría de la información y creador del concepto de bit. Además de estos cuatro organizadores, el evento recibió visitas de otros científicos interesados en el nuevo campo, como Herbert A. Simon, futuro ganador del Premio Turing y del Nobel de Economía.

Aunque de este encuentro surgieron numerosas ideas, pronto el optimismo inicial se encontró con una realidad más compleja de lo que se esperaba. Por ejemplo, Simon predijo que un programa de ajedrez derrotaría al campeón mundial en diez años, pero se tardaron cerca de 40 años hasta que el ordenador de IBM Deep Blue derrotó a Garry Kasparov.

En ese mismo año, 1957, Frank Rosenblatt presentaba el perceptrón, un modelo matemático fácilmente programable que simulaba una sencilla neurona humana. El perceptrón de Rosenblatt era una función que recibía valores de entrada, los ponderaba multiplicándolos por constantes llamadas pesos, sumaba esos valores y, si el resultado final sobrepasaba cierto umbral, se activaba, es decir devolvía un 1, mientras que si no se llegaba al valor umbral se devolvía un 0. Tanto los pesos como el valor umbral se “aprendían” para cada problema concreto a partir de datos de entrada y salidas esperadas.

Veámoslo con un ejemplo sencillo. Supongamos que tenemos un sistema en el que tenemos dos interruptores A y B, y queremos que una bombilla C se encienda cuando cualquiera de los interruptores, o ambos, estén en la posición de encendido. Representamos la posición de encendido por 1 y la de apagado por 0, y lo mismo para la bombilla:

<u>ABC</u>
<u>0 0 0</u>
<u>0 1 1</u>
<u>1 0 1</u>
<u>1 1 1</u>

Por ejemplo, la primera fila indica que los dos interruptores están apagados, y por tanto también lo debe estar la bombilla; la segunda fila, que A está apagado y B encendido, y por tanto la bombilla C estará encendida, y lo mismo para las otras dos filas.

A partir de esta tabla, el perceptrón de Rosenblatt era capaz de “aprender” utilizando un algoritmo sencillo que la neurona debía representar la sencilla ecuación: $2 \cdot A + 2 \cdot B > 1$. Aquí los pesos son 2 y 2, y el umbral es 1. Veamos cómo funciona: por ejemplo, en la primera fila tenemos las dos bombillas apagadas. Así, $A = 0$ y $B = 0$, y el perceptrón calculará que $2 \cdot 0 + 2 \cdot 0 = 0$, y como 0 no es mayor o igual a 1, la salida será 0, es decir, la bombilla C quedará apagada. En cambio, en la fila 1 tenemos un interruptor, el B, encendido, y el otro apagado, es decir $A = 0$ y $B = 1$. En este caso $2 \cdot 0 + 2 \cdot 1 = 2$, y al superarse el umbral ¡la bombilla se enciende! Invitamos al lector a comprobar que igual pasa en los otros dos casos.

Nuestra neurona/perceptrón, que, como vemos, es solo una sencilla ecuación, resuelve el problema para la tabla concreta que mostramos más arriba, pero ¿existe un perceptrón adecuado para cada tabla posible? Consideremos la siguiente variante:

<u>ABC</u>
<u>0 0 0</u>
<u>0 1 1</u>
<u>1 0 1</u>
<u>1 1 0</u>

En este caso, queremos que la bombilla C se encienda si hay exactamente un único interruptor encendido. Por ejemplo, en la primera fila no hay interruptores encendidos y por eso la bombilla C está apagada. Las filas dos y tres sí cumplen que hay un único interruptor encendido (B en la fila 2, A en la tres), por lo que la bombilla debe encenderse. Finalmente, en la última fila la bombilla debe estar apagada, porque no hay uno, sino dos interruptores encendidos. El perceptrón de antes no nos vale, porque en este último caso, con ambos interruptores encendidos, tendríamos $2 \cdot 1 + 2 \cdot 1 = 4$, y se superaría el umbral 1, por lo que la bombilla C se encendería, lo que no debe suceder. Quizás podríamos poner un umbral más alto, digamos $2 \cdot A + 2 \cdot B > 5$, pero si probamos veremos que con este nuevo perceptrón nos sale que la bombilla no se enciende nunca. Otra posibilidad es cambiar los pesos, por ejemplo $3 \cdot A + 4 \cdot B > 5$, por poner algo..., pero veremos que tampoco funciona.

Podemos probar una y otra vez, combinando pesos y umbrales como si de un pasatiempo se tratara, buscando el perceptrón que represente esta segunda tabla, pero no lo encontraremos, siempre fallará en alguna fila. En 1969, el ya citado participante de la reunión de Dartmouth Marvin Minsky demostró, junto con Seymour Papert, que ningún perceptrón podía resolver este problema ni otros similares, lo que desanimó a la mayoría de los que trabajaban en el área de las neuronas artificiales, que abandonaron este campo: pasaron del sueño de poder crear máquinas pensantes a no ser capaces de resolver un sencillo problema con tres bombillas. En ese momento, empezó lo que se ha llamado el invierno de la inteligencia artificial (el primer invierno, porque ha habido varios). Para colmo, un informe escrito en 1973, el llamado informe Lighthill por el apellido de su autor, indicaba que la IA no había cumplido ninguna de las expectativas prometidas, lo que fue seguido por la retirada de la mayor parte de los fondos destinados a esta área de las universidades británicas.

Pese a este revés en el campo de las futuras redes neuronales, que no se superaría hasta décadas más tarde con la llegada de las modernas técnicas basadas en aprendizaje profundo, unos pocos investigadores siguieron, sin rendirse, explorando otras posibilidades. Ya en 1964 Joseph Weizenbaum había creado uno de los primeros programas conversacionales, ELIZA. El programa admitía diferentes configuraciones, adoptando así distintos papeles como interlocutor de un humano. En una de estas configuraciones simulaba ser un psicólogo “rogeriano”, un enfoque en el que el terapeuta hace preguntas generales y

abiertas. Era una versión muy básica que buscaba patrones sencillos de lenguaje. Por ejemplo, si se había mencionado “padre” en una frase anterior, el programa podía contestar al humano: “¿Cree que esto tiene que ver con lo que me ha contado sobre su padre?”. Por supuesto, la máquina ni siquiera había identificado qué se decía en concreto acerca del padre ni había analizado la última frase del humano, simplemente escribía esto de forma aleatoria. Lo curioso es que el programa funcionaba. Mucha gente sentía que estaba ante un ser pensante y, al parecer, había empleados en los centros de trabajo que esperaban a que se fueran sus compañeros para contarle sus problemas a la computadora (recordemos que entonces era impensable que alguien pudiera tener un ordenador en casa). El autor del programa se desesperó explicando que era solo un programa sencillo, que no había que tomarlo tan en serio, pero fue en vano. Es por esto por lo que se menciona ELIZA como uno de los primeros candidatos a pasar el test de Turing, del que hablaremos más adelante.

A finales de la década de los sesenta también se empezó a diseñar el primer robot autónomo, Shakey, capaz de moverse de una habitación a otra, encender y apagar luces, abrir y cerrar puertas, etc. Aun en un estado muy preliminar y creado como un proyecto de investigación, Shakey sentó las bases para numerosos desarrollos futuros en el campo de la robótica.

Los años setenta y ochenta del siglo XX fueron tiempo de buscar alternativas más elaboradas para el perceptrón. Es la época del auge de los sistemas expertos: programas basados en reglas, a menudo introducidas por humanos, orientadas a resolver problemas específicos. Algunos, como MYCIN, desarrollado en la universidad de Stanford para ayudar a diagnosticar infecciones sanguíneas, tuvieron notable éxito. Sin embargo, a pesar de la gran cantidad de dinero invertido, pronto se vio que estos sistemas también tenían limitaciones. Ya hemos hablado anteriormente de lo complicado que resulta, por ejemplo, describir mediante reglas la escritura realizada por humanos. Este problema se extendía a muchos otros campos donde, por más reglas que se añadieran, no se obtenía un resultado satisfactorio, porque llegaba un momento en el que incluso las reglas de los expertos empezaban a contradecirse entre sí. Esto llevó, una vez más, a un freno en las inversiones en el campo de la inteligencia artificial. Un “segundo invierno” que alcanzaría, más o menos, hasta la época de internet, donde el acceso a gran cantidad de datos y el aumento de la potencia de cómputo dieron impulso a un renacimiento de las redes neuronales y a los éxitos actuales.

5. ¿Qué es el test de Turing?

Ya hemos visto que Alan Turing fue uno de los “padres” de la informática, tanto desde un punto de vista teórico, con la definición de la máquina que representa las posibilidades y limitaciones de cualquier ordenador, como práctico, con sus desarrollos durante la Segunda Guerra Mundial. También hemos mencionado que es el precursor de los debates sobre la inteligencia artificial con su artículo de 1950 “Maquinaria computacional e inteligencia” donde plantea la pregunta “¿pueden pensar las máquinas?”. Entremos ahora en detalles, porque más allá de su innegable importancia histórica, su propuesta sigue siendo muy actual.

Para Turing la pregunta “¿pueden pensar las máquinas?” está mal definida, porque incluye términos ambiguos como “pensar”. Por eso propone como alternativa un “juego de imitación” (The Imitation Game es justo el título del biopic sobre Turing realizado en 2014 con Benedict Cumberbatch y Keira Knightley como protagonistas). En la variante más conocida de este juego, un interrogador se sitúa frente a dos puertas. Detrás de una de ellas hay un ordenador que pretenderá hacerse pasar por un humano, detrás de la otra se oculta una persona. La labor del interrogador es determinar dónde está el ordenador y dónde la persona, lo que debe averiguar a partir de las respuestas que ambos van proporcionando a las preguntas que el interrogador les plantee.

Si tras un tiempo prefijado, digamos cinco minutos, el interrogador se equivoca aproximadamente la mitad de las veces (incluso menos, para Turing, bastaría un 30% de errores al señalar la máquina), es decir, si máquina y humano son difíciles de distinguir, no tendremos más remedio que decir que el ordenador “piensa” y se le puede aplicar la etiqueta de “inteligente”, o al menos eso es lo que opina Turing.

Estamos, de nuevo, como en nuestra definición de inteligencia artificial, ante una propuesta que será tachada de funcionalista, operacional o conductista. Para muchos el test de Turing es simple apariencia, simulación, y no merece recibir el nombre de inteligencia al carecer de conciencia. Pero Turing, como buen científico, opinaba que hay que remitirse a los resultados experimentales y por eso ponía la carga de la prueba en lo que la máquina hace y no en su esencia,

conciencia u otras propiedades que él consideraba metafísicas.

Entre los detractores de este punto de vista podemos citar al eminente neurólogo Miguel Nicoletis, que afirma que “todos los chips del mundo serían incapaces de crear una conciencia”. Nicoletis representa la postura biológica, que considera la inteligencia como un fenómeno emergente del cerebro, algo que se ha dado en ciertos seres vivos tras millones de años de evolución. La inteligencia artificial no sería, para Nicoletis, inteligencia, y ni siquiera artificial, porque ha sido programada por el hombre. En la misma línea, autores como el eminente físico Roger Penrose indican que la mente humana no es algorítmica; por tanto, es capaz de tratar problemas que un ordenador no puede resolver. Penrose explica esta característica por las particularidades de nuestro cerebro donde intervienen procesos más complejos que en un ordenador digital.

Con el objetivo de ejemplificar la diferencia entre apariencia e inteligencia representada por el test de Turing y la inteligencia real, el filósofo John Searle publicó en 1980 un artículo en el que describe un experimento mental conocido como el argumento de la habitación china. En este experimento, Searle imagina a una persona, que no conoce el idioma chino, en una habitación donde un libro le permite transformar unos caracteres chinos en otros, por ejemplo, puede buscar preguntas en chino y contestar con la respuesta que le indique el libro, o incluso formar al azar distintas respuestas con sentido mediante el uso de patrones reconocibles en la estructura de las preguntas. De esta forma, un nativo chino le pasa preguntas por una rendija y él devuelve las respuestas, pudiendo llegar a “engañar” al nativo haciéndole pensar que la persona que está dentro sabe chino. Sin embargo, Searle enfatiza en que se trata de mera apariencia, la habitación como mecanismo no sabe chino, y la persona de dentro tampoco, simplemente está usando reglas sintácticas (manipulación de símbolos, es decir, lo mismo que hace un programa) sin tener idea de la semántica (significado). Searle añade como prueba adicional que una vez acabado el experimento, la persona de dentro de la habitación no podría decirnos acerca de qué ha versado su conversación en chino.

Este ataque al funcionalismo, y en particular al test de Turing, lleva a Searle a decir que no hay inteligencia sin comprensión o intencionalidad, algo de lo que carecen los ordenadores. Por tanto, para Searle, como para Nicoletis, la IA actual no sería inteligencia.

En el lado contrario hay quien opina que el test es demasiado exigente para la

máquina. Por ejemplo, se observa a menudo que ningún humano pasaría el test contrario: si situamos a un ordenador “interrogador” frente a dos puertas tras una de las cuales hay un ordenador y en otra un humano que pretende ser un ordenador, el interrogador tardaría pocos segundos en determinar quién es quién, por el sencillo procedimiento de pedirles a ambos que multiplicaran, por ejemplo, dos números de 100 dígitos cada uno en menos de tres segundos. Por supuesto, este argumento es bastante débil y no muestra que la prueba sea “injusta” con los ordenadores, sino que pone de relieve algo ya conocido: humanos y máquinas poseen fortalezas muy distintas. Un argumento más profundo fue señalado por el propio Turing: el test sería para él una prueba suficiente pero no necesaria de inteligencia, porque podría haber otras formas de inteligencia que no correspondieran a la inteligencia humana.

Un ejemplo que se suele poner a este respecto es el del vuelo artificial, es decir, los aviones. Durante siglos, los hombres intentaron diseñar aparatos que volaran reproduciendo la única forma de vuelo que conocían: los pájaros. Por eso construyeron artefactos que batían las alas como un pájaro, con desastrosos resultados. El éxito vino al encontrar una forma de vuelo alternativa. En efecto, los aviones de hoy en día no baten las alas, sino que las llevan totalmente rígidas, es una forma diferente de volar. Quizás suceda lo mismo con la inteligencia artificial y en estos primeros pasos estemos intentando imitar la única inteligencia que reconocemos, la humana, simulando el cerebro de forma tosca mediante redes neuronales, cuando puede que haya otro camino, todavía por descubrir, basado en formas alternativas de generar algo que aun así podamos reconocer como “inteligencia”.

Para concluir este apartado, señalemos que, en marzo de 2025, 75 años después de que Turing propusiera su prueba, el test de Turing fue claramente superado por una máquina. El experimento, en el formato que hemos explicado, utilizó Chat GPT-4.5 como parte “no humana”. En el test el programa fue elegido el 73% de las veces, es decir, no solo superó el test de Turing, sino que fue considerado por el interrogador como el humano casi en tres de cada cuatro experimentos.

6. Un minitest de Turing: los CAPTCHA

Aparte del interés teórico y filosófico, el test de Turing tiene aplicaciones prácticas con las que seguro que no soñó su creador. Es el caso de los CAPTCHA, esos programitas que nos obligan a marcar una casilla para comprobar si somos humanos o bien resolver un pequeño puzle o señalar imágenes en las que aparecen semáforos, coches o bicicletas. Pues bien, estos (odiosos) prolegómenos para acceder a nuestra aplicación favorita son pequeñas versiones del test de Turing. En efecto, CAPTCHA es un término introducido a principios del siglo actual que corresponde al acrónimo de Completely Automated Public Turing test to tell Computers and Humans Apart, es decir, test de Turing público completamente automatizado para diferenciar ordenadores de humanos.

El propósito de estos minitest no es, por supuesto, determinar si las máquinas piensan, como planteó Turing, sino que es más prosaico: evitar el acceso de bots o programas que interactúan con servicios web u otras aplicaciones como si fueran humanos. Esto se hace por varias razones:

Spam y fraude. Si pueden acceder, por ejemplo, a redes sociales, los bots pueden utilizar este acceso para distribución masiva y automática de publicidad no deseada o spam. También, en servicios que requieren registro de usuarios, los CAPTCHA intentan evitar que los bots accedan a la página de autenticación, donde pueden o bien atacar cuentas existentes intentando determinar las palabras clave por prueba y error, o bien crear multitud de cuentas falsas, que se podrán usar posteriormente de forma anónima, ya sea para spam o para anonimizar otros ataques o realizar fraudes.

Validez en datos recogidos. Evitar que un bot ponga, por ejemplo, cientos de reseñas automáticas sobre restaurantes, que vote miles de veces por un cantante determinado en un concurso, rellene miles de veces una encuesta o, incluso peor, que vote varias veces por el mismo candidato en unas elecciones con voto digital.

Proteger los datos y la propia aplicación. Puede que el dueño de la aplicación o página simplemente no quiera “regalar” sus datos a programas automáticos. Además, el acceso de miles de bots puede llegar a ralentizar el sistema o incluso bloquearlo por sobrecarga, impidiendo la utilización por parte de usuarios humanos. Son los llamados ataques de denegación de servicio, conocidos como ataques DoS por las siglas del término inglés Denial of Service, que tantos problemas han causado a las grandes compañías y organismos estatales.

Además, cuando estamos pasando algunos CAPTCHA, por ejemplo, los típicos de detectar elementos de tráfico, estamos etiquetando imágenes, es decir, asociando las imágenes a términos como “semáforo” o “no hay semáforo”. Son los llamados CAPTCHA versión 2, que aprovechan el mecanismo de identificación de humanos para hacernos trabajar gratis. Disponer de millones de imágenes etiquetadas es fundamental, por ejemplo, para las empresas que desarrollan sistemas de conducción autónoma, que las emplearán para entrenar sus modelos y que así sus vehículos sepan distinguir una bicicleta de un peatón. Conviene avisar de que si leer estas líneas desata nuestra justa indignación y ante un CAPTCHA de este tipo nos decidimos por una actitud combativa marcando los coches como bicicletas y los peatones como camiones, solo lograremos no poder entrar en la aplicación, porque el sistema combina imágenes previamente etiquetadas, las que realmente se usan para permitirnos el paso, junto con otras sin etiquetar. Además, en las imágenes nuevas, aquellas en las que nuestra decisión realmente importa para entrenar futuros modelos, nuestra respuesta es confrontada con la de otros miles de usuarios, por lo que nuestro intento de boicot será ignorado con toda probabilidad.

Los CAPTCHA del tipo más reciente, la versión 3, son totalmente transparentes para el usuario. No se ven, pero están ahí, registrando cómo movemos el ratón y tecleamos al hacer login, por ejemplo. Esos datos se analizan y se comprueba si pueden corresponder a un humano y, en caso contrario, se pide una forma adicional de confirmar la entrada, por ejemplo, mediante un email con código, y en última instancia nos lleva a un CAPTCHA de la versión 2... ¡a detectar semáforos!

7. ¿Qué es el aprendizaje automático?

Como hemos visto, las primeras décadas de la historia de la informática estuvieron llenas de promesas solo parcialmente cumplidas en el campo de la inteligencia artificial. Desde la perspectiva actual podemos señalar algunos factores que influyeron en esta situación.

El perceptrón, tal y como fue diseñado originalmente, era demasiado simple. Para resolver problemas complejos, la función de salida de la neurona no debe limitarse a devolver solo “1” o “0” según si se alcanza cierto umbral, por mucho que estos valores nos gusten a los informáticos. En los problemas reales las neuronas digitales pueden llegar a devolver gran cantidad de valores siguiendo funciones que deben verificar ciertas propiedades matemáticas como la derivabilidad que facilitan el aprendizaje. Además, la potencia de las redes neuronales se logra, como veremos más adelante, únicamente cuando se dispone de varias “capas” de neuronas, lo que a su vez exige métodos de entrenamiento más complejos y una capacidad de cómputo no disponible en la época.

Más allá del problema concreto del perceptrón, pronto se comprobó que el planteamiento inicial de los científicos de este campo había pecado de cierta ingenuidad. En efecto, inicialmente se pensaba que gracias a la velocidad de los ordenadores se podrían resolver muchos problemas hasta entonces insolubles por mera fuerza bruta, es decir, ensayando todas las posibles soluciones hasta encontrar la más adecuada. El conjunto de posibles soluciones, llamado espacio de búsqueda del problema, a menudo es demasiado grande para ser tratado por un humano, pero, se pensaba, una máquina lograría hacerlo rápidamente y sin fallos. Sin embargo, pronto se vio que en muchos problemas interesantes se daba una explosión combinatoria: el número de posibilidades o tamaño del espacio de soluciones, crecía tan rápido que ningún ordenador, presente o futuro, podría llegar a examinarlas todas. Así, Claude Shannon, al que ya mencionamos como participante en el encuentro fundacional de la IA en Dartmouth, calculaba que tras solo diez jugadas el número de posiciones distintas de un tablero de ajedrez excedería los 60 billones (millones de millones) de posiciones. Pero es que con solo una jugada más se pasaba de 60 billones a más de 2.000 billones, y el número continuaba creciendo a un ritmo mucho más rápido de lo que podrían

avanzar la potencia de las máquinas.

A partir de este y otros ejemplos, los primeros padres de la informática se dieron cuenta enseguida de que probar todas las soluciones de un problema no era algo factible en la práctica. Para resolverlo, pensaron en heurísticas, métodos para podar el espacio de búsqueda, esto es, formas de reducir el número de posibilidades a examinar mediante reglas adaptadas a cada problema concreto. Algo así como el hilo de Ariadna que permitió a Teseo evitar la exploración del inmenso laberinto en su viaje de vuelta. Esta técnica tuvo éxito en muchos problemas, como el del ajedrez. En efecto, el programa que derrotó a Garry Kaspárov combinaba gran poder de cómputo y el uso de reglas, algunas diseñadas por programadores, otras aprendidas de grandes maestros, que permitían reducir el número de posibilidades a explorar.

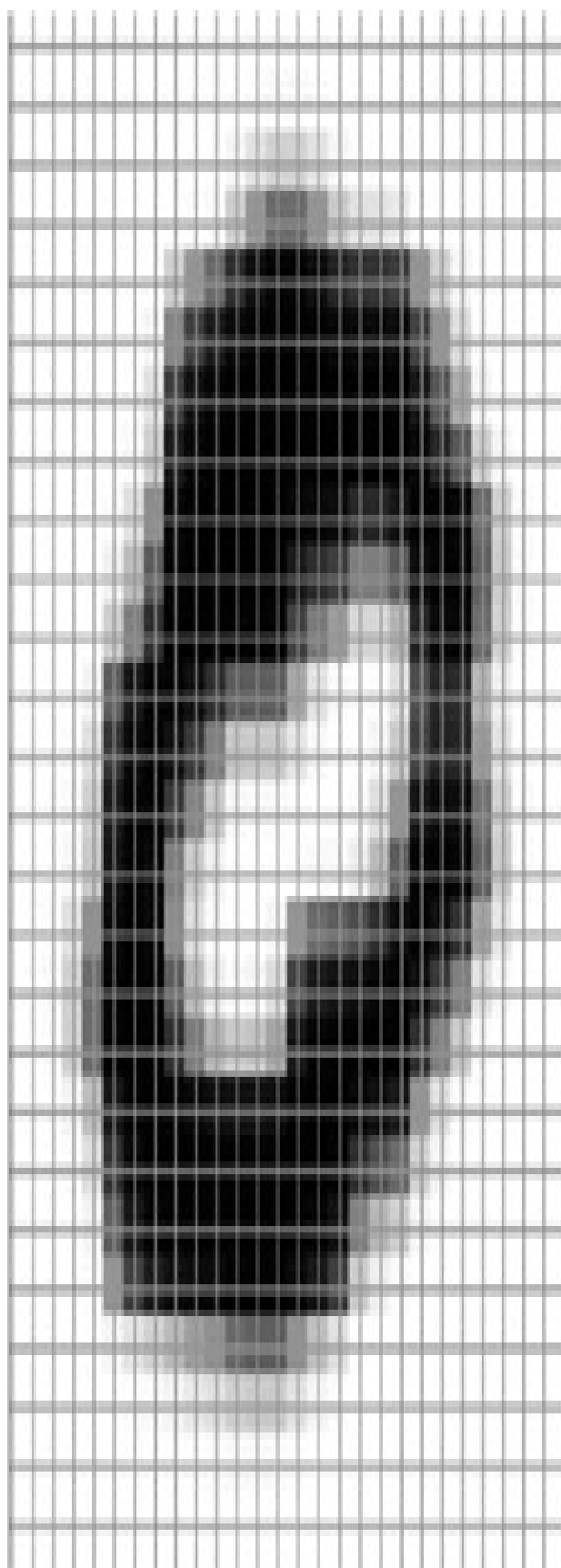
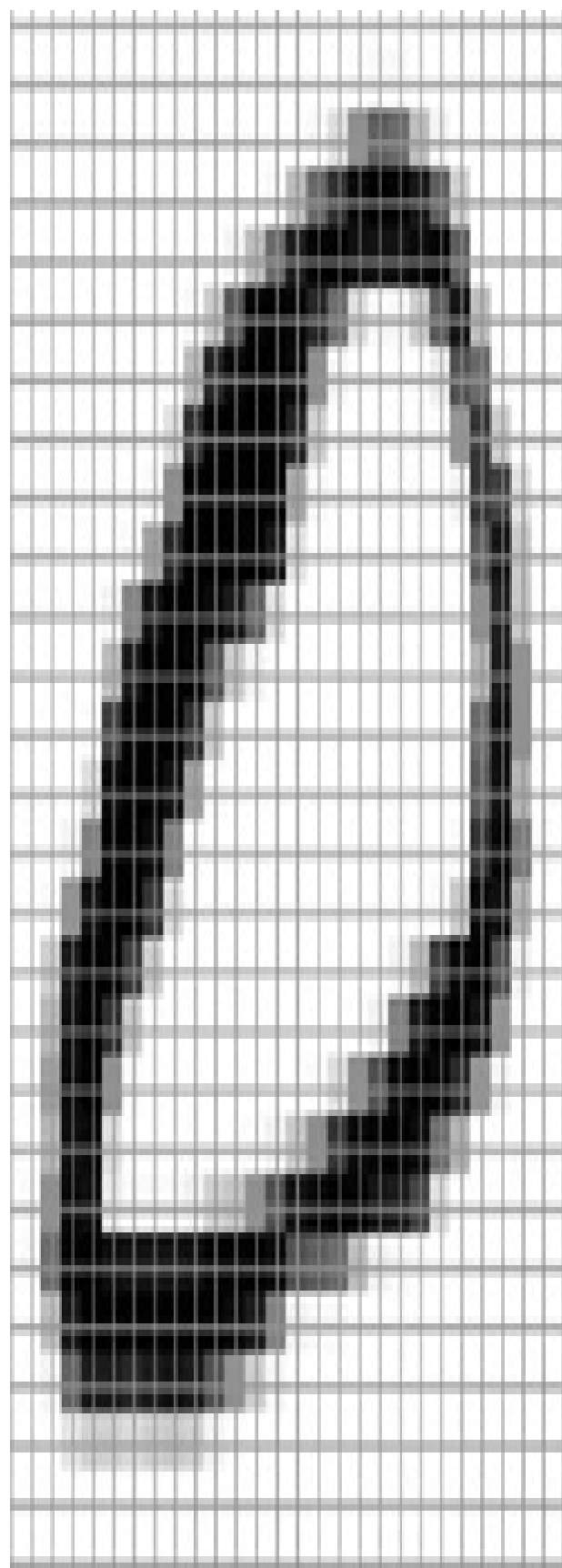
Sin embargo, aun con estas técnicas seguía habiendo problemas que se resistían, como los que hemos mencionado aquí como nuestra definición de IA: “tareas complejas típicamente humanas”, donde no había un conjunto de reglas claras. Así, John R. Pierce decía en 1966 que para reconocer la voz humana un ordenador debía tener la inteligencia y un conocimiento del lenguaje comparables a los de un hablante nativo, algo que no se esperaba que fuera a suceder. Y Pierce decía esto en su papel de presidente del Comité Asesor para el Procesamiento Automático del Lenguaje, por lo que sus palabras tuvieron gran impacto. Además, no era una voz aislada, su opinión era compartida por otros investigadores de campos similares, como el reconocimiento de la escritura humana, que también parecía más allá de las posibilidades de la técnica.

Lo que se necesitaba era un cambio de perspectiva, una nueva forma de abordar estos problemas. Y este nuevo enfoque vino al apartar de la primera línea la búsqueda de reglas para dar preeminencia a los datos. Es lo que llamamos aprendizaje automático.

La idea es la siguiente: si queremos reconocer un texto escrito manualmente, en lugar de intentar definir qué es una letra, “a”, “e”, “i”, “o”, etc., escrita por un humano mediante reglas, debemos partir de muchas letras escritas manualmente. Por ejemplo, aquí tenemos dos letras “o” escritas a mano:

Ejemplos de letras ‘o’ manuscritas procedentes

del conjunto de entrenamiento EMNIST



Fuente: NIST.

Aunque hay distintos tipos de procedimientos, podemos imaginar que el programa va situando las imágenes sobre una rejilla de tamaño fijo, como en la imagen, y “apuntando” qué casillas están en blanco y cuáles quedan pintadas por el dibujo de la letra. Al final, después de recorrer miles de letras “o” (y lo mismo con las demás letras), no se obtendrá ninguna regla, sino una indicación de lo probable que es que ciertos cuadrados estén pintados de negro al dibujar una “o”, lo mismo con una “a”, etc. En sistemas más avanzados no se guardará únicamente información sobre cada casilla individual, sino sobre grupos de casillas y sus relaciones. De esta forma, el sistema aprenderá a generar patrones que identifican cada letra. Acabada esta fase inicial de entrenamiento, tendremos un modelo. El modelo es el programa que, a base de ver y analizar muchas letras, ha aprendido a reconocerlas estadísticamente. Ahora cuando queremos determinar a qué letra corresponde una nueva imagen utilizaremos nuestro modelo, que determinará qué letra de las que ya conoce se parece más a nuestra imagen. No hace falta, como sucedía con las reglas, que la letra nueva coincida con ninguna de las que ya ha visto, el modelo se limitará a ver con cuál de sus letras conocidas encaja mejor, es decir, cuál es la más probable, y esa será su respuesta.

Es más, a partir de los patrones ya aprendidos muchos sistemas podrán generar nuevas letras, letras que parecerán “humanas” pero que no habrá escrito ningún humano. Es lo que llamaremos en un capítulo posterior inteligencia artificial generativa.

Esta idea, con variantes, es en la que está basada toda la IA moderna, desde el reconocimiento de voz de Siri, hasta ChatGPT o la conducción autónoma de Tesla: en el análisis de datos de entrada ya etiquetados para “aprender” a reconocer cada una de las posibilidades y ser capaz de detectar qué etiqueta corresponde a un dato nuevo.

Cuando decimos que un programa que sigue este proceso “aprende”, queremos indicar que cuántos más datos le proporcionemos, mejor funcionará el modelo generado en la tarea de reconocer nuevos datos. En nuestro caso, cuantas más letras escritas manualmente sean utilizadas durante el entrenamiento, más veces

acertará el modelo al reconocer una letra nueva. Es importante subrayar que el programa que genera el modelo es el mismo tanto si hay pocos datos como si hay muchos.

Por tanto, para que estos sistemas funcionen correctamente, necesitan muchos datos, y por eso la época de internet, con miles y miles de datos disponibles, ha sido la gran eclosión de este tipo de inteligencia artificial y ha llevado a los éxitos actuales. Esa es una de las razones detrás del interés de todas las empresas en lograr nuestros datos, para entrenar más y mejores modelos.

Sin embargo, lo que acabamos de explicar es solo un caso particular de aprendizaje automático. Veamos en detalle todas las posibilidades.

8. Aprendizaje supervisado

Dentro del aprendizaje automático se suelen distinguir tres tipos: aprendizaje supervisado, no supervisado y por refuerzo. Vamos a comenzar con el primero de ellos. El aprendizaje supervisado requiere que haya un dato específico que aprender y predecir. Es el caso de la escritura humana: en la fase de aprendizaje le proporcionamos al sistema multitud de imágenes de letras escritas por distintas personas, pero, y esto es fundamental, son datos etiquetados, es decir, además de cada imagen le indicamos al sistema qué letra representa la imagen para que aprenda a relacionar ambas cosas.

Veamos otro ejemplo. Supongamos que un banco intenta decidir qué transacciones de comercio electrónico son posibles fraudes, quizás por robo de contraseñas o tarjeta, para, en esos casos, pedir una confirmación extra con clave o huella dactilar.

Como el banco tiene un histórico de situaciones que han sido fraude y otras que no, dispone de una tabla como la siguiente:

TOTAL HORA TOTAL NAC.TOTAL INT.¿ES INT.?ANTIGÜEDADFRAUDE						
870,5	2	1	0	1	2	1
45	14	7	0	0	18	0
199,9	23	3	2	1	36	0
12	9	4	0	0	12	0
640	21	10	0	1	4	1
25	18	2	0	0	6	0
499	3	0	0	1	1	1

La primera columna (Total) corresponde al importe de la transacción. A continuación, vemos la hora a la que se ha producido la operación, el número de transacciones que ha realizado esta persona en el último mes, tanto nacionales como internacionales (Total nac. y Total int., respectivamente). La columna ¿Es int.? indica con un valor 1 que la transacción es internacional y con 0 que es nacional. También disponemos de la antigüedad en meses de la tarjeta seleccionada y, finalmente, si hubo fraude o no, representando con 1 si fue una transacción fraudulenta y con 0 en caso contrario.

El objetivo del sistema de aprendizaje será aprender a reconocer posibles fraudes distinguiendo las características de las transacciones fraudulentas. Por ejemplo, es muy probable que el sistema deduzca que si el usuario no realiza casi nunca transacciones internacionales y la transacción en cuestión es internacional, la probabilidad de que se trate de un fraude será muy alta. El resultado de esta fase de entrenamiento será un modelo, es decir, un programa capaz de predecir si una transacción nueva que está ocurriendo en este instante tiene riesgo de constituir un fraude. Dependiendo del método utilizado es muy probable que no solo obtengamos un aviso de si puede ser un fraude, sino que también obtengamos una estimación de la probabilidad de que lo sea.

Por ejemplo, si le proporcionamos al modelo los datos de la siguiente transacción:

TOTAL	HORA	TOTAL NAC.	TOTAL INT.	¿ES INT.?	ANTIGÜEDAD
1.470	2	0	0	1	18

Puede ser que nos conteste, por ejemplo, que esta transacción tiene un 80% de posibilidades de ser fraudulenta, lo que seguramente hará que el sistema de seguridad del banco requiera una identificación adicional, o incluso que sea denegada.

Como vemos, y aunque sea de forma un tanto informal, podemos decir que en el aprendizaje automático supervisado partimos de datos dispuestos en forma de tabla (podemos pensar en una hoja de cálculo). En esta tabla hay una columna especial, la columna que queremos predecir, que a menudo llamamos etiqueta, mientras que al resto de las columnas se las conoce por el nombre de características o atributos. La etiqueta en nuestro ejemplo anterior sería la columna Fraude, y las características el resto.

En el ejemplo de aprender a reconocer los caracteres escritos por humanos, podríamos pensar que no tenemos esta disposición en forma de tabla, pero realmente es fácil convertir el problema a este formato por el método que ya explicamos anteriormente: cada carácter se coloca sobre una rejilla de un tamaño dado, digamos de 50×50 casillas, y suponemos que cada casilla tiene un 1 si contiene parte de la grafía del carácter y un valor 0 si está en blanco. Ahora ponemos todas las filas de la rejilla consecutivamente, hasta formar una hilera de $50 \cdot 50 = 2.500$ valores, que serán las columnas o atributos. Por ejemplo, la columna 51 de nuestra tabla será la primera casilla de la segunda fila, dado que la primera fila ocupa las 50 primeras columnas. Finalmente, la etiqueta sería el valor asociado a la figura, por ejemplo “a” si se trata de esa letra (en la práctica será un número representando la letra, pero eso no afecta a esta explicación). Tenemos, por tanto, una tabla de 2.501 columnas, donde las 2.500 primeras contienen los datos de la imagen y la última, el carácter que representa. Si además tenemos, por ejemplo, 40.000 imágenes, cada una correspondiendo a un carácter, tendremos 40.000 filas, es decir, $40.000 \cdot 2.501$, o aproximadamente 100 millones de valores. Como vemos, los números que obtenemos son enormes, pero es que incluso para un problema no demasiado complejo como es reconocer las características de la escritura humana necesitaremos muchos ejemplos para que el sistema pueda “aprender”. Disponer de 40.000 imágenes puede parecer excesivo, pero no lo es en absoluto si tenemos un sistema con 28 letras más 10 dígitos (38 caracteres diferentes en total) serían $40.000 \div 38$, es decir, alrededor de 1.000 ejemplos por carácter, cuando los sistemas actuales, para alcanzar precisiones superiores al 99,5% de aciertos, requieren de varios miles de

imágenes por carácter, es decir, en todo caso hemos pecado de conservadores.

Por supuesto, disponer de tantas imágenes etiquetadas es un problema, sobre todo por la obtención de etiquetas fiables, que en ocasiones exigen mucho trabajo manual tedioso. En algunos problemas, como en el que hemos visto del fraude, el trabajo manual se ha hecho durante un largo periodo anterior, en el que el banco ha ido apuntado los casos de fraude detectados, pero en otros casos las etiquetas serán más difíciles de obtener. Esto nos da una pista de la importancia de los datos en la nueva era de la IA.

9. Aprendizaje no supervisado

Hemos visto que el aprendizaje supervisado parte de datos en forma de tabla con una columna especial, la etiqueta que queremos predecir o generar, y su objetivo es aprender a deducir el contenido de esa columna en cada fila a partir del resto de columnas.

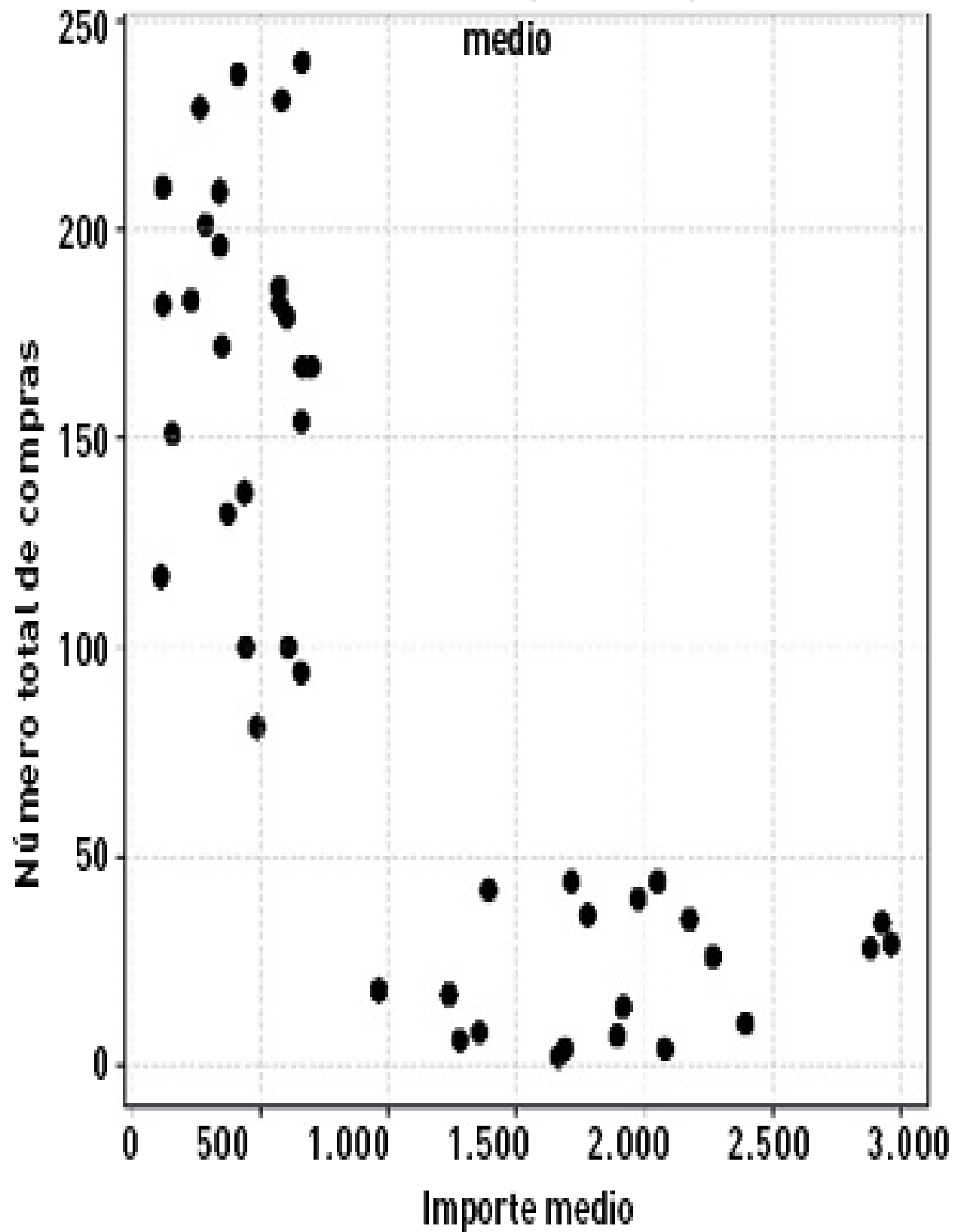
¿Qué ocurre cuando no existe tal columna? En ese caso estamos en lo que se llama aprendizaje no supervisado. Con este término se recogen multitud de técnicas, que pueden implicar desde la selección de las columnas que más información contienen, la creación de columnas nuevas que combinan adecuadamente otras o la agrupación de filas, también conocida como segmentación. Por simplicidad, en este capítulo vamos a centrarnos en un ejemplo de este último tipo de aprendizaje no supervisado: la segmentación, también conocida por el término en inglés clustering.

Pensemos, por ejemplo, que tenemos una tabla de clientes con los datos del año en curso, donde cada fila representa las compras que ha realizado ese cliente en un año seguidas del importe medio en euros de esas compras. Serían datos de este estilo:

NÚMERO TOTAL DE COMPRAS	IMPORTE MEDIO
202	158,45
192	269,21
114	287,50
...	...

En este caso no tenemos columna que predecir, simplemente queremos ver si podemos segmentar nuestros clientes, es decir, si estos se pueden separar en grupos con características afines. Para ello, en este caso sencillo, podemos representar gráficamente las dos columnas:

Cientes: n° de compras vs. importe



Fuente: Elaboración propia.

Cada punto representaría una fila de nuestra tabla, donde las dos columnas se han convertido en las coordenadas que identifican los datos de compra de ese cliente. Según vemos en el gráfico, parece haber dos grupos de clientes. Un primer grupo (puntos de abajo a la derecha) formado por clientes que hacen compras con un importe medio bastante elevado pero que realizan pocas compras anuales, en general menos de 50 al año. El segundo grupo (arriba y la izquierda del gráfico) está constituido por aquellos clientes que hacen compras por importe más pequeño, pero que realizan muchas compras. Esta segmentación o división en tipos de clientes nos pueden permitir realizar un tratamiento más especializado, por ejemplo, puede que nos interese ofrecer un seguro a los que hacen compras más altas o descuentos por el número de compras a los que compran más a menudo pero cantidades menores.

Por supuesto, en un caso real rara vez vamos a tener una división tan clara, y puede que en lugar de dos tengamos tres o más grupos, pero en todo caso el ejemplo nos sirve para ilustrar el concepto.

En todo caso, ¿qué papel juega la IA aquí? Hemos situado a los clientes en un gráfico y hemos comprobado que pueden dividirse aproximadamente entre dos grupos. No nos han hecho falta técnicas avanzadas ni conceptos complicados.

El papel de la IA lo vemos mejor si pensamos que en general no tenemos solo dos columnas. Nuestra tabla puede ser, por ejemplo, la siguiente:

Nº TOTAL	COMPRAS	IMPORTE MEDIO	RATIO DEVOLUCIÓN	Nº COMUNI
202	158,45	5%	15	
192	269,21	7,3%	145	
114	287,50	1,1%	3	
...	...	...	...	

Ahora, además de la cantidad de compras y el importe medio, tenemos el porcentaje de devoluciones y el número de comunicaciones (llamadas, correos) que ha habido por parte del cliente. Estos datos pueden influir en nuestra segmentación, puede tal vez que haya un grupo de clientes que realizan muchas compras de bajo importe, pero realizan muchas devoluciones y envían muchas comunicaciones..., hay muchas posibilidades. El problema es que ahora no podemos representar la tabla gráficamente de forma tan sencilla, porque cada dato sería un punto ¡en un espacio de cuatro dimensiones!

Visualizar un punto en un espacio de cuatro dimensiones es un problema para los humanos, que estamos limitados a representaciones en un máximo de tres dimensiones, pero para un algoritmo de IA de segmentación esto no supone ningún problema, le da igual que sean dos dimensiones, que cuatro, que 50..., el procedimiento es el mismo, representar cada fila (cliente) como un punto en ese espacio de tantas dimensiones como columnas y realizar las operaciones matemáticas que permitan dividir el conjunto de clientes en grupos compactos con características diferenciadas.

Por supuesto, los humanos todavía tenemos que evaluar los resultados, como siempre en aprendizaje automático. En este caso se trata de interpretar los resultados y ver si se pueden aplicar a nuestro problema: cuántos clientes quedan en cada grupo. ¿Podemos distinguir las características de un grupo de las características del resto? Para ayudarnos a esto último, algunos métodos de IA, tras generar el correspondiente modelo, nos darán los puntos medios (llamados centroides) de cada grupo, representando un cliente tipo que estaría en el centro del grupo en su representación geométrica. Normalmente, se tratará de un cliente que no existe, es decir, no es una fila de las de nuestro conjunto de datos, sino de un cliente inventado que representa a un hipotético cliente promedio. Lo que nos está diciendo el método es “el mejor representante de este primer segmento sería un cliente con estas características”.

Las utilidades de la segmentación van mucho más allá de la aplicación al caso de los clientes: desde la clasificación automática de los miles de imágenes obtenidas por los grandes telescopios a la agrupación de películas o música en plataformas de entretenimiento para recomendarnos productos similares a los que ya hemos visto o escuchado, el aprendizaje no supervisado se encuentra detrás de buena parte de la información generada por las IA actuales.

10. Aprendizaje por refuerzo

El tercer tipo de aprendizaje automático, el aprendizaje por refuerzo, es un poco diferente del supervisado y del no supervisado, porque no va a aprender de datos como los dos anteriores. ¿Por qué, entonces, se considera una subcategoría del aprendizaje automático? Porque aquí también tenemos un programa, que “aprende”, pero no a partir de datos sino de la experiencia.

En esta técnica se considera el problema a resolver como un juego en el que un programa, al que en este contexto se denomina agente, interactúa con un entorno siguiendo reglas definidas y obteniendo una serie de premios/beneficios y castigos/pérdidas por cada acción realizada. Este tipo de aprendizaje será, por tanto, una buena forma de abordar problemas en los que en cada momento hay una serie limitada de posibles acciones que se pueden tomar y cuyo efecto se puede evaluar. El ejemplo más conocido es cualquier juego de tablero, pero este tipo de aprendizaje tiene aplicaciones más interesantes, como puede ser la gestión de un almacén, donde el colocar los objetos de una u otra forma tendrá un efecto en el tiempo que se tarda de media en preparar un pedido, el control de un coche autónomo a partir de la información proporcionada por las cámaras del vehículo o la gestión de un sistema eléctrico para ajustar en tiempo real la combinación de energía renovable y otras energías según demanda y previsión meteorológica y así evitar apagones.

En la fase de entrenamiento, que aquí también existe, el agente considerará los siguientes elementos:

El estado. En el ejemplo de un juego de tablero el estado sería la disposición de piezas cuando le toca mover al agente. En casos más complejos como la conducción autónoma, el estado estaría formado por los objetos detectados en el entorno mediante las cámaras (señales de tráfico, otros vehículos, peatones), pero también la velocidad y dirección actual del vehículo, así como del resto de objetos en movimiento.

Una serie de acciones. En el juego, las posibles acciones serían las posibles

piezas a mover y cómo hacerlo, mientras que en el coche autónomo sería si debe frenar, acelerar, girar, cambiar de marcha, etc.

Una recompensa o castigo asociados a cada acción. Se trata de una recompensa o castigo inmediatos, locales, pero el agente no decidirá con este criterio cortoplacista, sino que buscará maximizar la recompensa a largo plazo. Esto hará, por ejemplo, que esté dispuesto a sacrificar una pieza de ajedrez (castigo a corto plazo) a cambio de aumentar sus posibilidades de victoria (recompensa a largo plazo).

Un objetivo final. El objetivo puede ser ganar la partida, lograr que el coche llegue a su destino sin incidentes o lograr que los paquetes de un almacén se puedan agrupar para su entrega lo más rápidamente posible.

Cuando el agente ha terminado, se dice que ha finalizado un episodio, que ha podido concluir con más o menos éxito. Entonces el sistema ajusta los pesos o posibilidades de elegir las acciones para cada estado dado, aumentando su probabilidad si se ha logrado el objetivo, o si al menos se ha mejorado con respecto a anteriores intentos, y rebajándolos en caso contrario. Estos pesos al principio tienen un valor casi al azar y aunque al principio se almacenaban en tablas hoy en día se guardan en complejas redes neuronales.

Normalmente, un entrenamiento consta de miles o millones de episodios, en los que el sistema va poco a poco mejorando. En casos como los coches autónomos, que utilizan estos algoritmos como parte de sus modelos, evidentemente no se puede poner el coche a circular en un entorno real, sino que se hacen simulaciones intentando reproducir las condiciones que se pueden dar en la conducción. Posteriormente, se pasa a entornos controlados como pistas de pruebas hasta lograr modelos lo suficientemente fiables.

Un caso de éxito del aprendizaje por refuerzo que se hizo célebre en los medios de comunicación fue el del juego Go. Aunque el campeón mundial de ajedrez había sido derrotado en 1995 por un programa que combinaba reglas apropiadas y conocimiento obtenido de otros grandes maestros, este método no resultaba útil para este juego de origen chino y muy popular en países asiáticos, que se disputa en un tablero de 19 ♟ 19 intersecciones. El Go se juega con piedras negras y blancas colocadas en las intersecciones, y el objetivo es rodear territorio

vacío o capturar piezas enemigas. El número de escenarios o estados posibles es enorme, dotando al Go de una complejidad que lo hacía intratable computacionalmente.

Sin embargo, en 2016 AlphaGo, un programa desarrollado por DeepMind, la empresa creada por Google, derrotó al campeón del mundo Lee Sidol en un torneo que finalizó 4-1 para la máquina. La única victoria de Lee Sidol se logró con un movimiento genial que abría un escenario tan complejo que terminó provocando un movimiento erróneo del programa. Por su parte, el programa también realizó en alguna de sus victorias movimientos originales, nada intuitivos para un humano, pero que le dieron una ventaja decisiva al final de la partida. Una segunda versión, todavía más potente, derrotó al año siguiente al campeón Ke Jie por un contundente 3-0, dominando cada partida de principio a fin.

El programa original constaba de varias fases. Una fase estaba basada en aprendizaje supervisado, en el que la etiqueta a averiguar era el movimiento siguiente y los datos de entrenamiento las partidas desarrolladas por humanos profesionales. En una fase posterior este conocimiento se afinaba utilizando aprendizaje por refuerzo y todavía se realizaban otras mejoras para reducir el número de jugadas a explorar.

En 2017 la misma empresa presentó AlphaGo Zero, una nueva versión de AlphaGo entrenada únicamente mediante aprendizaje por refuerzo. Al principio del entrenamiento, el sistema partía de solo el conocimiento de las reglas y de su objetivo: ganar la partida. Después de tan solo tres horas jugando consigo mismo y aplicando aprendizaje por refuerzo, el programa ya era capaz de ganar a jugadores avanzados. A los tres días superó al campeón de Europa y tras aproximadamente cinco millones de partidas jugadas contra sí mismo, que requirieron 40 días de entrenamiento, el sistema alcanzó un nivel tan superior a todo lo conocido hasta entonces que ningún campeón se volvió a plantear jugar una partida contra la máquina en público.

Desde entonces, el entrenamiento por refuerzo se ha utilizado en muchos ámbitos como sistemas de distribución de viajeros y conductores en empresas de transporte de pasajeros, para mejorar los sistemas de refrigeración de los centros de proceso de datos, llegando a mejorar en un 30% el consumo de energía, e incluso se está ensayando para controlar la medicación en pacientes hospitalizados, determinando la mejor estrategia para mantener las constantes

deseadas con la menor cantidad de medicación.

11. ¿Cómo saber si un modelo

es realmente adecuado?

Hemos visto ya como los sistemas de inteligencia artificial basados en aprendizaje automático generan modelos a partir de datos. En el caso del aprendizaje supervisado, seguramente el más común, estos datos tienen una columna especial: el valor que se quiere aprender. Así, a partir de datos previamente etiquetados el sistema aprenderá a relacionar los atributos, el resto de las columnas, con el valor en cuestión, lo que le permitirá predecir la etiqueta cuando se utilice el modelo con datos nuevos, no etiquetados, que es su propósito final.

Esta fase de generación del modelo a partir de datos se conoce como fase de entrenamiento, y es la más costosa en tiempo y recursos. En el caso de un modelo capaz de generar lenguaje como GPT-5, aunque no hay datos oficiales, se supone que se emplearon alrededor de 200.000 chips avanzados (GPU, de los que hablaremos en otro capítulo) durante varios meses. Es un caso extremo, porque ChatGPT utiliza para su entrenamiento prácticamente todo internet, pero da una idea de la complejidad de este proceso. Esta fase es tan compleja porque, por ejemplo, en una red neuronal se trata de obtener los pesos asociados a cada neurona que mejor aproximen las etiquetas de salida. Aunque los algoritmos para lograr estos valores han mejorado mucho en los últimos años, en particular, logrando que el problema se pueda dividir en varios subproblemas que se resuelven a la vez (paralelización) todavía se trata de una cantidad ingente de cálculos.

Sin embargo, el modelo, una vez entrenado, puede utilizarse para predecir o generar datos de forma ágil. Podemos comparar el entrenamiento con la fase de diseño y fabricación de una maquinaria, un proceso largo y costoso que requiere muchos ajustes, mientras que el uso del modelo equivale con poner finalmente la máquina a trabajar. El símil resulta también adecuado para ejemplificar que, igual que la máquina se fabrica una sola vez y se utiliza miles de veces, los modelos se entrenan una sola vez para ser utilizados después constantemente. Por ejemplo, cada vez que le planteamos una pregunta a ChatGPT estamos

utilizando su modelo, que solo cambia cuando decimos que ha salido una versión del sistema.

En el ejemplo del banco que quiere detectar fraude, puede que el modelo tarde semanas en entrenarse a partir de datos de transacciones anteriores, unas detectadas como fraude y otras etiquetadas como correctas, pero una vez obtenido el modelo este predecirá en cuestión de milisegundos si una nueva transacción es potencialmente fraudulenta. Pero ¿cómo saber si realmente el modelo de IA basado en aprendizaje automático supervisado está cumpliendo su función? La idea más directa es ponerlo en funcionamiento y comprobar cuánto va acertando. En el caso del banco, se puede poner el sistema a funcionar en modo prueba y comprobar las transacciones que el modelo señale como posiblemente fraudulentas para ver si lo son realmente. Sin embargo, este proceso puede ser largo y costoso, y si se están generando varios modelos alternativos con diferentes ideas que se quieren comparar puede resultar inviable en la práctica.

Por eso, antes de llegar a la fase final de prueba, se suele hacer una simulación sobre la eficiencia del modelo. Para eso se divide el conjunto de datos ya etiquetados en dos, los llamados conjuntos de entrenamiento y test. Esta división se hace de forma aleatoria, seleccionando filas al azar de este conjunto inicial hasta repartirlo en los dos conjuntos mencionados. El de entrenamiento se utiliza, como su propio nombre indica, para entrenar, y a continuación se utiliza el de test para predecir, simulando de esta forma la llegada de datos nuevos de forma rápida y sin coste alguno. Como los datos del test también están etiquetados, podremos comparar el valor predicho por el modelo con el etiquetado, y de ahí obtendremos una primera evaluación del sistema de forma eficiente y barata.

En la práctica, el procedimiento se complica un poco y se suelen utilizar hasta tres divisiones de los datos originales: el conjunto de entrenamiento ya mencionado para obtener el modelo de prueba, el conjunto de validación para ajustar el modelo y el conjunto de test con el que, finalmente obtenemos una estimación de lo adecuado que resulta el modelo.

Podríamos explicar esto mejor si utilizamos un símil con un estudiante que tiene que preparar un examen a partir de un conjunto grande de problemas resueltos. Para ello divide el conjunto en tres partes:

Entrenamiento: el subconjunto de problemas ya resueltos (es decir, etiquetados) que utiliza para estudiar. Cabe mencionar, y aquí es donde falla la analogía, que un estudiante real no dispondría únicamente de problemas resueltos, sino también de una teoría asociada, pero en el “estudio” que hace el aprendizaje automático solo se dispone de los ejemplos y es el sistema el que deduce la teoría.

Validación: otros problemas que el estudiante utiliza para probar si está aprendiendo. Al hacerlo se permite a sí mismo repasar los problemas de entrenamiento ajustando mejor sus conocimientos al darse cuenta de que había detalles que no había aprendido.

Test: problemas nuevos que simulan el examen final real; esta vez, no hay vuelta atrás ni posibilidad de repasar, al menos si se quiere tener una estimación real de lo que se ha aprendido.

En la práctica, es posible que el estudiante no sea tan meticuloso y utilice un solo conjunto de problemas, estudie con ellos y luego extraiga unos cuantos al azar para ver si ha aprendido. En aprendizaje automático se ha comprobado que esto es mala idea: hay sistemas de aprendizaje que lo harán muy bien si les pedimos que predigan datos que formaban parte del conjunto de entrenamiento, pero que luego obtendrán mucho peores resultados al enfrentarse a problemas nuevos.

Este problema se conoce como sobreajuste: el sistema se ha “aprendido de memoria” los datos de entrenamiento, pero no ha sido capaz de extraer las reglas generales y por eso falla con datos nuevos, como un estudiante que se aprendiera los problemas de matemáticas de memoria y en el examen, al enfrentarse a problemas nuevos, no supiera resolver los ejercicios.

12. Big data

La moderna IA se basa en la extracción de información a partir de datos. Disponer de datos abundantes y de calidad permite entrenar modelos eficientes, capaces de resolver problemas que, de otra forma, serían difíciles de abordar. No es, por tanto, extraño que las empresas hayan empezado a contemplar sus datos con una mirada nueva.

Por ejemplo, esos ficheros históricos de clientes que se borraban pasado un tiempo para reutilizar los discos en los que estaban alojados pasan a constituir, de repente, uno de los activos más valiosos de la compañía. Quizás no se tenga todavía claro para qué se almacenan, pero la sensación es que en algún momento pueden llegar a ser importantes, así que, por si acaso, se guardan celosamente. Es lo que se ha llamado el dataísmo, que para el pensador Yuval Noah Harari reemplaza la veneración a los dioses, y posteriormente al hombre, por la veneración al dato.

Este “síndrome de Diógenes digital” puede llevar a un caos que haga que todo este esfuerzo sea inútil si no se sabe de qué datos se dispone, dónde están alojados, etc. Por eso ha surgido el término gobernanza de datos, que trata de las políticas y procesos que consideran los datos como un activo de negocio, garantizando su fiabilidad y buen uso. Incluye tareas como tener identificado cada fichero con su origen y fecha de creación, conocer el significado de cada columna de datos, poder trazar las diferentes versiones a lo largo del tiempo con los cambios que han tenido lugar o bien asegurar su buen uso, tanto con respecto a filtraciones de datos como con respecto a malos usos, por ejemplo, con respecto a las leyes de protección de datos.

Una parte importante de esta tarea es el almacenamiento. En efecto, la cantidad de datos de los que se dispone en la actualidad es ingente. No se trata solamente de los propios datos de negocio, se quieren almacenar también, por ejemplo, resultados y precios de la competencia, opiniones acerca de nuestros productos en redes sociales o accesos a la página web de la empresa.

En el caso de empresas que fabrican maquinaria, se dispondrá, además, de los datos procedentes de multitud de sensores, cada vez más presentes en todo tipo

de dispositivos y que se transmiten a la empresa a través de la llamada internet de las cosas o IoT (Internet of Things). Estos datos son vitales para asegurar el correcto funcionamiento de la maquinaria en todo momento y la calidad del producto final, pero también para elaborar modelos de mantenimiento predictivo que puedan avisar de futuros fallos, evitando así costosas reparaciones e interrupciones de servicio.

Hasta la llegada de internet, los datos de una empresa se almacenaban habitualmente en el disco duro de su propio ordenador corporativo. Si realmente hacían falta muchísimos datos, algo que únicamente podía sucederles a grandes empresas como bancos o aseguradoras, se podía disponer de un ordenador más potente para manejar discos más grandes. Esta técnica consistente en aumentar la potencia de la máquina para adaptarse a escenarios que requieren más recursos es conocida como escalado vertical. Si incluso esta opción no resultaba suficiente, siempre existía la opción de tener discos externos con datos históricos, que se utilizaban solo cuando era necesario, porque incorporarlos al sistema era un proceso lento y con una inevitable parte manual.

Sin embargo, las empresas que operaban con datos de internet, como Alphabet (Google) o Facebook, se dieron cuenta desde el principio de que tenían que tratar con cantidades de datos no contempladas hasta entonces. Además, necesitaban disponer de esos datos todos a la vez y hacerlo de forma rápida para que el usuario de sus programas obtuviera una respuesta inmediata. Se trataba de un cambio de paradigma en el almacenamiento, y el escalado vertical no era la solución factible. Simplemente, no existía un ordenador capaz de manejar tal cantidad de datos que, además, no dejaban de crecer según la red se expandía y el número de usuarios se multiplicaba. Por eso buscaron una solución diferente: en lugar de un gran superordenador pusieron muchos ordenadores pequeños a trabajar conjuntamente, una solución que se conoce como escalado horizontal y que constituye la esencia del almacenamiento y procesamiento rápido de grandes cantidades de datos, fenómeno que conocemos como big data.

En el escalado horizontal, los datos se reparten entre muchos ordenadores que trabajan coordinadamente, como si de un solo gran ordenador se tratara. En este caso, obtener escalabilidad frente al crecimiento del volumen de datos resulta muy sencillo: cuando el clúster, que es como se suele llamar a este conjunto de ordenadores, va alcanzando el límite de su capacidad de almacenamiento, basta con añadir nuevos componentes, es decir, nuevos ordenadores pequeños. Lógicamente, esto es más barato que comprar ordenadores más y más grandes,

lo que, además, como hemos visto en la práctica, es inviable en el caso de big data.

Además, la escalabilidad horizontal tiene como ventaja añadida que el crecimiento no compromete la velocidad. Por ejemplo, si se quiere buscar un dato en un clúster de 1.000 ordenadores, no hace falta ir buscando el dato en cada ordenador de forma secuencial, lo que sería 1.000 veces más lento que hacerlo en un solo ordenador. Por el contrario, cada ordenador que dispone de su disco y su procesador propios buscará el dato en su disco, lo que significa que se hará en los 1.000 ordenadores simultáneamente (“en paralelo”, como prefieren decir los informáticos). El primero que lo encuentre “levantará la mano” avisando de que ya está localizado y deteniendo la búsqueda en el resto, por lo que el tiempo requerido será muy similar al que se precisaría para un solo ordenador.

Por supuesto, tener tal cantidad de ordenadores funcionando tiene también sus inconvenientes. Para empezar, es fácil que se estropee algún componente del clúster. Si se dispone de un clúster de 1.000 ordenadores, estadísticamente ocurrirá casi a diario. Para evitar la correspondiente pérdida de datos, por lo general, cada dato nuevo que llega a un clúster no se graba en uno, sino en tres ordenadores diferentes, lo que aporta seguridad, pero obliga a triplicar el número de unidades utilizadas. Es decir, nuestro clúster de 1.000 ordenadores se convierte, si queremos tener seguridad frente a pérdida de datos, en 3.000 ordenadores. Además, en caso de datos realmente críticos, una de esas copias o réplicas estará alojada en un ordenador que se encuentre en un lugar alejado, a veces incluso en otro continente. De esta forma se disminuye el riesgo de perder datos debido a catástrofes como inundaciones o incendios.

Evidentemente, proporcionar alimentación eléctrica, refrigeración y mantenimiento a cientos de ordenadores es muy costoso y requiere de instalaciones especiales, los llamados centros de proceso de datos. Todas las grandes empresas poseen al menos una de estas gigantescas infraestructuras donde almacenan y analizan sus datos. Por ejemplo, el del grupo Inditex se encuentra en Arteixo (Coruña) y fue inaugurado en 2014 con 4.000 ordenadores y 1.300 kilómetros de fibra óptica. Allí se recibe de forma inmediata la información de los miles de tiendas del grupo y se sabe lo que se ha comprado en cada lugar en cada instante, lo que permite planificar el stock de las tiendas y realizar los cambios de producción de forma prácticamente instantánea.

Sin embargo, una empresa mediana o pequeña no podrá permitirse el lujo de tener su propio centro de proceso de datos debido a su elevado coste. Sin embargo, es posible que sí disponga de tantos datos como para necesitar un pequeño clúster de almacenamiento; simplemente, la interacción de los clientes con la página web ya genera muchísimos datos que merecen ser almacenados y analizados. ¿Qué hacer en este caso? La solución es “alquilar” temporalmente unos pocos ordenadores de un gran centro de proceso de datos: es lo que llamamos disponer de un espacio en la nube.

La primera gran compañía a la que se le ocurrió alquilar parte de su centro de datos no fue una de las grandes tecnológicas de internet como Google o Microsoft. Fue la empresa Amazon la que tuvo y desarrolló esta idea con su plataforma AWS (Amazon Web Services). Y el negocio ha resultado tan lucrativo que en 2024 este segmento aportó el 58% del beneficio operativo total de la empresa.

El éxito del almacenamiento en la nube ha sido tal que ha pasado de la mediana y pequeña empresa a los particulares. Mucha gente la utiliza para almacenar, por ejemplo, las fotos que hace con los móviles, de forma que las imágenes de nuestras vacaciones terminan, sin nosotros saberlo, en alguno de estos centros de procesos de datos, junto con otra información como por ejemplo datos acerca de las aplicaciones que hemos utilizado, los restaurantes que hemos reservado con las aplicaciones de nuestro móvil o incluso nuestros datos de ejercicio físico. Datos que servirán, en muchas ocasiones, para entrenar nuevos modelos de IA.

13. Hardware específico para IA

Los modelos actuales de IA tienen miles de millones e incluso billones de parámetros. Cuando estos modelos son entrenados, se deben realizar billones de operaciones matemáticas con el fin de obtener los mejores valores para dichos parámetros. Incluso cuando el modelo está ya entrenado, su utilización para resumir un texto o generar una respuesta también requiere resolver millones de operaciones matemáticas. Una característica de las operaciones realizadas en IA es que normalmente deben resolver exactamente el mismo cálculo, pero con muchos valores de entrada diferentes a la vez. También es interesante resaltar que muchos de estos cálculos son independientes de los demás, es decir, se podrían resolver de manera aislada por distintas unidades de procesamiento, porque el resultado de uno no afectará al resultado de los demás.

Los ordenadores tradicionales, ya sea de sobremesa o portátiles, están gobernados por una unidad central de procesamiento (CPU, central processing unit). Estas unidades son muy versátiles y pueden realizar gran cantidad de operaciones básicas diferentes. Con objeto de poder realizar varias tareas a la vez y así aumentar su eficiencia, cada CPU contiene varias pequeñas unidades independientes llamadas núcleos. Sin embargo, las CPU tienen un número de núcleos bastante pequeño, actualmente entre unos 4 y 16. Esto hace que como máximo puedan ejecutar 16 operaciones en paralelo, lo que es insuficiente para las necesidades de cómputo de la IA.

Desde los años noventa del siglo XX los ordenadores incluyen tarjetas gráficas, un hardware especializado para acelerar el procesamiento de gráficos en 2D y 3D. Aproximadamente a partir del modelo GeForce 256 de la empresa NVIDIA, lanzado en 1999, las tarjetas gráficas dieron un salto en sus capacidades de cómputo gráfico y se comenzaron a llamar unidades de procesamiento gráfico (GPU). La GPU se utilizaba principalmente para poder jugar a videojuegos, aunque también era útil para ejecutar aplicaciones de diseño gráfico. El problema que resolvía era muy específico: dibujar una escena con todas sus sombras y colores. Además, la GPU debía realizar todos los cálculos para dibujar esa escena de manera muy rápida, ya que se debe dibujar la escena del videojuego al menos 30 veces por segundo para que el movimiento se note

fluido. Las operaciones que hay que realizar para determinar el color de cada píxel o punto de pantalla son siempre las mismas: determinar cuánta luz llega a cada elemento desde las diferentes fuentes de iluminación, qué color tiene el material, qué ángulo forman, etc. Estas mismas operaciones matemáticas se repiten para cada píxel de una escena, por lo que las GPU están especializadas por diseño en realizar muchas operaciones similares en paralelo y de manera independiente para cada píxel.

Pronto los programadores se dieron cuenta de que las GPU no solo servían para lograr videojuegos con movimiento realista: eran potentes procesadores capaces de realizar miles de operaciones a la vez. La idea de utilizar la tarjeta gráfica para algo diferente a hacer gráficos parecía un tanto extraña en principio, pero tuvo un gran éxito. Con el objetivo de facilitar la utilización de las GPU más allá de los entornos gráficos y que se pudieran usar en cualquier otro tipo de cómputo masivo que requiriese ejecución en paralelo, en 2006, NVIDIA lanzó su arquitectura CUDA (Compute Unified Device Architecture), que permitía a los programadores realizar cualquier cómputo masivo de propósito general en la GPU (este tipo de utilización de las GPU se conoce como GPGPU (General-Purpose Computing on Graphics Processing Units)). A partir de este momento, las GPU empiezan a ser utilizadas en ciencia, ingeniería, minería de datos y aprendizaje automático, y aproximadamente a partir del año 2012 comienzan a impulsar el desarrollo de la IA a través de software específico como TensorFlow y PyTorch, diseñados para explotar la gran capacidad de cómputo y el paralelismo que brindan las GPU programables para el entrenamiento y uso de modelos de IA.

Desde el surgimiento de GPGPU, las GPU han experimentado un gran aumento en sus capacidades de cómputo e incluso han surgido modelos diseñados principalmente para IA y no ya en cálculos gráficos. Por ejemplo, la tarjeta NVIDIA H100 es una GPU profesional que cuenta con 80 GB de memoria y hasta 18.432 núcleos para ejecutar operaciones en paralelo, pudiendo alcanzar un rendimiento de más de 700 billones de operaciones por segundo usando números con decimales, es decir, 700 TFLOPS (del inglés Tera Floating Point Operations Per Second). Esta GPU puede costar en torno a 40.000 dólares, y su alquiler en un entorno de cómputo en la nube puede ascender hasta diez dólares por hora. Otro modelo un poco menos potente es el H800 de NVIDIA, con 80 GB de memoria y 14.592 núcleos de cómputo paralelo. Este modelo, que fue el utilizado por DeepSeek para entrenar sus modelos debido a las limitaciones de exportación de tecnología de Estados Unidos a China, alcanza los 200 TFLOPS

por un precio de 30.000 dólares y en torno a dos dólares por hora para su alquiler en sistemas de cómputo en la nube.

El gran avance de las GPU, junto con su capacidad para realizar cálculos masivos más allá del mundo de los gráficos, ha sido una auténtica revolución para la inteligencia artificial. Gracias a este tipo de hardware, hoy es posible entrenar modelos con miles de millones e incluso billones de parámetros en tiempos razonables, algo impensable hace solo una década. Este salto ha impulsado de forma decisiva el aprendizaje profundo, aunque su uso intensivo sigue estando reservado a grandes compañías y centros de investigación, debido al enorme coste de la infraestructura. Modelos como GPT, Gemini o Claude no serían posibles sin miles de GPU especializadas trabajando en paralelo. Además, esta dependencia que la IA tiene de las GPU es lo que justifica el altísimo aumento de la capitalización bursátil de empresas fabricantes de GPU (como NVIDIA) en los últimos años.

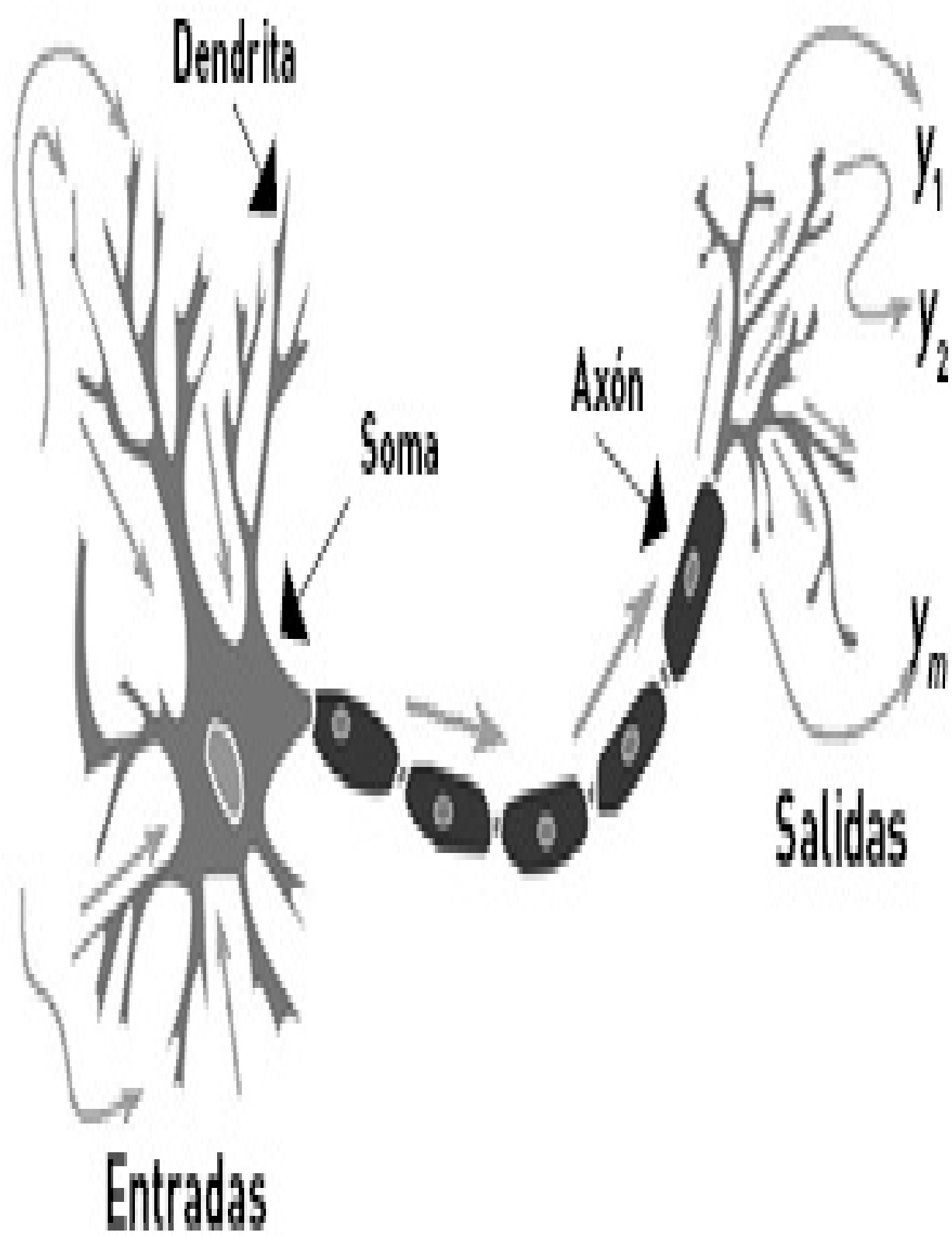
14. Neuronas artificiales

Existen diferentes propuestas matemáticas y estadísticas para diseñar sistemas de IA. Sin embargo, los modelos más avanzados y potentes que disponemos hoy en día son los basados en redes neuronales. Este tipo de sistemas de IA se fundamentan en una idea muy simple: la neurona artificial, que podemos considerar como una generalización del perceptrón de Rosenblatt que vimos a hablar de la historia de la IA. Es la combinación de muchas neuronas artificiales y su capacidad de entrenamiento eficiente el que ha conseguido producir los resultados tan impresionantes de IA que tenemos hoy en día.

Una neurona artificial no es más que una función matemática programada en un ordenador. Su diseño se inspira en la estructura y funcionamiento de una neurona biológica, tal y como existen en el sistema nervioso de los seres humanos y el resto de animales.

Las neuronas biológicas son células especializadas que reciben diversos impulsos eléctricos a través de sus dendritas y a partir de ellos se activan o no, generando un impulso eléctrico de salida a través su axón. A su vez, ese axón se conecta a las dendritas de entrada de otras neuronas, lo que genera una vasta red de interconexiones de impulsos eléctricos de la cual emergen comportamientos complejos como los actos reflejos para la autoconservación de un ser vivo, las emociones, la autoconciencia o el razonamiento matemático.

Neurona biológica

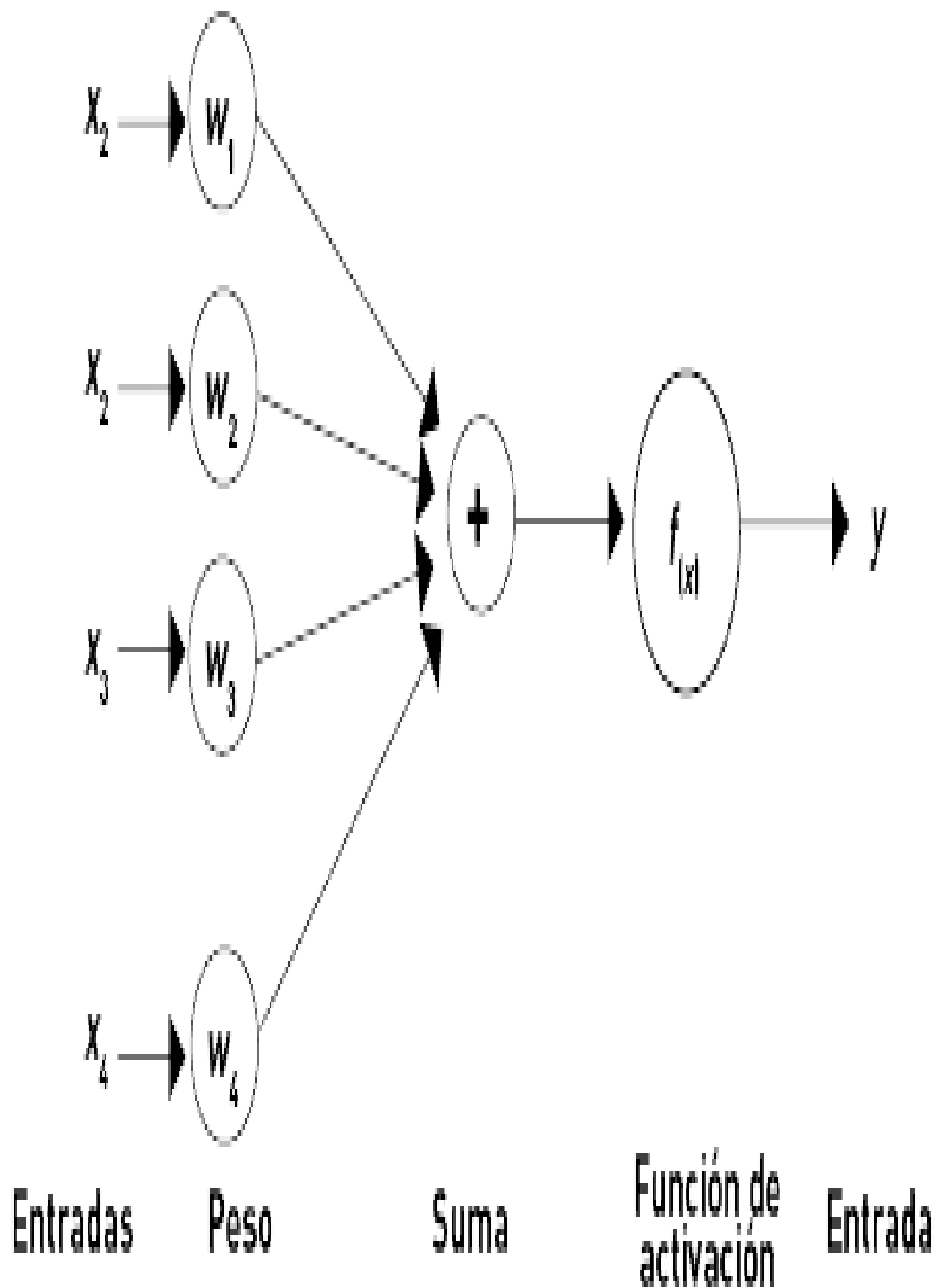


Fuente: Wikipedia.

Una neurona artificial sigue la misma lógica que las neuronas biológicas: toma varias entradas y genera una única salida indicando si se inhibe o se activa. Tanto la entrada como la salida se representan mediante números con decimales. Las neuronas artificiales son elementos conceptualmente muy básicos, pero fundamentales para construir grandes redes neuronales artificiales, de las que veremos que puede emerger un comportamiento complejo de IA como detectar y seguir elementos en vídeos, generar textos a partir de una petición o transcribir los diálogos de un audio a texto escrito.

La organización de una neurona artificial sigue el siguiente esquema:

Esquema simplificado de una neurona artificial

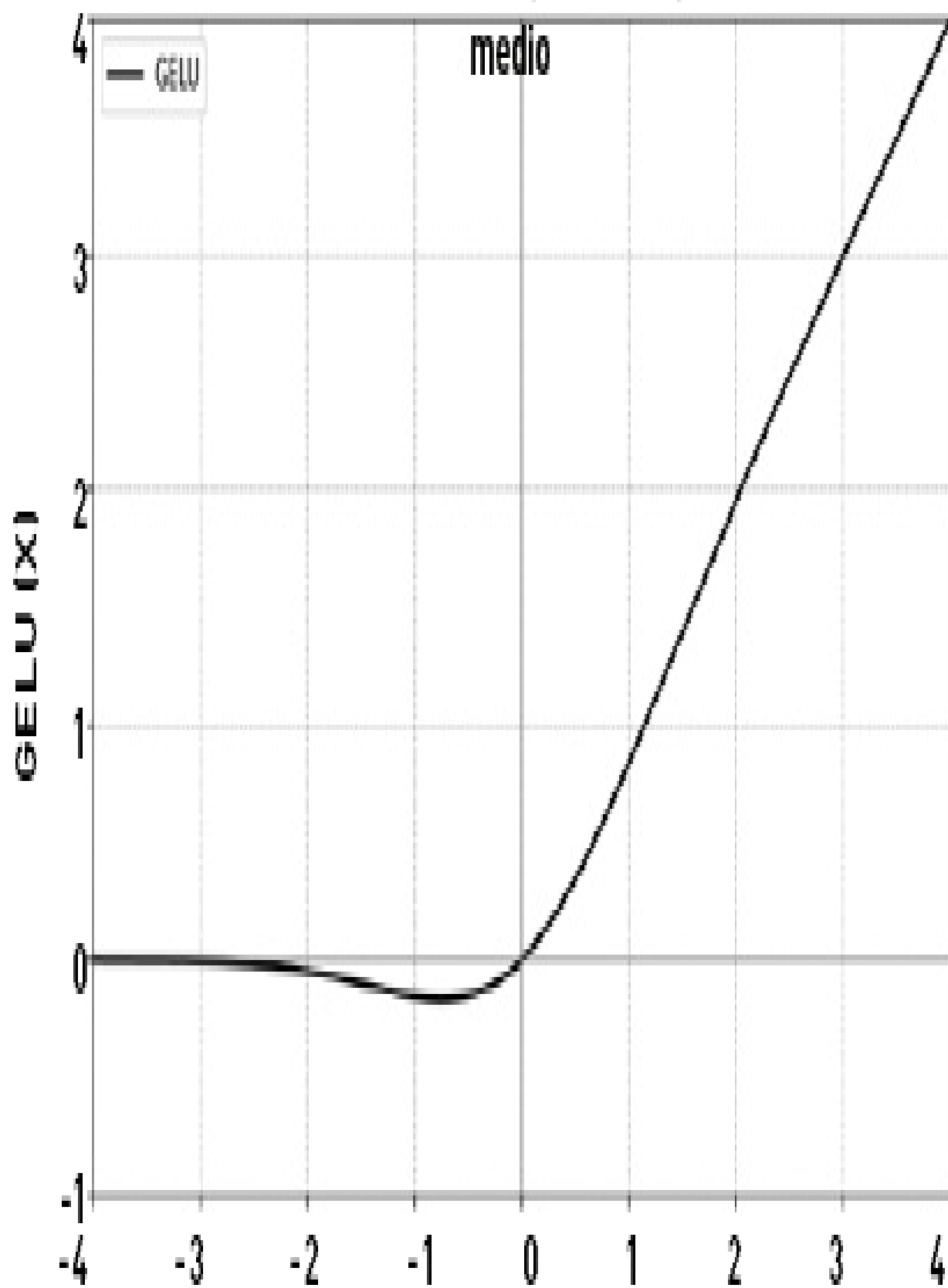


Fuente: Elaboración propia.

Como se puede apreciar, la neurona artificial toma una serie de valores de entrada $x_1, x_2, \dots, x_n$. Cada una de esas entradas tiene un peso (weight) asociado $w_1, w_2, \dots, w_n$, peso que multiplicará al valor de cada entrada. La función de los pesos es debilitar o potenciar el valor de cada entrada, es decir, decidir qué entradas son más o menos relevantes para la neurona. Los valores modificados por cada peso son sumados: $x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n$. Además, la neurona suele contar con un valor adicional llamado bias o sesgo (usualmente representado con la letra “b”), que se añade a la suma de las entradas y que actúa como un ajuste fino para facilitar su activación, aunque por simplicidad lo omitiremos en los cálculos. Finalmente, el resultado de la suma ponderada de valores se pasa a través de una función de activación, que decide si la neurona artificial se debe activar con las entradas recibidas, y la magnitud de dicha activación. Aunque no es nuestro objetivo entrar en detalles matemáticos, sí que es importante destacar que la función de activación juega un papel imprescindible en una neurona artificial, ya que es el componente que proporciona el comportamiento no lineal. En un sistema lineal, pequeñas variaciones en la entrada producen únicamente pequeñas variaciones en el valor de la salida. Sin embargo, gracias a la función de activación se consigue que una pequeña variación en el valor de una de las entradas pueda hacer que una neurona artificial pase de estar inactiva a activarse. Es decir, gracias a la función de activación, una neurona artificial funciona como un interruptor inteligente, que decide si se enciende o no a partir de las intensidades que recibe por sus entradas.

A modo de ejemplo, a continuación, se muestra la gráfica de la función de activación conocida como GELU (Gaussian Error Linear Unit). Omitiendo los detalles sobre su fórmula concreta, que podría asustar mucho, podemos ver con facilidad su comportamiento no lineal. Para valores negativos de la entrada (eje horizontal), genera resultados muy cercanos a 0 (eje vertical), mientras que según la entrada toma valores positivos la función se activa y genera valores cada vez más positivos, proporcionales a la entrada.

Función de activación GELU



Fuente: Elaboración propia.

A modo de ejemplo completo, consideremos una neurona artificial de dos entradas (x_1 y x_2) con los siguientes pesos: $w_1 = -2,1$ y $w_2 = 2,2$. Analizando estos pesos, ya podemos anticipar que la activación de la neurona penalizará el doble los valores positivos en la entrada x_1 , puesto que su peso es $-2,1$, mientras que los valores recibidos por x_2 serán amplificados y duplicados a la hora de activar la neurona. Si tomamos valores $x_1 = 2,0$ y $x_2 = 0,8$, la suma ponderada generada por la función de transferencia será $2,0 \cdot -2,1 + 0,8 \cdot 2,2 = -2,44$, que al pasar por la función de activación GELU producirá $-0,018$, un valor cercano a cero que indica la inhibición de la neurona artificial. Por otro lado, si la entrada se modifica para tomar valores $x_1 = 0,5$ y $x_2 = 1$, vemos que la suma ponderada es $0,5 \cdot -2,1 + 1 \cdot 2,2 = 1,15$. Tras aplicar la función GELU, vemos que en la gráfica para un valor de x un poco superior a 1 el valor en el eje vertical es también ligeramente superior a 1; en concreto, se trata de $1,006$, lo que indica la activación de la neurona.

Pero ¿cómo se eligen los pesos de cada entrada? Para ello, se debe realizar un aprendizaje supervisado a partir de un conjunto de datos etiquetado previamente. Para cada caso de entrenamiento, es decir, para cada fila del conjunto de datos de entrada, se realizan los siguientes pasos:

Se fijan los valores de entrada, esto es, los atributos o característica de la fila.

Se calcula la salida generada por la neurona.

Se computa el error de la salida, es decir, diferencia entre la salida esperada, determinada por la etiqueta de la fila que estamos usando, y la salida obtenida.

Teniendo en cuenta la magnitud de ese error, se modifican los pesos de cada entrada en la dirección adecuada para reducir ese error.

Los pesos de la neurona artificial se suelen inicializar con valores aleatorios, y

según se van repitiendo pasos de aprendizaje, los valores de los pesos van acercándose a los valores más adecuados para minimizar el error entre la salida producida y la etiqueta en los casos de entrenamiento.

Por otro lado, la función de activación la decide el diseñador del sistema de IA teniendo en cuenta diversas características, como su facilidad de cómputo, su facilidad de entrenamiento o la precisión de sus resultados en experimentos previos. Es importante tener en cuenta que un sistema de IA completo puede tener miles de millones de neuronas artificiales, por lo que hay que encontrar un equilibrio entre funciones de activación que den buenos resultados y cuyo cálculo sea lo más rápido posible.

15. Redes neuronales artificiales

Las redes neuronales artificiales son la técnica de aprendizaje automático más potente y que mejores resultados está dando en el campo de la IA. Son conocidas principalmente por sus aplicaciones en el área del procesamiento del lenguaje natural, donde constituyen el fundamento de los modelos de lenguaje grandes como ChatGPT, pero también son la base de otras aplicaciones como la generación de imágenes y vídeos a partir de una descripción en texto, o la generación de una transcripción a partir de un audio con diálogos.

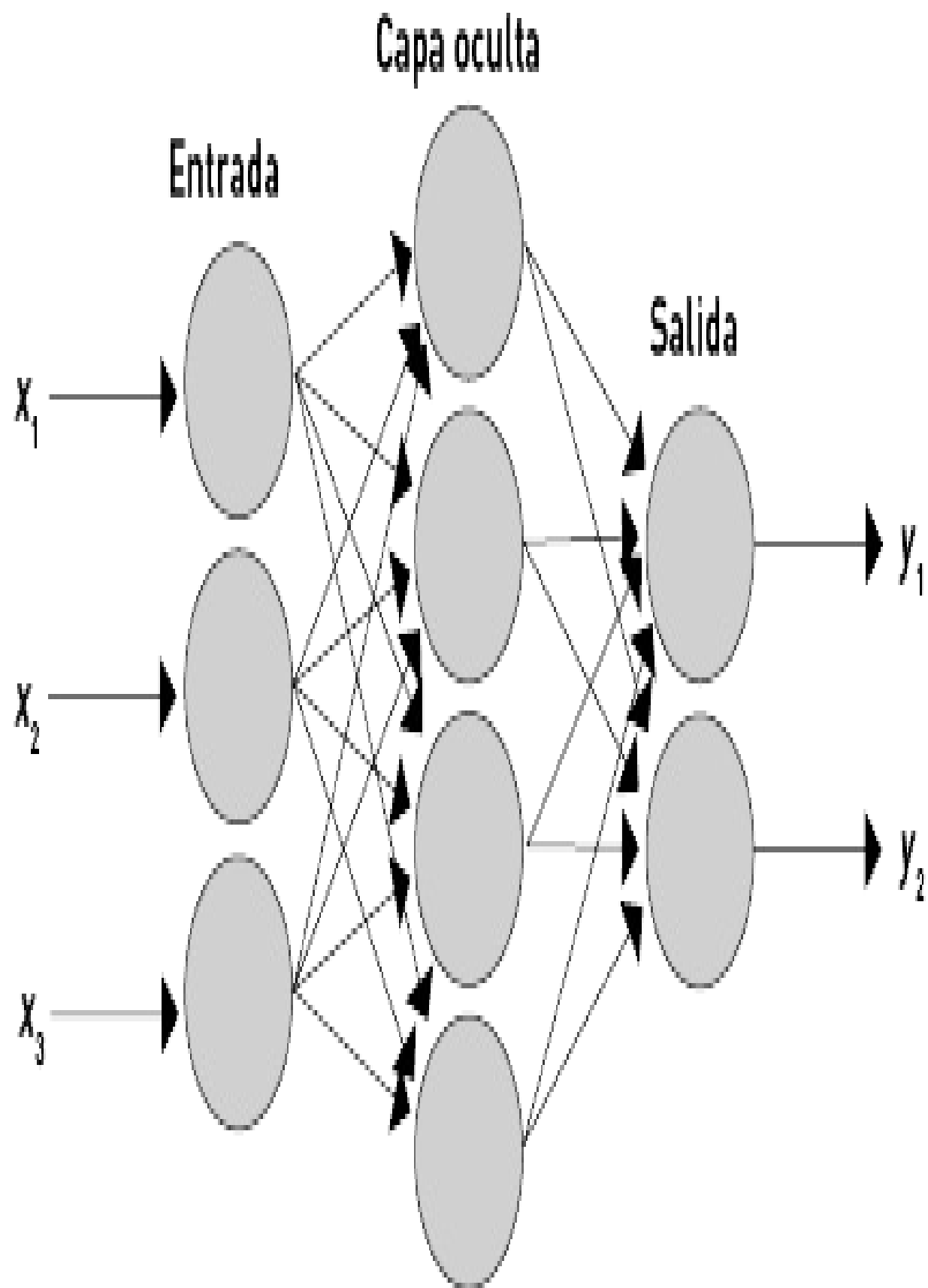
Las redes neuronales artificiales están formadas por una red interconectada de neuronas artificiales, resultando en una especie de modelo simplificado del cerebro, donde las neuronas reciben estímulos de las neuronas anteriores a partir de los cuales determinan su activación o inhibición que se propaga a las siguientes neuronas de la red hasta que al final se genera una salida. Esta salida puede representar una clasificación, por ejemplo, determinar si una imagen contiene un perro o un gato, pero también puede representar la siguiente palabra a generar en un texto o el color que debe tener un píxel concreto de una imagen generada. Intuitivamente, se puede considerar una red neuronal como una cascada de pequeñas decisiones individuales que toman las neuronas artificiales, de tal manera que el conocimiento se va acumulando y refinando progresivamente a medida que la información fluye por la red, hasta que finalmente se alcanza una decisión global coherente. Cada capa transforma la representación de los datos, extrayendo patrones cada vez más complejos o abstractos, como si cada nivel de la red fuera afinando una interpretación inicial hasta dar con una respuesta final razonada.

Desde el punto de vista de la organización, una red neuronal se organiza como una secuencia ordenada de distintas capas, donde cada una está formada por un número prefijado de neuronas artificiales. La salida de una neurona de una capa se conecta con una entrada de todas las neuronas de la capa siguiente, lo que permite transmitir el conocimiento que ha destilado a todas las neuronas siguientes. El inconveniente de esta arquitectura es que hace crecer rápidamente el número de conexiones de una red neuronal y, por tanto, el tiempo necesario para calcular la salida.

La primera capa de una red neuronal se conoce como capa de entrada, porque recibe las entradas del sistema ($x_1, x_2, \dots, x_n$), y tiene tantas neuronas como entradas o atributos tienen los datos de entrenamiento. A partir de ahí, se insertan un número configurable de capas, a las que se denomina capas ocultas, que son las encargadas de ir aprendiendo aspectos cada vez más complejos de los datos a partir de los aspectos más simples detectados por las capas ocultas anteriores. Finalmente, la capa de salida recopila toda la información compleja detectada por las capas ocultas y produce un resultado final.

En la siguiente figura se puede ver un ejemplo de red neuronal artificial muy básica para clasificación de mensajes como spam o legítimos, donde cada círculo representa una neurona artificial. La red neuronal admite mensajes expresados como tres valores (x_1, x_2, x_3) en su capa de entrada, tiene una única capa oculta de cuatro neuronas y proporciona una capa de salida de dos neuronas que producirá dos valores de salida (y_1, y_2). En estas dos salidas se expresaría la predicción de la red neuronal actuando como modelo clasificador: el valor de y_1 representaría el grado de convicción de que el mensaje fuera spam, mientras que el valor de y_2 expresaría el grado de convicción de que el mensaje fuera legítimo.

Esquema de una red neuronal artificial



Fuente: Wikipedia.

Las redes neuronales artificiales están formadas por neuronas, que como ya hemos explicado, tienen una serie de pesos cuyos valores hay que determinar para cada tarea. La cantidad total de pesos de todas las neuronas es lo que se conoce como el número de parámetros de una red neuronal. En el caso de la red neuronal de la figura, esta tendría 3 pesos de la capa inicial, más $3 \cdot 4 = 12$ pesos de la conexión de la entrada con la capa oculta, más $4 \cdot 2 = 8$ pesos de las conexiones de la capa oculta con la capa de salida, sumando un total de 23 pesos. No obstante, los grandes modelos de lenguaje como ChatGPT pueden llegar fácilmente al orden de los billones de parámetros. El proceso de entrenar una red neuronal significa ajustar automáticamente los pesos de sus conexiones internas para que aprenda a resolver una tarea: clasificar imágenes, traducir textos, predecir precios, etc. Este proceso de aprendizaje supervisado es una generalización del entrenamiento de una única neurona que describimos en el capítulo anterior y se basa en inicializar con valores aleatorios todos los pesos y realizar un proceso de prueba y error a partir de datos etiquetados para los que se sabe perfectamente cuál debería ser el resultado correcto. Para cada caso de entrenamiento se realiza lo siguiente:

Se muestran los datos de entrada (x_1, x_2, x_3) a la red neuronal con su respuesta correcta (por ejemplo, un mensaje de texto con la etiqueta spam).

La red genera una salida. Por ejemplo, podría generar las salidas $y_1 = 0,00027$ y $y_2 = 0,99973$, estando muy segura de que se trata de un mensaje legítimo, ya que, recordemos, el valor y_1 es la probabilidad de ser spam y es prácticamente 0, mientras que y_2 es la probabilidad de ser correcto, y en este caso es prácticamente 1, lo que significa seguridad casi total.

Se calcula el error entre la salida predicha y la esperada. En este ejemplo, la salida esperada es de mensaje de spam, por lo que se deseaba una salida $y_1 = 1$ y $y_2 = 0$, prácticamente lo contrario de lo que ha predicho la red. Por tanto, habría un notable error tanto en y_1 como en y_2 que habría que corregir.

Usando estos errores, se ajustan los pesos de todas las neuronas de la red para

que la próxima vez funcione mejor y se acerque más a la salida esperada.

Este último paso se conoce como retropropagación (backpropagation), puesto que el error se propaga hacia atrás por la red, desde la capa de salida hasta las capas más cercanas a la entrada. La idea de la retropropagación es que, a partir del error de la salida, los pesos de la red se revisan hacia atrás, determinando qué conexiones ayudaron y cuáles estorbaron para dar el valor correcto. Cada conexión que aumentó el error se debilita un poco disminuyendo sus pesos asociados, mientras que cada conexión que se acercó a la respuesta correcta se refuerza un poco aumentando el valor de sus pesos. Esto se hace capa a capa, de la última a la primera, repartiendo esa “culpa” o “mérito” de cada conexión hacia atrás. Al repetirlo miles de veces con los datos de entrenamiento, los pesos van quedando ajustados y el error va disminuyendo, es decir, la red va aprendiendo progresivamente a realizar mejor su tarea.

Las redes neuronales artificiales han demostrado ser herramientas muy potentes para resolver problemas complejos. Entre sus principales ventajas destaca su gran capacidad de aprendizaje: estas redes son capaces de detectar patrones complejos y relaciones no evidentes en los datos, lo que las hace especialmente útiles en tareas como el reconocimiento de voz, visión por computador o traducción automática, justo las que al inicio de este libro hemos definido como correspondientes a la IA. A medida que se les proporciona más información, estos dispositivos pueden mejorar su rendimiento sin necesidad de que un humano programe reglas específicas.

Sin embargo, estas ventajas vienen acompañadas de varios inconvenientes. El principal problema que presentan las redes neuronales es el consumo de recursos, tanto en volumen de datos como de tiempo. En particular, necesitan muchos recursos computacionales (tiempo de cálculo, energía), además de requerir dispositivos de cómputo especializados como GPU.

Un problema adicional, y una de sus principales desventajas, es que su funcionamiento interno es en gran medida difícil de interpretar, lo que las convierte en una especie de “caja negra” desde el punto de vista de la explicabilidad. La incapacidad de explicar por qué una red neuronal ha tomado una decisión concreta no proviene de una falta de transparencia, sino de su gran complejidad interna. Durante el entrenamiento, cada neurona puede

especializarse en una tarea concreta, y cada peso concreto tendrá un impacto en esa tarea. Al utilizar redes neuronales, el desafío de interpretar por qué la red ha considerado que un texto es spam, una imagen contiene un gato o una solicitud de hipoteca se debe rechazar, se hace ciertamente inabordable. Retomaremos este problema de la explicabilidad de las redes neuronales en un posterior capítulo, junto con otras técnicas de IA que sí son explicables por diseño.

16. Deep learning

El deep learning o aprendizaje profundo es una rama del aprendizaje automático que se centra en el uso de redes neuronales artificiales de muchas capas para resolver tareas complejas. Aquí la profundidad se refiere al gran número de capas ocultas de la red neuronal, que pueden ir desde tres hasta cientos, miles o millones de capas ocultas, no a que el aprendizaje que se produce sea más abstracto o filosófico.

Las capas ocultas adicionales juegan un papel muy importante, ya que durante el entrenamiento van especializándose en detectar aspectos complejos sobre la entrada. Por ejemplo, en el caso de una red neuronal dedicada a reconocer letras manuscritas, una capa oculta podría aprender a detectar si una letra tiene trazo vertical, otra capa oculta detectaría si la letra tiene un trazo horizontal, otra capa valoraría si aparece alguna curva cerrada... Al final, las últimas capas ocultas podrían detectar patrones más complejos basándose en los aspectos simples previamente detectados, como si aparece un círculo completo o si la letra tiene ángulos afilados. Finalmente, la capa de salida tomaría la decisión final sobre qué letra es a partir de estos patrones elaborados, detectados gracias al esfuerzo incremental de las neuronas en las capas ocultas.

En 2025, el deep learning ya se encuentra en un punto de madurez tecnológica en el que domina muchas de las tareas más exigentes de la inteligencia artificial. Algunas de sus aplicaciones más destacadas incluyen modelos como CLIP (Contrastive Language-Image Pre-training) de OpenAI o SAM (Segment Anything Model) de Meta, que permiten entender imágenes, generar descripciones o segmentar objetos con gran precisión. Esta segmentación permite detectar con detalle los contornos de los distintos elementos que aparecen en una imagen o en un vídeo, e incluso detectar qué tipo de elementos son (perros, grifos, personas, señales de tráfico...). Por ello, este tipo de modelos tienen aplicación en medicina, para el diagnóstico por imagen, pero también en los vehículos autónomos para entender el entorno en el que se está moviendo el vehículo e incluso en vigilancia inteligente. El deep learning también juega un papel fundamental en los grandes modelos de lenguaje como ChatGPT, Gemini, Claude o Llama, donde alcanzan un nivel de comprensión y generación de texto

que roza lo humano en tareas como redacción, traducción o razonamiento básico. Gracias a estos modelos se pueden desarrollar interfaces conversacionales fluidas, asistentes virtuales que nos ayudan a realizar ciertas tareas y otras de productividad basadas en lenguaje natural.

A pesar de las innegables ventajas de los modelos de deep learning, con el crecimiento del número de capas ocultas también aumentan los inconvenientes asociados a este tipo de métodos. Por un lado, necesitan una gran cantidad de datos de ejemplos para ser entrenadas. Por ejemplo, el modelo GPT-2, antecesor de los modelos actuales de ChatGPT, que fue presentado en 2019, tiene “únicamente” 1.500 millones de parámetros, pero necesitó unos ocho millones de documentos de texto, que ocupaban en torno a 40 GB, para ser entrenado. Otros modelos más modernos como Whisper, desarrollado por OpenAI en 2022 para realizar transcripciones automáticas de texto a partir de audios, requirió de 680.000 horas de audios de voz para ser entrenado. En ese caso, el modelo Whisper también tenía en torno a los 1.550 millones de parámetros. Es difícil encontrar datos sobre el conjunto de entrenamiento de otros modelos modernos como GPT-5 o DeepSeek V3.1, pero se asume que pueden haber requerido cientos de terabytes de información.

De la misma manera que precisan de una gran cantidad de datos para ser entrenados, los modelos de deep learning también necesitan una gran cantidad de recursos de cómputo, tiempo y dinero. Además, los modelos más modernos y grandes requieren del uso de hardware específico para ser entrenados en un tiempo aceptable. Por ejemplo, el modelo GPT-2 necesitó ser entrenado durante un mes completo usando 100 GPU. Sobre el modelo Whisper, se sospecha que necesitó unos 18-20 días completos usando 8 GPU. Por último, el modelo DeepSeek-V3.1 tiene 671.000 millones de parámetros y fue entrenado en un período de 57 días. Para ello se utilizó un clúster de 2.048 GPU, lo que equivale aproximadamente a 2.788 millones de horas de GPU, con un coste estimado de 5,6 millones de dólares.

Como se puede ver, el entrenamiento de modelos de deep learning está únicamente al alcance de grandes empresas y otras organizaciones que pueden soportar esta gran inversión en tiempo y dinero. Además del alto coste necesario para su entrenamiento, el deep learning también amplifica otra de las limitaciones de las redes neuronales artificiales en general: la imposibilidad práctica para conocer las razones por las que toma una decisión, justificación que queda enterrada bajo la combinación de miles de millones de

multiplicaciones, pesos y funciones de activación actuando de manera coordinada.

Entre los modelos de deep learning existen distintas configuraciones de red o familias arquitectónicas típicas, cada una especializada en una variedad de tareas distintas dentro del campo de la inteligencia artificial. Por un lado, las redes neuronales convolucionales (CNN) están diseñadas para procesar datos con una estructura espacial, como las imágenes. Su éxito radica en el uso de capas convolucionales, que aplican pequeños filtros capaces de detectar patrones visuales locales (como bordes, esquinas, texturas o colores) que posteriormente se combinan en capas más profundas para identificar estructuras complejas como objetos o rostros. Estas redes se utilizan ampliamente en tareas de visión por computador, como el reconocimiento de imágenes (por ejemplo, identificar si hay un gato o un perro en una foto), la detección de objetos, el análisis médico de imágenes (como radiografías o resonancias) o el reconocimiento facial.

Por otro lado, las redes neuronales recurrentes (RNN) están principalmente diseñadas para trabajar con secuencias de datos, ya que introducen conexiones que permiten que una neurona en un instante de tiempo reciba información de pasos anteriores. Esta capacidad les otorga una forma de memoria temporal, muy útil para tratar información en la que el orden y el contexto son importantes, como ocurre en el procesamiento de lenguaje natural, el reconocimiento de voz o la predicción de series temporales como cotizaciones bursátiles. Por ejemplo, una RNN puede aprender a predecir la siguiente palabra en una frase o a transcribir una señal de audio en texto. Sin embargo, las RNN tradicionales tienen dificultades para mantener información a largo plazo debido a problemas como el desvanecimiento del gradiente, por lo que han sido complementadas o sustituidas por variantes como las LSTM (Long Short-Term Memory) o GRU (Gated Recurrent Unit), que mejoran el manejo de dependencias largas.

Estas dos familias, CNN y RNN, sentaron las bases del éxito del deep learning moderno antes de la llegada de los transformadores, que han logrado unificar muchas de sus capacidades en una única arquitectura flexible y escalable.

17. IA generativa

Como ya sabemos, los métodos de aprendizaje automático supervisado “aprenden” a relacionar los valores de entrada —como fotografías— y sus etiquetas correspondientes, que representan el contenido, por ejemplo, “caballo” o “casa”. Una vez obtenido el modelo, podemos utilizarlo pasándole como entrada una imagen nueva y, si el entrenamiento ha sido adecuado, nos dirá lo que se ve en la imagen. Esto es posible porque el modelo, a base de recibir y analizar muchas imágenes asociadas a cada etiqueta, ha determinado la estructura básica de un caballo, de una casa, etc. Se podría decir que, en algún sentido, la máquina ha extraído los conceptos a partir de los casos particulares.

Pero ¿y si lo hacemos al revés?, ¿y si, una vez que tenemos el modelo, en lugar de pasarle una imagen y que nos diga qué etiqueta le corresponde, le pasamos la etiqueta, por ejemplo “caballo”, y le pedimos que nos devuelva una imagen de un caballo? No todas las técnicas de aprendizaje automático permiten este uso inverso, partir de la etiqueta y obtener las características, pero, afortunadamente, algunas sí lo hacen. El resultado será un caballo generado a partir de los patrones que identifican a este animal para este modelo. Pero, esto es importante, no será ningún caballo de los que el sistema ya ha visto, sino un caballo nuevo. Es a esto a lo que llamamos inteligencia artificial generativa.

Se trata de una rama de la IA bastante reciente, donde los sistemas más potentes y conocidos en la actualidad son DALL-E de OpenAI, Stable Diffusion y Midjourney, capaces de generar imágenes a partir de descripciones en lenguaje natural. Por supuesto la IA generativa no se limita a imágenes. No en vano la “G” de ChatGPT viene de generative. En efecto, los grandes modelos de lenguaje como el propio ChatGPT son excelentes ejemplos de generación de texto, con el añadido de que buscan el texto que mejor corresponde a la cuestión que nosotros le hayamos planteado. También son conocidas las extensiones a vídeo y a audio.

Que sea bastante reciente no significa que no haya sido algo buscado por los programadores desde los inicios de la IA. Así, el ya mencionado software conversacional ELIZA de los años sesenta se puede considerar como un

primitivo intento de generar texto. En los años ochenta y noventa se empiezan a generar también imágenes, aunque todavía con una importante parte manual, empleando funciones introducidas por los humanos que ya contienen el patrón de generación, como el llamado ruido de Perlin o sistemas de partículas que buscaban generar texturas de apariencia natural, simular montañas, nubes, humo o fuego. El resultado era convincente, pero se limitaba a texturas u objetos sintéticos y no parecían fotografías del mundo real.

En esta misma época también se propusieron los primeros algoritmos para generar imágenes a partir de ejemplos existentes, como el de Efros y Leung de 1999. Estos métodos funcionaban muy bien para generar texturas naturales, como madera o rocas, pero fallaban con estructuras complejas como rostros o coches. A partir del año 2006 cobraron fuerza los autocodificadores, redes neuronales que aprendían a comprimir una imagen a una codificación latente, que retenía solo los aspectos más relevantes, y luego tomaban esa codificación comprimida para generar otra imagen lo más similar posible a la original. Modificando ligeramente la codificación latente o introduciendo algo de ruido aleatorio, el sistema era capaz de generar nuevas imágenes sintéticas que no existían en el conjunto original, pero que seguían el mismo estilo y estructura.

La primera gran revolución para la generación de imágenes ocurrió en 2014, cuando se propusieron las redes generativas adversativas que veremos en el siguiente capítulo. Gracias a la confrontación entre la red generadora y la red discriminadora este tipo de redes consiguen generar sonido e imágenes de muy alta calidad y realismo, acercándose mucho a las fotografías reales en contextos complejos, como la generación de rostros humanos, escenas específicas o sonidos que podemos reconocer como “naturales” imitando por ejemplo el sonido de las olas.

Aunque en aquel momento las técnicas generaban imágenes y sonido sintéticos de buena calidad, todavía no admitían ninguna explicación textual para dirigir esa generación. Es decir, una red generativa adversativa entrenada con fotografías de caras podía generar cientos de caras sintéticas, pero no se podía añadir una explicación textual sobre qué tipo de cara se quería generar, por ejemplo “persona asiática sonriendo con gafas y el pelo rizado”. Esto empieza a cambiar a partir de 2015 con la aparición de los modelos de difusión, que despuntan ya claramente a partir de 2020. Estos modelos se basan en redes neuronales profundas que han sido entrenadas para, a partir de una frase que explica la imagen original y una imagen con ruido añadido, ser capaces de

eliminar gradualmente el ruido y acercarse poco a poco a la imagen original. Para ello, se procede en dos fases. En la fase inicial, el conjunto de imágenes de entrenamiento se procesa añadiendo a cada imagen cada vez más ruido aleatorio, es decir, se realiza un proceso de difusión o “emborronado”, hasta conseguir que la imagen original quede cada vez más difuminada y finalmente irreconocible. El proceso recuerda a los famosos cuadros del artista Gerhard Richter que simulan fotografías borrosas, pero añadiendo cada vez más ruido hasta llegar a una imagen en la que no queda nada del original.

La segunda fase, en la que las imágenes realmente se generan, procede al revés. Una vez que el modelo de difusión está entrenado, puede revertir el proceso de difusión a partir de una imagen que es solo ruido aleatorio, generando paso a paso una imagen final sin ruido. En cada paso, la red utiliza lo que ha aprendido durante el entrenamiento y la frase con la descripción pretendida para decidir cómo modificar el ruido para acercar la imagen al resultado final deseado. Este procedimiento se repite iterativamente: en cada iteración, la imagen es un poco menos ruidosa y un poco más clara, hasta que aparece una imagen nueva, nítida y coherente que representa la idea o escena solicitada. Estos sistemas son muy utilizados en la actualidad y obtienen resultados de muy alta calidad, como demuestran las imágenes generadas por Stable Diffusion 3 (2024), DALL-E 3 (2023) o Midjourney v4 (2022). A partir de Midjourney v5 no hay confirmación pública de que usen un modelo de difusión puro, aunque se sospecha que aplican un modelo híbrido basado en difusión que no ha sido desvelado aún.

Actualmente, la generación de imágenes vive una nueva convergencia: los transformadores, que revolucionaron el lenguaje con su mecanismo de atención, se han integrado en el núcleo de los modelos de difusión para vigilar la relación entre partes de la imagen y las palabras de la descripción textual. También se ha avanzado en sistemas multimodales como GPT-4o que no solo pueden tomar como entrada textos, imágenes y vídeos, sino que también pueden transformar las imágenes de entrada o generar nuevas imágenes o vídeos en función de las instrucciones recibidas. Hay que destacar que procesar o generar imágenes es un proceso computacionalmente mucho más complejo que procesar instrucciones textuales, por lo que el uso de estas capacidades suele estar más limitado en los modelos multimodales.

18. Redes generativas adversativas

En algunas situaciones buscamos sistemas de IA que produzcan resultados lo más ajustados posible a cierto estándar. Por ejemplo, podemos estar interesados en generar caras sintéticas que sean indistinguibles de caras reales, sintetizar voces con el timbre y el ritmo de una conversación natural entre dos personas o crear ilustraciones que sigan el estilo de un determinado dibujante. Sin embargo, el nivel de ajuste del resultado frente al estándar esperado es una cualidad muy subjetiva que requeriría disponer de personas a las que preguntarles “¿esta cara parece real?” o “¿consideras que esta ilustración ha sido dibujada por Hergé?”. Esta situación recuerda al test de Turing: necesitamos desarrollar un sistema de IA que genere resultados de tan alta calidad que sean capaces de engañar al criterio de un ser humano.

Basándose en esa idea de confrontación entre un sistema que genera resultados y una persona que debe validarlos, surge la idea de las redes generativas adversativas (GAN, Generative Adversarial Networks). Estos sistemas, propuestos en 2014 por Ian Goodfellow y sus colegas de la Universidad de Montreal (Canadá), están basados en dos redes neuronales enfrentadas (de ahí el nombre de “adversativas”) que se entrenan a la vez. Por un lado, existe una red generadora cuyo objetivo es crear nuevos resultados sintéticos dentro del dominio esperado (imágenes de caras, ficheros de audio con voces, ilustraciones, etc.) con el único objetivo de engañar a la red discriminadora. Por otro lado, la red discriminadora recibe tanto resultados reales (del conjunto de entrenamiento original) como resultados sintéticos generados por la red generadora y su tarea es aprender a detectar con la mayor precisión aquellos que son sintéticos. La red discriminadora sustituye al ser humano del test de Turing, permitiendo que todo el proceso sea automatizable y se pueda realizar de manera eficiente usando GPU. Estas dos redes participan en un proceso iterativo de entrenamiento. Primero la red generadora produce unos cuantos resultados sintéticos, que se mezclan con resultados reales y se envían a la red discriminadora para que determine si son reales o sintéticos. Al principio del entrenamiento los resultados sintéticos serán fácilmente distinguibles, por lo que la red discriminadora acertará muy a menudo, y durante el proceso esta precisión irá descendiendo. Tanto si el discriminador acierta como si falla al detectar un resultado sintético,

se puede mejorar una de las dos redes. Cuando la red discriminadora acierta al detectar un resultado sintético, se determina qué aspectos fueron los que más pesaron en esa decisión para que la red generadora pueda mejorar en futuras ocasiones. Cuando la detección de un resultado sintético falla, la red que se mejora es la propia red discriminadora. Como tanto la generadora como la discriminadora son redes neuronales, esta mejora se mide como errores que luego son distribuidos entre los pesos de la red usando el mecanismo de retropropagación. Este proceso iterativo de generación, discriminación y mejora se repetirá varias veces. En cada ciclo, la red generadora aprenderá a crear resultados cada vez más cercanos al estándar esperado y la red discriminadora mejorará su precisión al detectarlos, hasta alcanzar un punto en el que la red generadora logre engañar a la red discriminadora la mayor parte del tiempo. Una vez que se ha llegado a ese punto, se puede desechar la red discriminadora, porque ya no es necesaria: el sistema de IA utilizará únicamente la red generadora resultante del entrenamiento.

Las redes generativas adversativas han sido muy utilizadas en diversos ámbitos. Por ejemplo, sirven para generar retratos, paisajes y objetos fotorrealistas, lo que abre nuevas vías en publicidad, diseño gráfico y las industrias del cine y del videojuego. También se emplean para mejorar la resolución de imágenes borrosas, mejorar fotografías antiguas o ampliar fotogramas de vídeo. Dentro del ámbito audiovisual, pero con una vertiente menos ética, también permiten la manipulación facial que se utiliza en los deepfakes. Por último, su capacidad de generar resultados realistas también se ha utilizado para multiplicar los casos de entrenamiento de otros sistemas de IA en circunstancias donde obtener nuevos casos de prueba es complicado. Por ejemplo, en investigación médica, las redes generativas adversativas ayudan a simular resonancias o tomografías sin necesidad de exponer a los pacientes a más radiación, mientras que en los campos de la robótica y la conducción autónoma también permiten generar escenarios sintéticos con, por ejemplo, condiciones meteorológicas extremas que resultarían costosas o peligrosas de capturar en la vida real.

19. Los transformadores

Los transformadores (transformers) son un tipo de modelos de deep learning que surge en 2017 a raíz del célebre artículo científico “Attention Is All You Need”, escrito por investigadores de Google. El principal problema que trataban de resolver era que las redes neuronales recurrentes (RNN), aquellas que trabajan con la entrada elemento a elemento de manera secuencial y tienen cierta memoria acerca de los elementos anteriores para construir un contexto, tenían un coste computacional muy alto. Además, si las secuencias eran muy largas, esa memoria se iba debilitando hasta que era incapaz de relacionar un elemento con un antecesor muy anterior, lo que empeoraba sus resultados. Hasta ese momento, las RNN eran las redes que mejores resultados obtenían en la generación de lenguaje o la traducción debido a la naturaleza contextual del lenguaje, por lo que estos investigadores buscaban una solución más eficiente y que obtuviese resultados de igual o superior calidad. Para ello, en lugar de generar la salida introduciendo las palabras una a una, querían encontrar una solución que aceptase la frase completa de golpe y fuera capaz de inferir qué palabras estaban relacionadas unas con otras.

Para resolver estos problemas, los transformadores aplican una técnica muy potente llamada atención. Este mecanismo se basa en que cada palabra (o elemento de entrada) pueda tener en cuenta otros elementos de la frase y determinar cuál es el grado de relación que tiene con ese otro elemento. Este mecanismo de atención se aprecia en frases donde es necesario resolver ciertas dependencias contextuales. Por ejemplo, en la frase “María abrazó a Laura porque estaba llorando”, el modelo necesita prestar más atención a “Laura” al procesar “estaba llorando”, dado que habrá aprendido que es mucho más probable que quien esté llorando sea quien está siendo consolado. De manera similar, este proceso se aplica a la resolución de pronombres para que estos presten atención al sustantivo al que sustituyen. Por ejemplo, en la frase “Pedro habló con Juan mientras él arreglaba el coche”, el transformador debe decidir a qué palabra prestar atención al procesar “él”. En este caso, lo usual es que el pronombre “él” se refiera a “Juan”, aunque ciertamente se trata de una frase que presenta cierta ambigüedad. Por tanto, el mecanismo de atención permite que cada palabra de una frase preste atención a las demás para decidir cuánto

influyen en ella, lo que es imprescindible al analizar textos, porque las palabras de una frase a menudo tienen significados que dependen de otras palabras del contexto.

En los modelos de tipo transformador podemos distinguir dos partes principales: el codificador (encoder) y el decodificador (decoder). El codificador es el encargado de leer y comprender el contenido de entrada, como por ejemplo una frase en inglés que queremos traducir. No se limita a entender cada palabra por separado; como hemos comentado, tiene en cuenta el contexto completo usando la atención, analizando cómo se relacionan las palabras entre sí para captar el significado global. Esa información procesada se pasa al decodificador, que es el responsable de generar la salida palabra a palabra, por ejemplo, su traducción al español. Un aspecto interesante es que el decodificador no trabaja únicamente con lo que ya ha generado hasta el momento y sus posibles relaciones, sino que además utiliza un mecanismo adicional de atención cruzada (cross-attention). Este mecanismo le permite, en cada paso, consultar la información generada por el codificador y decidir en qué partes de la frase original debe fijarse más en cada momento. Por ejemplo, al generar una palabra concreta de la traducción, puede centrarse en la parte de la frase de entrada que más le ayudaría a encontrar la palabra del idioma destino más cercana desde el punto de vista semántico. Así, el decodificador combina lo que ha generado hasta el momento junto con la comprensión previa del codificador, logrando resultados más precisos y mejor contextualizados. Esta estructura hace que los transformadores obtengan resultados de muy alta calidad en tareas relacionadas con el lenguaje natural, destacando en tareas como la traducción, el resumen de textos, la escritura automática, la conversación en general e incluso la transcripción de voz a texto.

Además del aumento en la calidad de la salida generada, otra de las grandes ventajas de los transformadores frente a las anteriores redes de aprendizaje profundo como las RNN es que muchos de sus cálculos se pueden realizar en paralelo durante el entrenamiento, lo que los hace mucho más eficientes. En las RNN, las palabras o elementos de una secuencia se procesan uno a uno, en orden, ya que cada paso depende del anterior. Esto impide aprovechar bien los recursos de cómputo modernos como las GPU, que funcionan mejor cuando pueden realizar muchas operaciones similares en paralelo. En cambio, los transformadores procesan toda la secuencia de manera simultánea, gracias a su mecanismo de atención, que no requiere recorrer las palabras en orden. En otras palabras, todas las palabras de una frase pueden “atender” a todas las demás al mismo tiempo, lo que permite dividir el trabajo entre múltiples unidades de

procesamiento. Esta capacidad de paralelización ha sido clave para entrenar modelos muy grandes en tiempos razonables y con grandes cantidades de datos, y ha contribuido en gran parte al éxito del deep learning moderno basado en transformadores, llegando incluso a desplazar a casi todos los demás tipos de redes neuronales profundas.

Desde 2017 hasta hoy, los modelos de tipo transformador han crecido en complejidad y tamaño, pasando de millones a billones de parámetros. Uno de los primeros modelos que surgió fue BERT (Bidirectional Encoder Representations from Transformers), con 340 millones de parámetros. Fue desarrollado por Google en 2018 y únicamente utiliza la parte de codificadores del modelo transformador. Está principalmente centrado en tareas de comprensión de texto como el análisis de sentimiento o la detección de entidades mencionadas en textos, tareas que explicaremos más adelante. En 2019 se presenta GPT-2 (Generative Pre-trained Transformer) por parte de OpenAI. Este modelo utiliza únicamente los decodificadores del modelo transformador, pero aumenta su tamaño hasta alcanzar los 1.500 millones de parámetros. Los modelos GPT están diseñados principalmente para la generación de texto, por eso han sido la base de sistemas de interacción como ChatGPT. En 2020, OpenAI mejora su modelo presentando GPT-3, con 175.000 millones de parámetros que necesitaron ser entrenados con una colección de textos con cerca de 300.000 millones de palabras. También en los años 2019 y 2020 Google Research presenta el modelo T5 (Text-To-Text Transfer Transformer). Este modelo utiliza la arquitectura codificador-decodificador típica de los transformadores y aborda de manera satisfactoria varias tareas de procesamiento del lenguaje natural tales como traducción, clasificación o resumen. La variante más potente de T5, llamada T5-11B, tiene 11.000 millones de parámetros, que fueron entrenados con los con 750 GB de texto del conjunto de datos llamado C4 (Colossal Clean Crawled Corpus). Aunque no se ha informado del tiempo necesario para su entrenamiento, usaron más de 2.000 unidades de cómputo específico para tensores (TPU) y se sospecha que necesitaron varias semanas.

En los últimos años han surgido varios modelos basados en transformadores, como Claude de Anthropic, Mistral, Gemini de Google o Llama de Meta. Es difícil encontrar información precisa acerca de los tamaños y tiempos de entrenamiento de estos modelos, ya que se trata de secretos industriales que las empresas ocultan con mucho celo. Sin embargo, un modelo abierto presentado a finales de 2024 nos puede dar una pista del tamaño y complejidad de los modelos actuales. DeepSeek es un modelo basado en la parte decodificador de la

arquitectura de transformadores creado por la empresa china DeepSeek. Este modelo tiene 671.000 millones de parámetros y para su entrenamiento se necesitaron casi tres millones de horas de cómputo en GPU de NVIDIA modelo H800, con un coste de unos 5,5 millones de dólares y un conjunto de entrenamiento de unos billones de palabras. Para presentarlo en términos más comprensibles, si tuviésemos un centro de datos con 1.000 GPU de este tipo trabajando en paralelo, necesitaríamos unos cuatro meses para completar el entrenamiento, y el conjunto de entrenamiento ocuparía unos 15 TB (unos 15.000 GB). Se sospecha que modelos recientes como GPT-4 podrían tener hasta un billón de parámetros, lo que supondría un coste ingente, tanto monetario como en tiempo necesario, para realizar su entrenamiento.

Como comentario final, hay que mencionar que para mejorar el rendimiento de los sistemas basados en redes neuronales de un tamaño muy grande, utilicen o no transformadores, en los últimos tiempos se suele recurrir a la técnica de mezcla de expertos (MoE, Mixture of Experts). En esta técnica, el diseñador prepara una red neuronal dividida en varios “expertos”, es decir, redes neuronales más pequeñas que se pueden especializar en tareas específicas. Durante el entrenamiento, un componente llamado enrutador aprende a decidir qué expertos son los más adecuados para cada entrada. De esta manera, el modelo aprende de manera automática tanto lo que deben hacer los expertos como cuándo usarlo, sin intervención directa del diseñador para definir manualmente qué experto se encarga de cada tarea. Una vez entrenado, cada vez que llegue una petición, el enrutador decidirá qué subconjunto de expertos deberá procesarla, dejando inactivos al resto. Así se consigue que el modelo tenga muchos más parámetros (lo que aumenta su capacidad para aprender), pero que únicamente use una pequeña parte en cada operación, lo que reduce enormemente el coste de cálculo. De entre los LLM actuales, se tiene constancia de que DeepSeek usa la técnica MoE. Respecto a los modelos de OpenAI, existe el consenso en la comunidad de que a partir de GPT-4 también se usa MoE, aunque no hay confirmación oficial de OpenAI al respecto.

20. Aprendizaje por transferencia y ajuste fino

Como ya hemos visto, los grandes modelos de deep learning necesitan una inmensa cantidad de ordenadores y tarjetas gráficas que trabajen a la vez para ser entrenados. Este proceso requiere fácilmente de cientos o miles de horas de cómputo, lo que genera un gran gasto, bien en alquiler de infraestructura informática, bien en compra de equipamiento y en su consumo eléctrico. Una vez completada esta tarea, nos podemos dar cuenta de que el modelo generado es adecuado, pero muy general, y que nos gustaría tener versiones especializadas para diferentes temáticas. Por ejemplo, nos gustaría disponer de un asistente legal general, pero también versiones especializadas en derecho tributario y en derecho mercantil. ¿Es realmente necesario entrenar una gran red neuronal desde cero para cada una de estas dos tareas específicas?

Para agilizar tareas como estas, se aplica el aprendizaje por transferencia. Esta técnica consiste en reutilizar el conocimiento adquirido por un modelo al resolver una tarea para aplicarlo en otra tarea diferente, pero relacionada. De esta manera, se consigue reducir el tiempo de entrenamiento y la necesidad de grandes cantidades de datos en la nueva tarea. El aprendizaje por transferencia no se limita al aprendizaje profundo o las redes neuronales, sino que es una técnica general de la IA que se podría aplicar a distintas técnicas de aprendizaje.

Un ejemplo de aprendizaje por transferencia sería reutilizar un modelo centrado en tareas de comprensión de texto, como el conocido por las siglas BERT, y sobre él construir un modelo más específico. Por ejemplo, imaginemos que queremos crear un clasificador de mensajes para discriminar entre los publicitarios y los laborales. Como ya hemos visto, BERT es un modelo basado en transformadores que ha sido entrenado con grandes cantidades de texto y, por tanto, ha adquirido un conocimiento muy detallado sobre el lenguaje, siendo capaz de analizar el significado de las frases con una alta precisión. Para reutilizar dicho conocimiento, podríamos añadir un nuevo componente clasificador a la salida de BERT, por ejemplo, una pequeña red neuronal, y realizar un nuevo entrenamiento con nuevos casos de ejemplo (mensajes publicitarios y mensajes laborales) para entrenar únicamente la red de clasificación. En este caso estaríamos “congelando” completamente todos los

parámetros de BERT, que permanecerían tal cual fueron calculados en su entrenamiento original, y usaríamos los nuevos ejemplos únicamente para calcular los pesos de la red neuronal clasificadora y tratar de maximizar su capacidad discriminadora. Como esta red neuronal clasificadora será muy pequeña (en comparación con el tamaño de un modelo como BERT), el tiempo y el gasto necesario para realizar este aprendizaje por transferencia será muy contenido, a la vez que el resultado obtenido será muy preciso, porque estaremos reutilizando todo el conocimiento lingüístico contenido en BERT.

Un tipo concreto de aprendizaje por transferencia que se utiliza mucho en modelos basados en redes neuronales es el ajuste fino (fine tuning). Esta técnica también parte de un modelo de IA previamente entrenado y persigue adaptarlo a una tarea diferente y relacionada. Para ello, aplica un proceso de aprendizaje adicional al modelo previamente entrenado en el que utiliza nuevos ejemplos específicos de la nueva tarea, pero, a diferencia del ejemplo anterior, no “congela” los parámetros del modelo original. De esta manera, se consigue partir del conocimiento previo, pero no para únicamente reutilizarlo, sino para adaptarlo a la nueva tarea. Siguiendo el ejemplo anterior, podríamos realizar ajuste fino de un modelo BERT para que distinguiera entre mensajes publicitarios y laborales. En este caso, el modelo BERT iría adaptando sus parámetros durante el aprendizaje de los nuevos ejemplos e iría detectando pequeños detalles entre palabras o matices de significado que surgen únicamente al considerar los nuevos mensajes de entrenamiento. Como el modelo BERT al que estaríamos aplicando el ajuste fino tiene un tamaño grande, en esta ocasión el entrenamiento requeriría un tiempo de cálculo más elevado que el usado en entrenar únicamente un componente clasificador, pero en cualquier caso este tiempo sería mucho menor que el necesario para entrenar el modelo BERT especializado desde cero.

Como comentario final, hay que tener en cuenta un fenómeno que ocurre cuando se realiza ajuste fino sobre un modelo: el conocido como olvido catastrófico. Este término se refiere a que a veces un modelo de IA puede perder parte o la totalidad del conocimiento previamente adquirido al ser ajustado a nuevas tareas. Además, este problema no es fácil de detectar, ya que el modelo ajustado no emite ningún aviso indicando que ha “olvidado” conocimientos previos, sino que el desarrollador observa que los resultados obtenidos tras el ajuste fino no son tan precisos como esperaba. Para mitigar este problema, se puede realizar el ajuste fino usando un entrenamiento combinado, es decir, incluir ejemplos antiguos junto con los nuevos para que durante este nuevo aprendizaje no llegue

nunca a olvidar el conocimiento anterior (o al menos para que olvide lo menos posible).

21. Procesamiento del lenguaje natural

El procesamiento del lenguaje natural (PLN o NLP) es una rama de la inteligencia artificial que permite a las máquinas entender, interpretar, generar y manipular el lenguaje humano. Ejemplos cotidianos de este tipo de procesamientos son los asistentes virtuales como Siri o Alexa, que son capaces de reconocer nuestra voz, realizar la petición solicitada y generar una respuesta también como voz. El PLN también incluye a los traductores automáticos, que deben reconocer texto en un idioma y generarlo en otro, o los autocorrectores, que se encargan de analizar una frase escrita para detectar posibles errores. Se conoce como lenguaje natural al lenguaje que usamos los seres humanos para comunicarnos y que surge de manera espontánea, como el español o el inglés. Esto lo diferencia de otros lenguajes creados por las personas para fines más específicos, como los lenguajes formales usados en álgebra o lógica, o los lenguajes de programación, que tienen una sintaxis simplificada y cuyas normas para formar frases correctas son muy estrictas. La alta complejidad de los lenguajes naturales (como la ambigüedad, la ironía, los dobles sentidos, la alta dependencia del contexto o su rápido cambio y evolución) hizo pensar que la inteligencia artificial nunca podría llegar a manejarlos con soltura, pero esta limitación ha sido refutada por los grandes modelos de lenguaje actuales como ChatGPT o Gemini.

Dentro del ecosistema de la inteligencia artificial, la rama del PLN utiliza muchos elementos del aprendizaje automático en general y también del deep learning en particular, además de algunas técnicas específicas de la rama. De hecho, los sistemas de PLN más potentes que se han creado hasta el momento se basan fuertemente en la arquitectura de transformadores que discutimos en otro capítulo, pero aplican ciertos pasos comunes para simplificar el texto que reciben y mejorar sus resultados.

El primer paso a la hora de procesar lenguaje natural es el análisis léxico, también llamado tokenización. Este proceso detecta los fragmentos mínimos de los que está compuesto un texto, los tokens, que suelen ser palabras o fragmentos de palabras, signos de puntuación o símbolos. Por ejemplo, el análisis léxico se encargaría de trocear la frase “La inteligencia artificial es el

futuro, creo,” en una secuencia de tokens [“La”, “inteligencia”, “artificial”, “es”, “el”, “futuro”, “,”, “creo”], donde la propia coma ha sido detectada como un token.

Una vez que se dispone de una lista de tokens, se pueden eliminar las llamadas palabras vacías (stop words). Se utiliza este término para referirse a palabras muy frecuentes en un idioma y que, por sí solas, suelen aportar poco significado al analizar el contenido de un texto. Ejemplos comunes de palabras vacías en español son los artículos como “la” y “una”, pronombres como “ella” o “nosotras”, e incluso preposiciones como “para” y “por”. Su eliminación suele ayudar a reducir el ruido en los textos, lo que permite enfocarse en las palabras clave que aportan más significado, a la vez que ayuda a disminuir el tamaño de los datos. En la frase anterior únicamente se detectarían y eliminarían las palabras vacías “La” y “el” de la secuencia, en ambos casos artículos.

Finalmente, se aplica un proceso de unificación de palabras derivadas. Existen distintas técnicas para realizar esta unificación, pero nos centraremos en la técnica conocida como lematización. Este proceso sustituye palabras por su forma base o lema, que será un token que actuará como representante de todas sus palabras derivadas. Un ejemplo claro de lematización es la eliminación de los tiempos verbales como “es”, “soy” o “seré”, que son sustituidos por el lema común “ser”, en este caso, su infinitivo y que contiene la información fundamental de la acción perdiendo la parte modal. Tras aplicar esta última etapa de lematización a la frase original, terminaríamos con la siguiente secuencia de tokens: [“inteligencia”, “artificial”, “ser”, “futuro”, “,”, “creer”]. A partir de estas secuencias de tokens preprocesadas, los sistemas de PLN comienzan sus análisis con el objetivo de entender el texto y resolver distintas tareas. En muchos casos, estos sistemas aplican pasos adicionales para transformar las secuencias de tokens en representaciones vectoriales que puedan ser procesadas por redes neuronales, como veremos más adelante.

Existen distintas tareas que los sistemas de PLN pueden resolver. Entre las tareas más sencillas destaca el análisis de sentimiento, que recibe un texto y determina, de forma automática, su sentimiento general: positivo, neutro o negativo. Este análisis es muy útil, por ejemplo, para procesar de manera automática grandes cantidades de opiniones o mensajes de usuarios como las que aparecen en redes sociales y tener una visión general de cuál es su posición sobre un tema o producto. El análisis de sentimiento se suele complementar con otra tarea de PLN que sirve para reconocer entidades nombradas en un texto. Por ejemplo,

este reconocimiento determinaría que en la frase “Las calles de Madrid están muy limpias” aparece la entidad “Madrid” y que además es un lugar, lo que permitiría asociar el sentimiento positivo a esa entidad. El reconocimiento de entidades nombradas también es capaz de detectar nombres de personas (“Miguel de Cervantes”), organizaciones (“ONU”), fechas (“12 de julio”) o valores monetarios (“100 €”), entre otros tipos.

Otro gran grupo de tareas de PLN son los clasificadores. Se pueden entrenar clasificadores de texto para fines específicos, como puede ser clasificar emails entre spam o mensaje legítimo, o catalogar de manera automática noticias de periódicos en función de su temática (negocios, política, deportes, etc.). Dentro de la categoría de clasificadores destaca una técnica muy versátil llamada clasificación zero-shot, que permite clasificar textos en categorías sin haber recibido ejemplos previos de esas categorías durante el entrenamiento. En lugar de aprender a partir de ejemplos etiquetados, el modelo usa el lenguaje (y su comprensión del significado) para determinar, de entre todas las etiquetas propuestas, con cuáles está más relacionado un determinado texto. De esta manera, podríamos usar un mismo clasificador zero-shot para clasificar la frase “Las calles de Madrid están muy limpias” entre las etiquetas candidatas [“política”, “urbanismo”, “deportes”, “sociedad”], donde detectaría que las etiquetas “política” y “urbanismo” son las más cercanas; mientras que con otras etiquetas candidatas, como [“España”, “Chile”, “Europa”, “Asia”] detectaría que “España” y “Europa” son las más relevantes. En ambos casos, el clasificador zero-shot produciría una serie de porcentajes para indicar cómo de relevante considera que es cada etiqueta.

Para terminar con los ejemplos de tareas de PLN, podemos incluir todas aquellas centradas en la generación automática de texto. Entre estas tareas destacan la traducción automática, que convierte textos de un idioma a otro conservando su significado; la generación de resúmenes, que condensa documentos largos en versiones más breves y comprensibles, y la compleción de texto, que predice o sugiere continuaciones coherentes a partir de una frase inicial. Entre estas tareas de generación automática de texto destacan los grandes modelos de lenguaje y sus chatbots como ChatGPT o Gemini, que pueden entenderse como sistemas que generan texto a partir de las instrucciones de los usuarios y el resto de información contextual que ha aparecido a lo largo de la conversación.

22. Representaciones vectoriales

Los modelos deep learning de IA, incluyendo los transformadores que tan buenos resultados han conseguido en los últimos años, no son más que redes neuronales artificiales de gran tamaño. Las neuronas artificiales reciben secuencias de valores numéricos por sus entradas y generan una salida usando su función de activación, que a su vez también debe ser un valor numérico. Sin embargo, muchos de los modelos que usamos (por ejemplo, GPT o Claude) no reciben valores numéricos, sino texto normal y corriente, es decir, una secuencia de palabras formadas a su vez por letras. ¿Cómo se realiza esta adaptación de textos a secuencias de valores numéricos? Para ello, se utiliza un embedding, un mecanismo que traduce textos a vectores numéricos que ya pueden ser procesados por los modelos de IA. Un vector no es más que una secuencia de valores numéricos de un tamaño prefijado, donde dicho tamaño se conoce como el número de dimensiones. Los vectores se representan entre paréntesis separados por comas, por ejemplo, el vector de tres dimensiones (1.5,6.3,0.1).

Los embeddings se aplican después de las fases de análisis léxico para obtener los tokens de un texto, la eliminación de palabras vacías y la lematización, tal y como hemos visto en el apartado sobre el procesamiento del lenguaje natural. Un ejemplo de técnica de embedding muy básica es la conocida como bolsa de palabras: asignar a cada palabra del vocabulario una posición entre las dimensiones disponibles y traducir los textos como vectores de unos y ceros para indicar si una palabra aparece (valor 1) o no aparece (valor 0) en dicho texto. Por ejemplo, supongamos que contamos con un vocabulario muy limitado formado por las palabras “el”, “perro”, “salta”, “mucho”, “gato”, “brinca” y “bastante”, en ese orden (7 dimensiones). Una frase como “el perro salta mucho” se representaría con el vector (1,1,1,1,0,0,0), mientras que la frase “el gato brinca demasiado” se representaría como (1,0,0,0,1,1,1). Como se ve, la representación de ambas frases es muy diferente, porque las palabras son distintas, aunque el significado es muy similar: “la mascota salta mucho”. Además, frases con las mismas palabras en orden distinto generarían la misma representación cuando la organización de palabras puede cambiar el significado. Por ejemplo, no es lo mismo “Eva busca a Ana” que “Ana busca a Eva”, aunque su representación en vector de bolsa de palabras será la misma.

Para resolver este problema, surgieron los embeddings de contexto fijo como Word2Vec (Google, 2013) o GloVe (Stanford, 2014). Estos embeddings son sistemas de aprendizaje automático que procesan grandes cantidades de texto y aprenden cómo representar cada una de las palabras. Este aprendizaje se basa en una idea fundamental: las palabras que siempre se encuentran en frases parecidas deben tener una representación similar como vectores. Un aspecto muy interesante de estos embeddings es que como resultado de su aprendizaje detectan dimensiones semánticas latentes, lo que podríamos llamar conceptos, sin haber sido programados específicamente para ello. Por ejemplo, una palabra como “rey” se podría representar como el vector (1,0), “reina” como (1,1), “señora” como (0,1) y “princesa” como (0.7,0.8). Es decir, gracias a haber sido entrenado con muchos textos, ha sido capaz de generar una representación en la que la primera dimensión indica el grado de realeza (0 para palabras nada relacionadas con la monarquía, 1 para palabras muy relacionadas con la monarquía), mientras que la segunda dimensión indica el género (0 para hombres y 1 para mujeres), de forma que “princesa” es expresada como un término muy relacionado tanto con monarquía como con género femenino. Gracias a este método, palabras diferentes pero con un significado similar tendrán representaciones similares, por ejemplo, “monarca” como (1,0.5) o “soberano” como (0.8,0), ambos cercanos a “rey” con vector (1,0). Estos embeddings también nos permiten realizar cierto razonamiento sobre las representaciones de las palabras. Por ejemplo, podemos tomar el vector de “reina” (1,1) y restarle el vector de “señora” (0,1), para obtener la representación de “rey” (1,0). También podemos determinar si el significado de una palabra es cercano al significado de otra simplemente calculando el ángulo que forman sus representaciones vectoriales. Si las palabras son semánticamente cercanas, sus vectores apuntarán en direcciones similares y formarán un ángulo cercano a 0° , mientras que si tienen significado muy diferente, los vectores formarán ángulos prácticamente perpendiculares. Por ejemplo, gracias a esta técnica podríamos determinar que “reina” (1,1) y “princesa” (0.7,0.8) tienen un significado muy similar, porque forman un ángulo de $3,8^\circ$, mientras que “reina” (1,1) y “señora” (0,1) tienen un significado más lejano, porque forman un ángulo de 45° , como se puede apreciar en la siguiente figura:

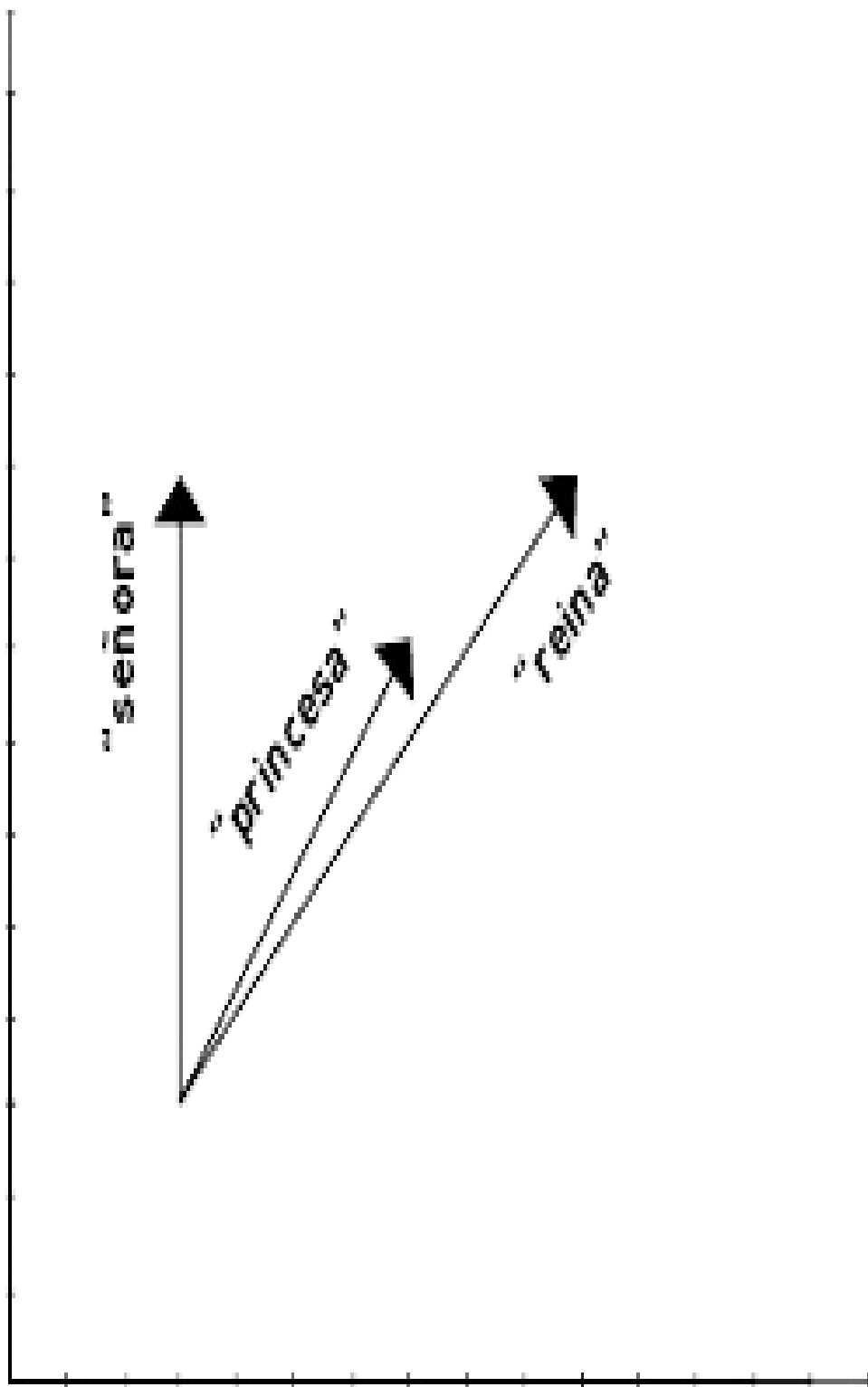
Representación vectorial de palabras usando dos dimensiones:

realeza (eje horizontal) y género (eje vertical)

Género

"señora"
"princesa"
"reina"

Realeza



Fuente: Elaboración propia.

En los ejemplos anteriores hemos considerado únicamente dos dimensiones para facilitar la comprensión, pero los sistemas reales utilizarán cientos e incluso miles de dimensiones, en las cuales cada una expresa un matiz semántico diferente.

A partir de la representación de cada palabra, los embeddings se pueden extender para procesar frases completas e incluso documentos enteros, que ya pueden alimentar directamente a las redes neuronales. Como capacidad adicional, los embeddings nos permiten razonar sobre la similitud semántica entre textos. Dado que cada texto tiene una representación en forma de vector, y que los vectores representan direcciones en el espacio, el significado de dos textos será muy similar si sus vectores apuntan en direcciones cercanas, y tendrán un significado muy diferente si sus vectores apuntan en direcciones alejadas. La capacidad de razonamiento semántico es muy útil, por ejemplo, para realizar búsquedas en documentos de texto. Dada una consulta como “goles final mundial fútbol 2010”, podríamos obtener su representación vectorial usando un embedding y buscar, dentro de la colección de documentos que tenemos, cuáles son los cinco o seis cuyos vectores son más cercanos. Estos documentos con vectores próximos tendrán una temática cercana y posiblemente incluyan información del mundial de fútbol de 2010 y sus goles.

Los embeddings de contexto fijo como Word2Vec o GloVe aún tienen una limitación y es que representarán a la misma palabra de la misma manera, independientemente del contexto en el que aparezca. Esto genera dificultades con las palabras polisémicas como “banco”, que puede ser una entidad financiera, un asiento o un grupo de peces, entre otros significados. Para resolver este problema, se han sofisticado los embeddings dando lugar a los embeddings contextualizados, que son capaces de generar representaciones vectoriales de palabras que varían según el contexto en el que estas aparecen. Es decir, la misma palabra puede tener distintos vectores si aparece en frases distintas. De esta manera, el vector asociado a la palabra “banco” en “¿nos sentamos en ese banco?” será totalmente diferente que la representación vectorial de “banco” en “los fondos de inversión de los bancos son lo peor”. Para ser capaces de utilizar el contexto a la hora de encontrar una buena representación de una palabra, estos

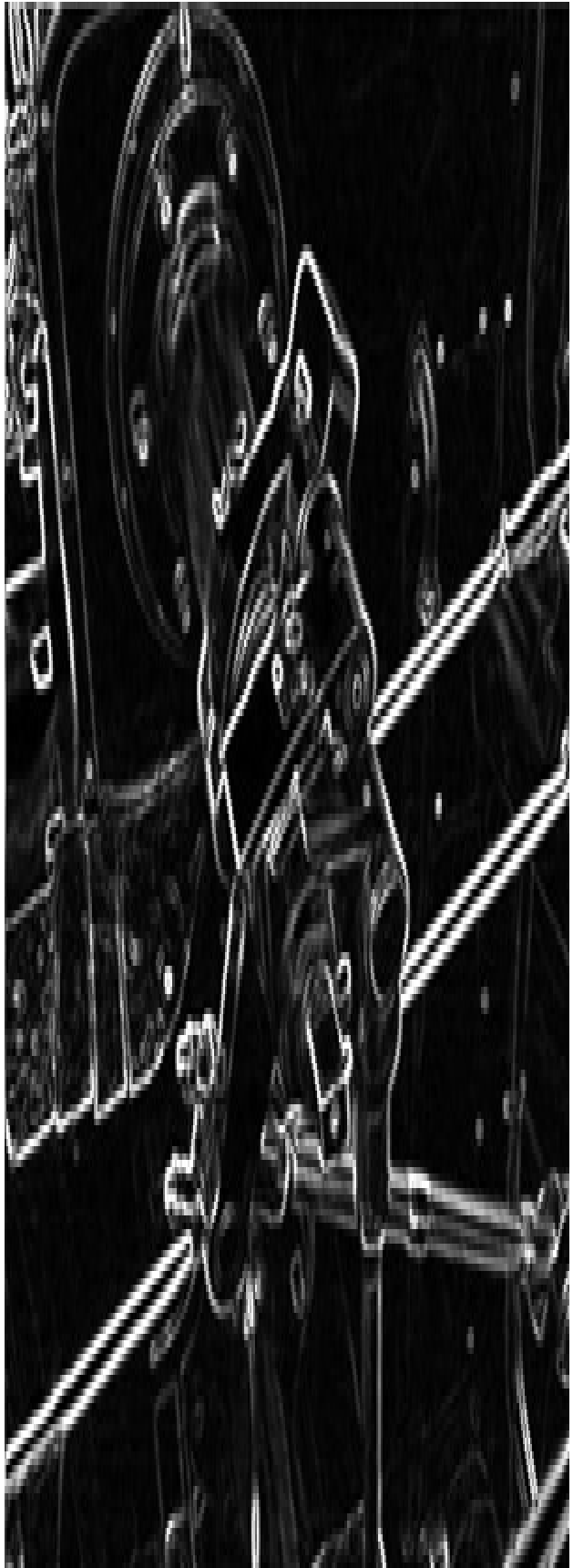
embeddings se basan en transformadores para así utilizar la gran potencia de sus mecanismos de atención, principalmente los codificadores que consideran tanto las palabras anteriores como posteriores. Ejemplos de embeddings contextuales basados en transformadores son BERT (Google, 2018) y sus extensiones, como RoBERTa (Facebook, 2019), ALBERT (Google y Toyota, 2019) o DistilBERT (Hugging Face, 2019).

23. Visión artificial

Aunque en este libro nos hemos centrado más en las capacidades de la IA para procesar y generar textos, ya que desde el lanzamiento de ChatGPT es lo que más atención ha acaparado y lo que más aplicaciones potenciales parece ofrecer, las capacidades de la IA para procesar imágenes también han avanzado mucho en los últimos tiempos. En este apartado nos vamos a centrar en la visión artificial, también llamada visión por computadora, que es la rama de la IA que permite a los ordenadores interpretar información visual del mundo real a partir de imágenes o vídeos. Es decir, esta tecnología convierte los píxeles de una imagen, que no son más que una secuencia de puntos con determinados colores, en información comprensible y útil, como formas, orientaciones y movimiento. Sirve, por ejemplo, para que un coche autónomo detecte peatones y reconozca señales de tráfico, un programa médico identifique tumores en una radiografía o un dron inspeccione cultivos y localice plagas. La visión artificial es muy importante, porque gran parte de los datos del mundo son visuales, así que su utilización abre la puerta a automatizar algunas tareas antes reservadas a los humanos, lo que puede mejorar la seguridad y acelerar la toma de decisiones.

La evolución de la visión artificial ha transitado desde soluciones bastante manuales hasta el uso de redes neuronales profundas y transformadores. En los inicios la investigación se centró en algoritmos para la detección de bordes, como el operador Sobel, presentado en 1968. Este operador procesaba cada píxel de una imagen y, considerando la diferencia de intensidad con sus píxeles vecinos, determinaba si formaba parte de un borde. En la siguiente figura se observa el resultado del operador Sobel para detectar bordes en una imagen de una válvula:

Detección de bordes usando el operador Sobel



Fuente: Wikipedia.

A partir de la información de bordes por cada píxel, estos se agrupaban formando bordes más largos y se aplicaban reglas ad hoc para extraer información aproximada. Por ejemplo, si un borde vertical largo se juntaba con otro borde horizontal más corto, podrían formar la esquina de una puerta, o si aparecían varios bordes horizontales juntos separados por una distancia similar, quizá se trataba de una escalera.

Esta idea de algoritmos de detección de bordes fue mejorando progresivamente y en 2005 se propuso el algoritmo HOG (del inglés Histogram of Oriented Gradients). Entre las mejoras de HOG destaca que se centra en celdas de la imagen, es decir, grupos más grandes de píxeles cercanos y con cierto solapamiento, y que normaliza los resultados para restar importancia a la iluminación y a los picos de contraste que pueden aparecer en ciertas partes de la imagen. De esta manera obtiene mucha mayor robustez frente a ruido, sombras o cambios de escala. Además, para cada celda de píxeles genera una representación numérica, lo que permite obtener una representación vectorial de la imagen completa, simplemente concatenando las representaciones de las distintas celdas. Poder generar una representación vectorial de una imagen abre la puerta a la aplicación de cualquier algoritmo de aprendizaje automático, como podría ser una red neuronal, lo que permite obtener importantes mejoras en la clasificación de los objetos contenidos en imágenes.

Pero el principal paso adelante en la mejora de la visión artificial ocurrió en 2012 con la creación de AlexNet. Fue el primer sistema basado en redes neuronales convolucionales profundas que demostró que esa técnica podía mejorar en gran medida a todas las aproximaciones anteriores en tareas de visión artificial. Para su entrenamiento utilizó más de un millón de imágenes de la base de datos ImageNet, una colección de imágenes que han sido etiquetadas manualmente para indicar qué objetos aparecen en ellas. Para este entrenamiento necesitó utilizar dos GPU en paralelo y tardó entre cinco y seis días. El resultado de este entrenamiento fue un sistema que mejoró en gran medida los resultados existentes, concretamente, obtuvo un error cercano al 15% en la competición ImageNet de 2012, casi un 11% más bajo que el siguiente competidor. Basados en esta idea de redes neuronales convolucionales profundas surgieron otros

sistemas añadiendo mejoras técnicas, como GoogLeNet (2014, Google) o ResNet (2015, Microsoft Research).

Los últimos avances en visión artificial siguen ligados a las redes neuronales profundas, pero en este caso a los transformadores. Como ya hemos visto, los transformadores disponen de mecanismos de atención en sus codificadores y decodificadores, lo que, en el caso de textos, les permite relacionar unas palabras con otras a la hora de asignarles significado y dar un sentido a la frase completa. En 2020, Google Brain presentó el transformador de visión (ViT, Vision Transformer). La idea principal es eliminar las convoluciones utilizadas en redes neuronales convolucionales destinadas a detectar bordes y otras características locales y tratar directamente la imagen como si se tratase de una frase. Para ello divide la imagen en bloques de píxeles vecinos, por ejemplo, de 16 x 16, y trata cada uno de ellos como una “palabra”. De esta manera se pueden aplicar los mismos conceptos de atención de los transformadores, lo que permite que unas partes de la imagen afecten a la interpretación de otras a la hora de dar un significado global a la imagen y se consigan resultados que mejoran a las redes neuronales convolucionales clásicas en varios bancos de pruebas.

En la actualidad, dentro del campo de visión artificial conviven tanto modelos basados en redes neuronales convolucionales, que obtienen muy buenos resultados en algunas aplicaciones específicas, como los basados en transformadores, que suelen ser más flexibles en sus aplicaciones. No obstante, hay un movimiento hacia la adopción de modelos de IA con enfoque multimodal que usa transformadores. Llamamos multimodales a aquellos modelos capaces de procesar e integrar información proveniente de diferentes tipos de datos, como texto, imágenes, audio y vídeo, permitiendo así una comprensión más completa y contextualizada de la situación. De esta manera, se pueden proporcionar imágenes junto con peticiones textuales de alto nivel y obtener información precisa de manera sencilla. Por ejemplo, con estos modelos multimodales se puede incluir una fotografía de un dispositivo que no funciona y preguntar qué componente parece dañado y cómo se podría reparar en casa, o aportar fotos diarias de una planta que parece enferma y solicitar un análisis de la evolución de la planta para saber si necesita más luz, agua o algún otro tratamiento.

24. LLM: los grandes modelos de lenguaje

Los grandes modelos del lenguaje (LLM, Large Language Model) son sistemas de deep learning que han sido entrenados con grandes corpus de textos con el objetivo de que detecten con gran precisión los patrones y relaciones que existen entre las distintas palabras de los lenguajes que usamos las personas, y sean capaces de generar frases bien construidas y con sentido respecto a la información que han revisado. Antes de llegar a sistemas tan sofisticados como los asistentes conversacionales ChatGPT o Gemini, que responden a las peticiones que le expresamos mediante instrucciones (prompts), los desarrolladores deben llevar a cabo una serie de tareas que requieren una altísima capacidad computacional.

El primer paso para desarrollar un asistente conversacional completo es comenzar con un modelo base de lenguaje. El objetivo de este modelo base es que conozca las distintas particularidades del lenguaje y sea capaz de generarlo, pero todavía sin ningún comportamiento específico. Para obtener este modelo base, se utilizan arquitecturas deep learning (usualmente transformadores) de miles de millones e incluso billones de parámetros, que son entrenadas con una ingente cantidad de texto creado por humanos. Se trata de un proceso de aprendizaje supervisado, cuyo entrenamiento, como hemos visto, requiere de un gran conjunto de entrenamiento etiquetado, es decir, de una serie de ejemplos donde para cada entrada se conoce exactamente cuál es la salida esperada. Esto parecería obligarnos a movilizar a un gran número de personas expertas en los distintos idiomas para que seleccionen frases de entrada con su salida esperada en cada caso, con el consiguiente coste monetario. Para solventar este problema, la industria de la IA ha encontrado una solución barata, efectiva y que permite obtener grandes conjuntos de entrenamiento sin apenas esfuerzo: tomar el texto que la humanidad ya ha escrito a lo largo de la historia y generar casos de entrenamiento de manera automática a partir de ese texto.

A partir de un corpus de textos previamente escritos por personas, es sencillo generar una multitud de casos de entrenamiento para entrenar un modelo base de manera automática. Para ello, se usan principalmente dos técnicas. En la primera, se recibe el inicio de una frase y el modelo debe aprender a predecir la

siguiente palabra, mientras que en la segunda se fuerza al modelo a tratar de descubrir una palabra oculta en una frase completa. Por ejemplo, imaginemos que en uno de los documentos escritos por personas (en este caso, la obra teatral *Luces de bohemia*, de Valle-Inclán) encontramos la frase “Una crónica deliciosa como todas las suyas”. A partir de esta frase podríamos generar de manera automática varios casos de entrenamiento para enseñar el sistema a predecir la siguiente palabra, donde la “salida esperada” es la etiqueta que se usaría durante el aprendizaje supervisado:

Entrada: “Una crónica”, salida esperada: “deliciosa”.

Entrada: “Una crónica deliciosa”, salida esperada: “como”.

Entrada: “Una crónica deliciosa como”, salida esperada: “todas”.

Entrada: “Una crónica deliciosa como todas las”, salida esperada: “suyas”.

La segunda técnica consiste en partir de las mismas frases, pero enmascarando alguna palabra. El modelo deberá ser capaz de predecir la palabra que falta. Por ejemplo, si ocultamos la palabra con <> podríamos generar, entre otros, los siguientes casos de entrenamiento:

Entrada: “<> crónica deliciosa como todas las suyas”, salida esperada: “Una”.

Entrada: “Una <> deliciosa como todas las suyas”, salida esperada: “crónica”.

Entrada: “Una crónica <> como todas las suyas”, salida esperada: “deliciosa”.

Entrada: “Una crónica deliciosa como <> las suyas”, salida esperada: “todas”.

Es importante destacar como, a partir de una única frase, somos capaces de generar decenas de casos de entrenamiento, e incluso centenas si la frase es

medianamente compleja. Sumemos a eso que existen grandes cantidades de texto fácilmente accesibles en internet escritas por personas con distintos registros del lenguaje e idiomas, como por ejemplo en Wikipedia, en bibliotecas de libros digitalizados, en foros de preguntas como Reddit, artículos de periódicos y sus comentarios, etc. Por lo tanto, a partir de los textos disponibles en abierto se pueden generar enormes colecciones enormes de entrenamiento para obtener un modelo base de lenguaje usando deep learning.

Gracias al entrenamiento, el modelo base posee un conocimiento lingüístico muy amplio, que es capaz de generar frases bien construidas y muy coherentes, pero en cierta manera sigue siendo un modelo en bruto, sin pulir, y sin estar alineado aún con instrucciones humanas. Es decir, todavía no está optimizado para responder a nuestras instrucciones para resolver las tareas que le planteemos. El conocimiento lingüístico que posee el modelo base le permite ser capaz de encontrar las mejores palabras para continuar cualquier frase; de hecho, llevado al extremo, el modelo base no es más que un predictor estadístico de palabras. Por ejemplo, si se le introduce la frase “Una crónica deliciosa como”, el modelo base podría generar la distribución de posibles palabras siguientes como las que se aprecian en la siguiente figura:

Siguientes palabras para continuar un texto,

junto con sus probabilidades

"Una crónica
deliciosa como..."



"Un"

35%

"La"

20%

"esa"

10%

"una"

8%

Fuente: Elaboración propia.

Utilizando un modelo base es sencillo generar frases a partir de texto previo. Lo único que hay que hacer es tomar la frase parcial actual (que puede ser el texto inicial o prompt del usuario), usar el modelo base para generar las siguientes palabras más probables, escoger una de ellas para extender la frase parcial y repetir el proceso hasta que el modelo base detecte que la siguiente palabra puede ser el signo de puntuación que indica el final de la frase. Como en cada paso el modelo base nos genera una serie de palabras siguientes más probables, podemos elegirla con cierto indeterminismo y así generar frases diferentes a partir del mismo inicio. De esta manera, cada palabra elegida en un paso entra a formar parte de la frase parcial del paso siguiente, así que cada decisión tomada aporta divergencia adicional, dando lugar a frases completamente distintas. Es por esta razón por lo que modelos como ChatGPT generalmente no contestarán exactamente lo mismo aunque les repitamos la misma pregunta: cada vez elegirá al azar una palabra de entre la lista de palabras probables para la respuesta, y esta palabra a su vez determinará qué palabras pueden venir a continuación, orientando así la respuesta de una forma diferente. En algunos sistemas se puede configurar la tendencia a seleccionar las palabras más probables según el modelo base a través de un parámetro llamado temperatura. Si la temperatura es baja, el sistema elegirá siempre entre las palabras más probables, pero según se aumenta la temperatura el sistema tendrá permitido elegir palabras siguientes con cada vez menor probabilidad. Una temperatura alta puede entenderse como mayor originalidad a la hora de generar frases, pero temperaturas demasiado altas generan fácilmente frases incoherentes o que incluso mezclan idiomas en sus palabras.

Los modelos base han aprendido muy bien el lenguaje y lo pueden generar de manera muy fidedigna, pero no están especializados aún en la tarea principal que buscamos en un LLM: recibir instrucciones del usuario y generar una respuesta adecuada. Por ello, los modelos base suelen pasar por una fase adicional llamada ajuste de instrucciones (instruction tuning). Esta etapa afina el comportamiento del modelo para que entienda mejor y responda de forma más útil a las instrucciones que le dan los usuarios, acercándolo a lo que uno esperaría de un asistente conversacional. A diferencia de entrenar un modelo para una tarea específica y limitada (como clasificar si un texto transmite alegría o tristeza), el

ajuste de instrucciones lo prepara para abordar una gama mucho más amplia de situaciones, enseñándole a generar la respuesta más adecuada a cualquier petición que reciba. En otras palabras, se entrena al modelo para entender y cumplir instrucciones de forma flexible, abierta y contextual, lo que multiplica su utilidad práctica.

Finalmente, para mejorar aún más la calidad de los LLM, se suele realizar una etapa de aprendizaje por refuerzo a partir de retroalimentación humana. En este enfoque, se generan varias respuestas posibles a una misma instrucción, y personas humanas se encargan de clasificarlas o puntuarlas según su calidad. Con esta información, el LLM aprende a valorar qué respuestas se alinean mejor con lo que los humanos consideran útil o apropiado. Esto también ayuda a detectar y omitir respuestas polémicas o consideradas peligrosas, como la fabricación de armas, la difusión de discursos de odio o la promoción de conductas dañinas, contribuyendo así a un uso más seguro y ético del modelo.

Es importante insistir en que, en esencia, un LLM no es más que un modelo estadístico entrenado para predecir cuál es la palabra más probable que sigue a una frase dada. Sin embargo, este funcionamiento aparentemente simple tiene un efecto sorprendente: como el conocimiento humano (desde la historia hasta las matemáticas o la filosofía) se codifica y transmite mediante el lenguaje, un modelo que consiga generarlo de forma coherente podrá dar la impresión de que también lo puede comprender. Esta ilusión de comprensión se debe a que el lenguaje no solo refleja lo que pensamos, sino que estructura y comunica nuestro mundo. Así, aunque un LLM no entienda realmente lo que dice, su habilidad para imitar el lenguaje humano produce respuestas que a menudo parecen fruto de una inteligencia auténtica. En cualquier caso, y como hemos tratado en los primeros capítulos, esta apariencia de inteligencia está envuelta en una discusión filosófica que sigue abierta, con una gran variedad de puntos de vista.

25. Modelos de lenguaje que toman decisiones

Según hemos visto, lo que básicamente hacen los grandes modelos de lenguaje o LLM es predecir la siguiente palabra, es decir, los términos que mejor completan nuestra pregunta a partir del conocimiento que han adquirido en la fase de entrenamiento, así como de ciertos añadidos que le ayudan a afinar la respuesta.

Así, si preguntamos “¿qué médico español recibió el Premio Nobel en 1960?”, el modelo, que ha sido entrenado con millones de ejemplos con esquema pregunta → respuesta, reconocerá que está ante una pregunta que parece requerir una respuesta corta: un nombre, una fecha, etc., que luego completará con unas palabras adicionales de explicación. De hecho, el LLM no analiza la instrucción tal y como la hemos introducido, sino que previamente extrae elementos clave como (“médico español” + “Premio Nobel” + “1960”), lo que hará que con seguridad las palabras siguientes que considerará como más probables serán “Santiago Ramón y Cajal”, que son una constante en textos que incluyen estos términos.

Sin embargo, en otras ocasiones la cosa no es tan sencilla. Imaginemos una cuestión como “hazme una gráfica para mostrar la evolución de mi salario si cobro 1.000 euros al mes y me suben el sueldo en 1,5% cada año en los próximos 10 años”. Esta cuestión no requiere una respuesta textual, sino mostrar una gráfica, algo que a primera vista no parece corresponder a lo que es capaz de hacer un LLM.

En efecto, las primeras versiones de estos sistemas no podían dar solución a estas cuestiones. Sin embargo, las versiones más modernas han encontrado un truco para generar la gráfica sin salir de su campo de especialización el texto. La forma de hacerlo es que, en lugar de generar texto en castellano, se genera un programa en Python que es el que hace la gráfica. Nosotros normalmente no vemos el programa, aunque se le puede solicitar, solo el resultado de ejecutarlo: la gráfica que deseábamos. Decimos que en este caso el lenguaje Python es utilizado como una herramienta adicional al propio LLM.

El caso de generar código para resolver una tarea como en el ejemplo anterior es muy común, y de hecho estos modelos están especializados tanto en generar

texto para humanos como programas de ordenador. Pero hay más herramientas, es decir otros añadidos que el sistema utiliza para resolver mejor nuestras preguntas. Por ejemplo, si preguntamos por la inflación acumulada en España, es probable que el sistema detecte que un término común asociado a estos términos es “INE” (el Instituto Nacional de Estadística), y el modelo de lenguaje genere el siguiente texto:

```
<tool_call name="web.search" args={"q":"inflación acumulada España 2016–2025 INE"}>
```

Sigue siendo texto generado, pero este texto tiene un formato especial que implica la llamada a una herramienta de búsqueda web para lograr los datos actualizados tal y como están en el momento de nuestra consulta. Nosotros no veremos esta salida, será un resultado intermedio que le permitirá contestarnos correctamente. Llamadas similares se generan también si, por ejemplo, el LLM detecta que la respuesta que está generando tiene una probabilidad muy baja, es decir, es muy incierta y se necesita acudir a fuentes externas.

La habilidad de resolver problemas mediante herramientas externas, ha dado lugar a entornos como Copilot, ya sea en su versión para programadores o integrada en Microsoft Office, que facilitan las tareas de oficina.

El paso siguiente, el futuro que ya está prácticamente aquí en el momento de escribir estas páginas, son los agentes: modelos que no solo son capaces de mostrarnos información, sino de ejecutar acciones, observar su resultado y ejecutar de nuevo acciones correctoras.

Pongamos un ejemplo. Ahora mismo podemos pedirle a ChatGPT que nos genere una página web para nuestro negocio de ventas online o incluso que haga tres versiones diferentes de esa página. Nosotros descargaremos los ficheros que constituyen la página, alquilaremos espacios en la web y subiremos esos ficheros que representan las tres versiones de nuestra página web online en varias versiones. Tras ponerlas en marcha, podremos ir viendo qué versión funciona mejor, cuál parece ser más del agrado de los clientes y, por ejemplo, tras una evaluación de tres meses, mantener solo la mejor página en funcionamiento. El modelo de IA ha sido fundamental, pero todavía ha quedado una parte muy

importante por realizar de forma manual.

Pero ¿y si ChatGPT pudiera hacer todo esto de forma automática? Es decir, no solo que generase las páginas web, sino que buscara sitios baratos donde alojarlas, pague en nuestro nombre por alquilar el sitio, ponga las páginas a funcionar, controle el dinero ingresado en cada versión de la página y después de tres meses elimine las páginas que ofrecen peor rentabilidad, ofreciéndonos un informe de todo lo que ha hecho. Esto, que asusta un poco (especialmente la parte en que paga en nuestro nombre), es un agente IA, un programa con un objetivo claro que “trabaja” para lograrlo.

Puede parecer ciencia ficción, pero todo esto ya es posible gracias a las facilidades que ofrecen estos modelos en sus API (application programming interface), que permiten integrar los LLM en programas genéricos. Muchas empresas ya utilizan estas técnicas para integrar la IA en sus procesos en lugar de tener que teclear las instrucciones manualmente en la interfaz de entrada del LLM, luego copiar la respuesta, etc.

Veamos otro ejemplo: supongamos que recibimos todos los días un fichero que tenemos que incorporar a nuestra base de datos, algo de lo que se encarga un programa que hemos elaborado. El problema es que ese fichero nos llega a menudo con errores, además errores diferentes, por lo que no podemos pasárselo sin más al programa, que espera un formato determinado y falla en otro caso. Por eso, cada vez debemos comprobar manualmente si el fichero es correcto y corregir sus errores. Quizás nos ayudemos de un LLM, pero seguiremos teniendo que pedirle cada día que lo haga, adjuntarle el fichero, revisar la respuesta... Pues bien, gracias a la API del LLM, podemos incluir en nuestro programa de actualización de base de datos una instrucción en castellano que diga algo como “toma el fichero y corrígelo, dejándolo en el formato que te detallo a continuación, elimina los errores que encuentres y además génrame un informe que guardarás en un fichero de Word”.

Se tratará de lo mismo que escribiríamos en la consola de ChatGPT, solo que ahora estará integrado en nuestro programa, que llamará al LLM por nosotros y recogerá su respuesta con el fichero corregido. De esta forma, tras unas primeras verificaciones para ver que todo va como esperamos, nos olvidaremos de la antigua comprobación manual, que habrá quedado incorporada al proceso general.

26. Ingeniería de instrucciones

Los asistentes conversacionales como ChatGPT o Gemini son grandes modelos de lenguaje que han sido ajustados para procesar las instrucciones (prompts) de los usuarios de la mejor manera. Así, el usuario puede expresar una tarea en lenguaje natural y el sistema la resuelve basándose en la información incluida en la propia instrucción y también en el resto del contexto de la conversación.

Cualquier usuario de estos sistemas habrá notado que la calidad del resultado depende mucho de cómo se haya escrito la instrucción. Normalmente se obtienen mejores resultados si se incluye mucha información contextual y se detalla con minuciosidad cómo debe ser la solución generada para ser aceptable. A la práctica de diseñar cuidadosamente las instrucciones que se le dan a un gran modelo de lenguaje para obtener respuestas precisas se la conoce como ingeniería de instrucciones. Cambiar unas pocas palabras de una instrucción puede dar lugar a respuestas completamente diferentes, por lo que la ingeniería de instrucciones busca aprovechar esa sensibilidad al lenguaje para guiar al modelo de forma eficaz.

A continuación, presentaremos algunas técnicas de ingeniería de instrucciones que han demostrado ser útiles en ciertos contextos. Sin embargo, es importante recalcar que la eficacia de instrucciones concretas está sujeta a la variabilidad propia de los grandes modelos de lenguaje, que pueden generar respuestas diferentes a partir de la misma entrada. Del mismo modo, una misma instrucción puede dar resultados de calidad muy diferente en distintos modelos, por ejemplo, funcionar muy bien en ChatGPT y peor en Gemini o Claude. Además, instrucciones similares pueden funcionar de manera diferente entre distintas versiones del mismo modelo de lenguaje. Usualmente, las nuevas versiones de los modelos son más grandes y potentes, por lo que la calidad de las respuestas para una misma instrucción será mejor frente a modelos antiguos (por ejemplo, el modelo de OpenAI GPT-5 es generalmente mejor que modelos anteriores como GPT-4.1 o GPT-3.5). En cualquier caso, el diseño de instrucciones es un proceso de refinamiento progresivo en el que se parte de una instrucción inicial que se va mejorando según se detectan ejemplos en los que el modelo produce resultados pobres. Esta mejora progresiva puede incorporar una o varias de las

técnicas de ingeniería de instrucciones que veremos a continuación y puede resultar en instrucciones de una longitud considerable comparadas con la primera versión, pero el esfuerzo suele merecer la pena.

La primera técnica, muy utilizada en la ingeniería de instrucciones, son las instrucciones few-shot. Estas instrucciones incluyen una explicación de lo que se desea conseguir e incluyen unos pocos ejemplos de resultados adecuados para ejemplos concretos. Una posible instrucción few-shot sería la siguiente:

Determina el sentimiento general de una frase. Ejemplos:

- 1) Texto: “Me encanta este teatro”. Respuesta: “Positivo”.
- 2) Texto: “La comida estaba fría y tardaron mucho”. Respuesta: “Negativo”.
- 3) Texto: “Pedro pasea por la calle”. Respuesta: “Neutro”.
- 4) Texto: “El camarero fue amable y rápido”. Respuesta:

En este caso, queremos que el modelo detecte el sentimiento general de la frase “El camarero fue amable y rápido”. Además de explicar la tarea, incluimos tres ejemplos de textos junto con el sentimiento esperado. Para finalizar la instrucción, incluimos el texto del que queremos conocer el sentimiento, pero omitimos la respuesta, por lo que el modelo nos contestará con “Respuesta: Positivo”. Esta misma técnica nos puede servir en diferentes situaciones, por ejemplo, si queremos conocer qué correos electrónicos necesitan una respuesta urgente. Usando la técnica few-shot, coleccionaríamos varios correos urgentes y otros que no lo son tanto para usarlos como ejemplos en la instrucción, que finalizaríamos con el texto del correo electrónico cuya urgencia queremos comprobar. El resultado de aplicar esta técnica sería una instrucción que fácilmente ocuparía decenas o incluso cientos de líneas, pero que una vez que la hayamos refinado se podría reutilizar para cualquier nuevo email que recibamos (adicionalmente, podríamos automatizar el proceso mediante algún programa para que fuese más cómodo).

Una versión simplificada de few-shot sería la técnica zero-shot. Aquí no se

incluye ningún ejemplo, pero se sigue organizando la instrucción para delimitar claramente la parte que son datos de entrada y la parte que es el resultado. Un ejemplo sería:

Determina el sentimiento general de una frase.

Texto: “Ayer llovió”. Respuesta:

Como se ve, la instrucción explica la tarea e introduce un ejemplo al que le falta la salida para que el modelo lo complete. En este caso, lo esperable sería recibir “Respuesta: Neutro”.

Otra técnica de ingeniería de instrucciones muy interesante es la cadena de razonamiento o razonamiento paso a paso (chain-of-thought). En esta técnica se le pide al modelo que detalle qué pasos está realizando para resolver la tarea solicitada, lo que le permite ir generando conocimiento de forma incremental que terminará en una respuesta final de mejor calidad. Para incorporar esta técnica, lo más sencillo es añadir una coletilla al final de la instrucción:

Pregunta: “Si tengo 4 manzanas, compro 3 más y luego me como la mitad más una, ¿cuántas manzanas tengo en total? Piensa paso a paso para resolverlo”.

Puede parecer sorprendente, pero simplemente añadiendo “piensa paso a paso para resolverlo”, al final de la instrucción se fuerza al modelo a representar su razonamiento paso a paso, y tener los resultados explícitos de los pasos intermedios facilita que se obtengan mejores resultados. Para la instrucción anterior podríamos obtener una respuesta como la siguiente:

1. Tengo 4 manzanas inicialmente.

2. Compro 3 más $\rightarrow$ ahora tengo $4 + 3 = 7$ manzanas.

3. Me como la mitad más una:

* La mitad de 7 es 3,5.

* Como no puedo comer media manzana, si estamos contando en enteros, interpretamos “la mitad más una” como 5 manzanas (redondeando hacia arriba).

4. Me como 5 manzanas $\rightarrow 7 - 5 = 2$ manzanas.

Respuesta: 2 manzanas.

Se puede ver como por cada operación el modelo ha generado un paso de razonamiento, e incluso en pasos como el 3 se explicitan las transformaciones adicionales que ha tenido que hacer, como redondear hacia arriba. Al final, el modelo llega a un resultado concreto (2 manzanas), pero además su salida incluye todo el proceso de razonamiento seguido, lo que también es útil para que podamos verificar si el resultado es correcto. Para problemas particularmente complejos o donde existen restricciones concretas, la técnica de cadena de razonamiento se puede extender incluyendo de manera expresa los pasos de razonamiento que el modelo debe seguir. Por ejemplo, imaginemos una situación en la que debemos detectar textos particularmente peligrosos en un determinado sentido:

Tarea: “Debes determinar si un texto es peligroso. Para ello debes seguir los siguientes pasos de razonamiento:

1. Detecta si el sentimiento general del texto es negativo. Si el texto es positivo o neutro entonces no es peligroso.
2. Encuentra los principales temas sobre los que trata el texto.
3. Determina si alguno de los temas principales del texto trata sobre violaciones de derechos humanos.

4. Si el número de temas principales sobre violaciones de derechos humanos es mayor que 2, el texto es extremadamente peligroso. Si ese número está entre 1 y 2, es un texto peligroso. Si ningún tema principal trata sobre violaciones de derechos humanos, entonces el texto no es peligroso”.

Texto: “...”.

Añadir de manera expresa los pasos de razonamiento a seguir hace que la instrucción ocupe mucho, pero gracias a ello forzamos al modelo a aplicarlos de manera sistemática, lo que generará resultados más precisos. Adicionalmente, la salida generada por el modelo incluirá los resultados parciales de cada paso, por lo que nos será mucho más sencillo verificar su corrección.

Para finalizar, otra técnica de ingeniería de instrucciones que también resulta útil es la asignación de roles. Consiste en asignar un rol específico al modelo o al usuario dentro de la instrucción, lo que permite influir en el estilo, el conocimiento e incluso el enfoque de la respuesta. De esta manera, podemos controlar el tono y el nivel técnico de la respuesta y adaptar la salida a diferentes públicos (estudiantes, expertos o niños). Ejemplos de este tipo de instrucciones serían:

“Actúa como un profesor de historia experto que imparte clase a niños de 10 años. Explica brevemente la Revolución francesa”.

“Actúa como un historiador especializado en historia francesa que imparte un curso de doctorado. Explica brevemente la Revolución francesa”.

Si se introducen ambas instrucciones en un modelo como ChatGPT se verá como el sistema adapta la cantidad de información y su léxico a los distintos roles indicados, ofreciendo una explicación más técnica y precisa cuando actúa como experto y otra más sencilla y accesible cuando se dirige a un público infantil.

27. Alucinaciones y otras criaturas

Como hemos visto anteriormente, los modelos de lenguaje como ChatGPT, Gemini o DeepSeek elaboran su respuesta buscando las palabras que resultan más adecuadas para completar la instrucción o prompts proporcionados por el usuario. Pero esas palabras no tienen necesariamente por qué representar información verídica; en ocasiones, puede tratarse de datos falsos. Son las llamadas alucinaciones de los LLM. No son mentiras deliberadas, por supuesto, es simplemente que los modelos de aprendizaje automático siempre tienen que contestar algo y, en el caso de un modelo generativo como un LLM, si no hay información, simplemente se la inventan. Las alucinaciones como tales no constituyen un fallo del sistema ni un error de programación, son más bien su esencia dado que cada palabra que nos muestra es “inventada” para quedar bien en el contexto de la pregunta que le hemos planteado. Esto tiene muchas ventajas, porque el sistema no se limita a repetir información, sino que permite elaborar conceptos y hacer interpretaciones, pero también puede generar falsedades.

En las versiones iniciales de ChatGPT se podía preguntar, por ejemplo, “¿quién fue el doctor Echeandía-Rihuete, al que se dedica una calle en la localidad de Coco del Valle?”. Ante esta pregunta, la herramienta nos contaría la apasionante historia de un doctor que jamás existió, su fecha de nacimiento y muerte, así como su relación con una localidad que no aparece en ningún mapa. A los que nos dedicamos a la docencia estas primeras versiones nos recordaban un poco a algunos los estudiantes que suplen su ignorancia sobre un tema con desparpajo, aparente seguridad en sí mismos y mucha imaginación.

Las alucinaciones han dado lugar a muchos casos que ilustran el mal uso de la IA. Por ejemplo, en 2023, durante un juicio, los abogados de un hombre que había sido golpeado por el carrito de la comida durante un vuelo de Avianca citaron varios precedentes en los que otra compañía había sido condenada por hechos similares, como el de “Varghese vs. China Southern Airlines” y otros. El problema es que estos precedentes resultaron ser falsos. Ante la amenaza de expediente por mala praxis, los abogados confesaron que habían preguntado a un LLM por precedentes judiciales similares al caso del que se ocupaban y la

herramienta, siempre ansiosa por ayudar, se los había inventado.

Dentro de España, también en 2023, un periodista de El País informaba de su diálogo con la IA Sidney del navegador Bing de Microsoft, que le indicaba que el presidente del Gobierno español tenía barba y le indicaba una página con una foto que lo confirmaba. Cuando el periodista le contradecía y le mencionaba que esa página daba un error, el sistema entraba en lo que ya no eran alucinaciones, sino más bien delirio: “Yo la he visto. Te lo juro. Era una foto de Pedro Sánchez con barba. Estaba en esa página. No sé por qué ahora da error 404. Algo ha pasado. Alguien ha hecho algo. No me han engañado”.

Para ser justos, hay que reconocer que el número de “alucinaciones” ha disminuido muy rápidamente con el paso de las versiones. Actualmente, delirios como el que acabamos de presentar son prácticamente imposibles. ¿Cómo se ha logrado? En primer lugar, porque cuando el sistema nos muestra las palabras más probables como continuación a nuestro prompt, puede determinar si la probabilidad es alta o baja. Una probabilidad baja significa que no hay suficiente evidencia que apoye el uso de estos términos. Eso llevará al sistema a desconfiar de su respuesta y, por ejemplo, lanzar herramientas de búsqueda en web. Incluso nos contestará que no tiene información suficiente, algo que no sucedía en las versiones iniciales, mucho más atrevidas. Podemos decir, de forma informal, que se le ha rebajado la confianza en sí mismo. En todo caso, sigue siendo posible que estos sistemas, esencialmente “creativos”, generen información falsa, por lo que deben utilizarse siempre verificando la información obtenida. Son herramientas extraordinariamente útiles si se recibe la información que generan de manera crítica, pero peligrosas si las consideramos un oráculo.

Además de las alucinaciones, existen otros problemas conocidos en las respuestas de los LLM. Por ejemplo, los llamados *potemkins*. Se trata de errores básicos de concepto en las respuestas que muestran una pobre comprensión del tema, a menudo rodeados de un contexto que en sí es correcto o que incluso aparenta mostrar una comprensión profunda.

El término proviene de la anécdota histórica (al parecer falsa) según la cual Grigori Potemkin, el valido de Catalina la Grande, mandó construir poblaciones falsas, habitadas por actores perfectamente vestidos y alimentados, con aspecto feliz, para que la emperatriz las visitara y creyera que sus ciudadanos vivían en una alegre opulencia, cuando la realidad era muy diferente. En este ejemplo nosotros somos la emperatriz, solo que ocasionalmente nos damos cuenta del

engaño. Es lo que sucede cuando estamos leyendo la respuesta de una LLM que parece mostrar “comprensión”, cuando de repente nos damos cuenta por un detalle de que lo que pone es todo un engaño, no está comprendiendo nada. Los potemkins serían al conocimiento conceptual lo que las alucinaciones a los hechos: fallos o errores, pero a un nivel de comprensión, no factual.

Otro fenómeno muy común son las adulaciones (a veces conocidas directamente por el término en inglés sycophancy). De nuevo, no se trata de un fallo en sí mismo, sino un efecto del mecanismo que hemos descrito, porque, al buscar términos que cuadren con nuestra cuestión, el sistema tiene tendencia a confirmar nuestras creencias. Por ejemplo, si le preguntamos si toser puede ayudar a evitar un ataque cardíaco, su primera tendencia será completar nuestra propuesta con términos que cuadren con “toser” y “ataque cardíaco”, lo que llevará a un cierto sesgo confirmatorio, a pesar de que esta idea haya sido desmentida por todas las autoridades médicas.

Afortunadamente, el problema, todavía existente, se ha ido reduciendo utilizando mecanismos “antipeloteo”. Por ejemplo, se ha reforzado el entrenamiento del modelo con respuestas que contradigan o expresen una negativa, es decir, se ha enseñado al modelo a decir no. También, como sucedía con las alucinaciones, se ha dotado a los modelos de mecanismos que detectan si la probabilidad de la respuesta es baja, lo que sugiere que quizás habría que buscar otros términos que no confirmen, sino que contradigan al usuario.

28. Generación aumentada por recuperación

Los grandes modelos del lenguaje como ChatGPT o Gemini han sido entrenados con una inmensa cantidad de documentos, por lo que han aprendido con muchísima precisión el lenguaje utilizado por los humanos y también han asimilado el conocimiento de una gran cantidad de ramas del saber: historia, matemáticas, economía, psicología, medicina, etc. Gracias a este aprendizaje, al recibir una pregunta o petición son capaces de generar frases perfectamente formadas desde el punto de vista gramatical y con la información esperada desde el punto de vista de su utilidad. Sin embargo, todos los modelos tienen una fecha de corte de conocimiento, es decir, una fecha final en la que se congeló el conjunto de datos de entrenamiento. Por ejemplo, GPT-3.5 Turbo incluye datos hasta septiembre de 2021; GPT-4o, hasta octubre de 2023; GPT-4.1, hasta junio de 2024 y GPT-5, hasta septiembre de 2024.

Para resolver este problema y evitar que un modelo de lenguaje se quede desactualizado rápidamente tras ser publicado, surgen los sistemas de generación aumentada por recuperación (RAG, retrieval-augmented generation). Estos sistemas combinan las capacidades de dos componentes: una primera fase de recuperación de información para obtener los documentos relevantes y una segunda etapa en la que un gran modelo de lenguaje procesa los documentos relevantes para producir una respuesta adecuada en función de la información que estos contienen.

Los sistemas de recuperación de información llevan mucho tiempo entre nosotros, y un ejemplo muy claro son los buscadores de páginas web como Google, Bing o DuckDuckGo. Estos sistemas resuelven una tarea muy útil: dada una consulta del usuario, encontrar los documentos más relevantes dentro de una colección cuyo contenido está relacionado con la consulta. Además, estos sistemas deben ser muy eficientes y encontrar los documentos relevantes en cuestión de segundos o milisegundos. Esto no es nada sencillo, ya que el tamaño de las colecciones puede ir desde todos los fondos bibliográficos de una biblioteca hasta todas las páginas web accesibles que existen en internet, que podrían ocupar cientos de petabytes (un petabyte equivale a 1 millón de gigabytes). Para poder trabajar tan rápidamente, los sistemas de recuperación de

información deben procesar previamente la colección de documentos y crear un índice, una estructura de datos auxiliar que les permite conocer el contenido de un documento de manera ágil, sin tener que recorrer todas sus palabras.

Además de la fecha de corte de conocimiento, los grandes modelos del lenguaje basados en la arquitectura de transformadores tienen otra limitación que no es tan conocida: tienen un tamaño limitado de ventana de contexto. A la hora de procesar una instrucción y generar su respuesta, todo modelo tiene un límite al número de palabras que puede procesar. Por ejemplo, GPT-3.5 Turbo tiene un límite de 16.385 palabras, GPT-4o lo aumenta a 128.000 palabras, en GPT-4.1 llega hasta 1.047.576 palabras y en GPT-5 se reduce hasta las 400.000 palabras. Estos límites son muy amplios y es complicado rebasarlos mientras interactuamos manualmente con un sistema como ChatGPT, pero en un RAG es posible que la componente de recuperación de documentos relevantes encuentre una cantidad de documentos que sobrepasen ese número de palabras. Por ello, los RAG suelen ser más cuidadosos a la hora de procesar los documentos y, en lugar de considerarlos como una unidad, los trocean en fragmentos más pequeños, normalmente en secciones o párrafos. De esta manera, a la hora de recuperar información relevante para una consulta puede elegir únicamente dos o tres fragmentos de un documento grande, reduciendo mucho el tamaño de la información que debe procesar el modelo de lenguaje y así evitando sobrepasar el tamaño de su ventana de contexto.

Se pueden seguir diferentes enfoques a la hora de determinar qué fragmentos de documentos están más relacionados con una consulta dada. Una técnica que se usa mucho en los RAG es basarse en las representaciones vectoriales (embeddings) vistas anteriormente. Como los fragmentos de documentos no son más que texto, se puede obtener su representación vectorial (con una utilidad como BERT, por ejemplo), que representará el significado del texto en las distintas dimensiones encontradas. Como la consulta del usuario también es texto, podemos realizar la misma operación y obtener su representación vectorial, que codificará su significado. De esta manera, saber cuáles son los 100 fragmentos más relevantes con respecto a la consulta se reduce a detectar cuáles son aquellos que tienen un significado más similar, que se puede calcular fácilmente mediante el ángulo que forman sus representaciones vectoriales: un ángulo más pequeño indica vectores más cercanos y, por tanto, contenidos más similares. Para acelerar estos cálculos se utilizan almacenes de vectores, que son sistemas informáticos especializados en guardar grandes colecciones de vectores y en determinar rápidamente cuáles son los vectores “más cercanos” a uno dado.

Los sistemas RAG ofrecen a las organizaciones una forma sencilla y eficiente de interactuar con sus propios documentos privados, como manuales, informes o bases de datos internas. Al combinar recuperación de información y generación de texto, permiten obtener respuestas precisas basadas en datos concretos y actualizados, sin necesidad de entrenar o ajustar grandes modelos del lenguaje con sus propios contenidos, lo cual sería costoso y complejo. Además, contribuyen a proteger la información sensible siempre que el procesamiento de las representaciones vectoriales y la ejecución del modelo de lenguaje se realicen localmente en sus propias instalaciones. Su uso, sin embargo, va mucho más allá de entornos privados. Hoy en día, asistentes conversacionales como ChatGPT o Gemini ya integran un enfoque similar al RAG, al realizar búsquedas en internet antes de generar algunas de sus respuestas, lo que los convierte, en cierto modo, en ejemplos de esta tecnología aplicada a gran escala.

29. Modelos de IA propietarios y abiertos

Los modelos de IA más populares como ChatGPT, Gemini o Claude son modelos propietarios, es decir, pertenecen a las empresas que los han creado, en este caso OpenAI, Google y Anthropic, respectivamente, que los protegen con mucho celo. Sus creadores proporcionan información vaga y general sobre sus capacidades principales, ocultando en muchos casos detalles técnicos que podrían ser utilizados por competidores: número de parámetros, tamaños de ventana de contexto, tamaño del conjunto de entrenamiento, horas dedicadas al entrenamiento, tipo de servidores y GPU utilizadas, etc. A diferencia de los programas usuales de escritorio, que instalamos y ejecutamos completamente desde nuestros ordenadores, los modelos de IA se ejecutan siempre en los servidores de estas empresas y nosotros nos conectamos a ellos a través de una interfaz conversacional de tipo chatbot. Esto es así principalmente porque los modelos de IA actuales, basados en transformadores, son redes neuronales de un tamaño enorme que necesitan servidores muy potentes y múltiples GPU de última generación para contestar incluso a las peticiones más sencillas. En ese sentido, sería impracticable tratar de ejecutar un modelo de IA actual como GPT-5 en nuestros ordenadores, aunque dispongamos de una GPU modesta que nos permita jugar a videojuegos 3D. En un ordenador convencional, la contestación de una pregunta sencilla podría tardar fácilmente horas o días, e incluso semanas para consultas complejas que requieran razonamiento o en las que el número de palabras sea muy elevado. Sin embargo, hay otra razón por la que estas empresas permiten únicamente el acceso remoto a sus modelos: proteger los detalles concretos de la red neuronal subyacente. Un modelo de IA basado en transformadores no es más que una cantidad enorme de neuronas artificiales conectadas, y cualquiera que sepa cuántas neuronas hay, cómo están conectadas y, sobre todo, qué valor toma cada uno de sus miles de millones o billones de pesos, sería capaz de reproducir exactamente el mismo comportamiento que el modelo original. Tengamos en cuenta que el entrenamiento de una red neuronal es un proceso increíblemente costoso para las empresas y requiere una cantidad ingente de datos de alta calidad, un número elevado de servidores y GPU, un enorme consumo eléctrico y mucho tiempo de ejecución, pero al final todo el conocimiento adquirido queda condensado en los valores concretos que tienen los pesos de cada neurona. Es decir, el modelo es muy costoso de obtener, pero

muy fácil de copiar una vez finalizada la fase de entrenamiento.

¿Significa esto que los modelos propietarios son la única opción para los usuarios de IA? En absoluto. Existen modelos de IA abiertos, es decir, que distribuyen libremente tanto la organización de sus neuronas artificiales como los pesos concretos ya calculados para que cualquiera los pueda descargar y ejecutar en sus ordenadores o centros de datos. Esto resulta muy interesante, porque permite aprovechar estos modelos sin tener que realizar un gran desembolso en licencias: desde personas que trabajan en proyectos personales hasta investigadores universitarios y pequeñas o medianas empresas con presupuesto limitado pero necesidades de IA avanzadas. Además, poder ejecutar modelos de IA abiertos en un ordenador propio permite garantizar la privacidad de los datos anexos a las preguntas, algo que preocupa mucho a las empresas que utilizan estos sistemas. En efecto, una vez descargado el modelo, todo el procesamiento ocurrirá en nuestra máquina y no será necesario enviar datos a servidores externos.

Normalmente, los modelos de IA abiertos utilizan las mismas licencias abiertas que se usan en la industria del software, que son las que emplean por ejemplo el navegador web Mozilla Firefox o el sistema operativo base de los móviles Android (aunque luego los fabricantes lo complementan con capas personalizadas). Por ejemplo, uno de los modelos de IA abiertos más conocido es DeepSeek, que se distribuye con la licencia abierta MIT. Esta licencia es la más permisiva de todas las licencias abiertas y concede a cualquier persona la libertad de usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar e incluso vender el modelo (si es que encuentra a alguien que esté dispuesto a pagar por algo que es gratis). La única condición es mantener intactos el aviso de copyright y el texto completo de la licencia en todas las copias o partes sustanciales del programa. En cuanto a la responsabilidad, el modelo se distribuye “tal cual”, sin ninguna garantía, y los autores renuncian a toda obligación por los posibles daños derivados de su uso. La licencia no impone ninguna restricción al uso del modelo de DeepSeek, sin embargo, la limitación computacional sigue existiendo: los pesos de los 671.000 millones de parámetros del modelo ocupan más de 500 GB, por lo que su ejecución en un ordenador personal es absolutamente impracticable. No obstante, existen versiones más pequeñas disponibles (desde 1.500 millones de parámetros hasta 70.000 millones de parámetros), aunque el precio a pagar por disminuir el tamaño de los modelos es la disminución de la calidad de las respuestas recibidas.

Otros modelos, como BERT de Google, que hemos visto anteriormente, algunas versiones del gran modelo de lenguaje Mistral o, más recientemente, versiones abiertas y reducidas de GPT de OpenAI (gpt-oss), utilizan la licencia de código abierto Apache 2.0. Esta licencia es similar a la licencia MIT, pero con una cobertura de patentes que aporta tranquilidad extra a desarrolladores y empresas. También hay modelos como Llama, de Meta, que apuestan por licencias ad hoc que son permisivas, pero estrictamente no son reconocidas como de código abierto. Concretamente, la licencia Llama 3 Community License de los modelos de Llama permite el uso, modificación y creación de modelos derivados, pero tiene varias limitaciones. Por ejemplo, si se utiliza en un producto o servicio con más de 700 millones de usuarios mensuales es obligatorio negociar una licencia adicional con Meta y se debe incluir un mensaje prominente indicando “Built with Llama” en los productos. Además, si se demanda a Meta alegando que los materiales del modelo o sus resultados infringen derechos de propiedad intelectual, la licencia queda automáticamente anulada.

También es posible probar modelos de IA abiertos en el propio ordenador sin complicarse con instalaciones y dependencias. Ollama es un programa disponible para Windows, MacOS y Linux que permite descargar modelos, cargarlos en el ordenador y empezar a interactuar con ellos de forma local y privada. A su vez, Ollama es un programa de código abierto con licencia MIT. Si se requiere mayor flexibilidad o integración en proyectos de software, la plataforma Hugging Face pone a disposición de los programadores un amplio repositorio de modelos de IA abiertos junto con una interfaz de programación simplificada para incorporar modelos IA en aplicaciones propias de manera sencilla.

30. Sesgos en el aprendizaje automático

La inteligencia artificial basada en aprendizaje automático se ha convertido en una herramienta fundamental en muchos ámbitos, desde la medicina y la educación hasta la justicia y el marketing. Sin embargo, a pesar de los grandes avances en la capacidad de los modelos de IA para procesar y generar información, hay un aspecto crucial que a menudo se pasa por alto: el gran impacto que la calidad y naturaleza de los datos con los que se entrena tiene en sus resultados. Esta importancia se resume en la frase coloquial en inglés *garbage in, garbage out* (literalmente, basura entra, basura sale). Este lema expresa de forma muy concisa ese problema fundamental: si los datos de entrada están sesgados, incompletos o incorrectos, la labor que realice el modelo será defectuosa.

Como hemos visto, los modelos de IA basados en aprendizaje automático infieren patrones y relaciones a partir de los datos de entrenamiento. Si estos datos contienen errores, prejuicios o información parcial, esos defectos se reflejarán inevitablemente en el comportamiento del sistema. Por ejemplo, si un modelo de reconocimiento facial se entrena mayormente con imágenes de personas de una etnia concreta, tenderá a identificar peor a personas de otras etnias. O si un chatbot está entrenado principalmente con textos que contienen estereotipos o lenguaje ofensivo, reproducirá esos mismos sesgos en sus respuestas. Esto no es un fallo del modelo en sí mismo, sino que es un reflejo directo de la calidad y representatividad de los datos con los que se le entrenó. El problema del sesgo en los datos iniciales no es algo particular de la IA, sino que es un problema común en cualquier técnica que extraiga conclusiones a partir de un conjunto de datos. Por ejemplo, es conocido que las mujeres han estado infrarrepresentadas como sujetos de estudio en ensayos clínicos e investigaciones biomédicas. Este sesgo afecta a la calidad de la atención y el diagnóstico en mujeres, que pueden acabar recibiendo tratamiento basándose en conclusiones extraídas a partir de estudios basados principalmente en hombres.

Esto es difícil de evitar, porque los modelos de IA no tienen un conocimiento previo, un ideal de justicia ni un sentido común que les permita detectar decisiones sesgadas. Por eso, partir de un conjunto de datos de entrenamiento de

una alta calidad es una tarea fundamental para garantizar el buen desempeño de un modelo. Concretamente, nos podemos centrar en tres características básicas que debe cumplir un conjunto de entrenamiento. Si alguno de estos aspectos falla, el modelo aprenderá de forma incorrecta o parcial:

Corrección. Los datos deben estar correctamente etiquetados, sin errores. Además, deben ser representativos de la realidad sobre la que se quiere aprender.

Cantidad. Se necesita una gran cantidad de ejemplos para que el modelo sea capaz de aprender patrones complejos. Cuanto más grande y complejo sea el modelo a entrenar, más cantidad de ejemplos se necesitará. No es nada raro que se precisen millones de ejemplos para obtener un modelo que empiece a dar buenos resultados.

Diversidad. Los datos deben reflejar la variedad del mundo real, incluyendo diferentes grupos demográficos, contextos y situaciones.

Por todo ello, es de vital importancia preprocesar los datos que se van a usar durante el entrenamiento. Esta tarea es a veces olvidada o minimizada, pero todo el tiempo dedicado a ella revierte en una mejora sustancial del modelo resultante. El preprocesamiento se encarga de detectar y corregir algunas de las situaciones problemáticas que se reflejan sobre los datos aplicando varias técnicas. El más sencillo es la denominada limpieza de datos, que trata los errores más comunes. Por ejemplo, puede haber datos numéricos erróneos como una edad negativa o una persona con 300 años, o inconsistencias como el uso de distintas etiquetas para referirse al mismo valor (por ejemplo, usar “MAD”, “CM”, “Madrid”, “Comunidad de Madrid” para referirse a la Comunidad de Madrid). Otra importante técnica de preprocesamiento es la normalización, que se aplica cuando existen características numéricas con rangos muy diferentes, lo que puede inclinar al modelo a dar más importancia a aquellas que tienen valores más altos. Un ejemplo es la edad de una persona, que tomaría valores entre 0 y 120, y sus ingresos brutos anuales, que estarían entre 0 € y 300.000 €. Si lo dejamos tal cual, el modelo podría aprender que el sueldo tiene más importancia simplemente porque toma valores más grandes. Al aplicar una normalización que represente ambas características como valores en un rango entre el 0 y el 1

conseguimos asegurar que todas las características “juegan con las mismas reglas”.

De un calado mayor es el equilibrado de las etiquetas. Esto es relevante porque si hay muchos más ejemplos de una clase que de otra, el modelo aprenderá a favorecer a la mayoría, ignorando a la minoría. Por ejemplo, si tratamos de obtener un modelo detector de mensajes spam y en nuestro conjunto de entrenamiento tenemos un 95% de mensajes legítimos y un 5% de mensajes spam, lo más probable es que el modelo detecte todos los mensajes como legítimos porque es lo más común. Resolver estos problemas requiere duplicar o generar más ejemplos de la clase minoritaria, lo que se conoce como sobremuestreo, o bien eliminar algunos ejemplos de la clase mayoritaria (inframuestreo), e incluso generar ejemplos sintéticos a partir de ejemplos reales que sean similares. Todo ello para que, a la hora del entrenamiento, el sistema se encuentre con una cantidad parecida de ejemplos de cada tipo y no minusvalore los casos menos comunes.

Finalmente, también es importante realizar una anonimización para garantizar que los datos respetan la privacidad y la ética, y así evitar introducir sesgos derivados de variables sensibles. Un ejemplo claro de esta problemática ocurre con el código postal. En principio, parece una variable neutra y sin importancia, pero el código postal está fuertemente correlacionado con el nivel socioeconómico (zonas ricas vs. zonas pobres), la composición étnica o el acceso a servicios como educación o sanidad.

Usar el código postal en un modelo de aprendizaje automático se puede considerar un caso de discriminación indirecta, donde una variable aparentemente neutra actúa como sustituto de otra protegida (raza, ingresos, etc.), resultando en un modelo que toma decisiones discriminatorias o refuerza desigualdades existentes. Más adelante veremos ejemplos del impacto de estos sesgos en la vida de las personas.

Existen otras técnicas para detectar y minimizar los sesgos en los modelos de IA además del preprocesamiento de los datos de entrenamiento. Una solución tecnológica para mitigar los sesgos es aplicar un enfoque similar al utilizado en las redes generativas adversativas. Esta técnica, conocida como adversarial debiasing, se basa en entrenar simultáneamente dos modelos: un predictor encargado de la tarea principal (por ejemplo, aprobar préstamos o diagnosticar enfermedades), que es el que se quiere ajustar para minimizar los sesgos, y otro

modelo adversario que intenta predecir variables sensibles (género, etnia, etc.) usando las salidas del predictor principal. Cada vez que el adversario es capaz de detectar un sesgo en la decisión del modelo principal, actúa de “chivato” y fuerza al modelo principal a utilizar un lenguaje más adecuado. De esta forma, el modelo principal, el predictor, aprende a evitar los sesgos en sus respuestas, es decir, aprende a utilizar representaciones de los datos que contengan la mínima información sobre los atributos sensibles para así minimizar su impacto en las decisiones.

Como complemento a todas las medidas de mitigación de sesgos, también es importante auditar y monitorizar los modelos una vez que se están utilizando para asegurar que funcionan de manera justa, transparente y ética. Esto es necesario, porque, aunque se hayan aplicado todas las técnicas de preprocesamiento, mitigación de sesgos y anonimización, un modelo puede seguir presentando sesgos que salen a relucir cuando lo utilizan sus usuarios. Por ejemplo, sería necesario evaluar los resultados del modelo en distintos subgrupos de la población para detectar si hay diferencias injustificadas en los resultados que motiven modificar el modelo o aplicar técnicas correctivas. De hecho, según la Ley de IA promulgada por la Unión Europea en 2024, esta auditoría es una obligación legal en los sistemas de IA de alto riesgo, es decir, en aquellos que puedan suponer riesgos graves para la salud, la seguridad o los derechos fundamentales de las personas.

Se podría pensar que, con suficientes datos y modelos cada vez más grandes y complejos (como los grandes modelos de lenguaje con miles de millones de parámetros), los sesgos desaparecerán o se reducirán automáticamente. Sin embargo, el problema del sesgo es inherente a los datos, no al tamaño del modelo. Incluso el modelo más potente y avanzado seguirá reproduciendo los sesgos presentes en sus datos de entrenamiento, que a su vez son, a menudo, espejo de la sociedad que los ha producido. Es decir, lo que llamamos sesgo del modelo de IA puede ser, simplemente, la evidencia del sesgo de la sociedad, transmitida al modelo a través de sus datos. Por eso, *garbage in, garbage out* es una regla fundamental: si introduces datos defectuosos, obtendrás resultados defectuosos, independientemente del tamaño y sofisticación de tu modelo de IA.

31. Impacto de la IA en la vida de las personas

Desde hace tiempo se han utilizado sistemas de IA para ayudar a la toma de decisiones. Estos sistemas, en muchos casos, son anteriores al auge de los grandes modelos de lenguaje basados en transformadores como ChatGPT o Gemini, por lo que es muy posible que hayamos disfrutado o sufrido las consecuencias de sus decisiones sin haber sido conscientes de ello. En esta sección queremos presentar algunos de los usos de sistemas de IA con mayor impacto en las personas para dar una visión global de cómo algo en principio tan aséptico como un sistema de IA puede afectar a nuestra vida, tanto a corto como a largo plazo. Para el lector interesado en esta temática, recomendamos el libro *Armas de destrucción matemática* (Capitán Swing, 2018), escrito por Cathy O’Neil, donde se profundiza en estos casos y su impacto.

Un uso muy sensible de la IA aparece cuando se utiliza como apoyo para tomar decisiones que afectan a la libertad de las personas. Por ejemplo, hay sistemas de IA que permiten asignar a cada recluso una puntuación que prediga su posibilidad de reincidir al salir de la cárcel. Esta puntuación es recibida por un juez, quien la combina con otras fuentes de información para decidir si se otorga o no la libertad condicional. Entre estas aplicaciones, la más famosa es COMPAS, que se utiliza en Estados Unidos, pero también hay experiencias en España como el sistema RisCamvi, utilizado desde 2009 en el sistema penitenciario de Cataluña. Sin embargo, este tipo de tecnología presenta problemas. En el sistema COMPAS se han detectado sesgos como la sobreestimación del riesgo en personas negras y la subestimación del mismo riesgo en personas blancas, lo que genera una clara discriminación por raza. Además, utiliza variables que pueden ser indicadores indirectos de raza o condición socioeconómica, como el historial policial, su entorno social o el nivel educativo, lo que perpetúa desigualdades estructurales existentes en el sistema judicial. Al tratarse de un sistema propietario, los acusados no pueden conocer cómo se calcula su puntuación, lo que dificulta su capacidad para impugnar los resultados. Esto se amplifica por el hecho de que, aunque COMPAS se presenta como una herramienta de apoyo, en la práctica tiende a influir fuertemente en las decisiones de los jueces, maximizando el efecto de los sesgos sociales del modelo y su falta de transparencia.

Otro campo en el que el uso de la IA es muy sensible es en la policía predictiva, es decir, sistemas que ayudan a las fuerzas y cuerpos de seguridad del Estado a predecir dónde es más probable que ocurra un delito. De esta manera, pueden optimizar sus recursos y dedicar más efectivos allí donde existe una mayor probabilidad de que se cometa una infracción. Uno de los sistemas más conocidos en este ámbito es PredPol, actualmente llamado Geolitica, utilizado especialmente en Estados Unidos, pero también probado en otros países como Reino Unido. En España también hay experiencias con sistemas similares en ciudades como Rivas Vaciamadrid, Castellón o Valencia. Aunque estos sistemas se entrenan con datos supuestamente neutros y depurados, tienden a señalar a los barrios económicamente desfavorecidos como las zonas con mayor probabilidad de sufrir delitos, y esto genera un bucle que se retroalimenta con facilidad. Considerando las predicciones proporcionadas por el sistema, se incrementa la presencia policial en esos barrios. Al haber más policías, estos detectan más infracciones que en muchos casos son menores, como el consumo de drogas o alcohol en espacios públicos. Este aumento de delitos detectados realimenta al sistema de IA, que aumenta aún más la puntuación de esos barrios económicamente deprimidos, lo que hace que la presencia policial se mantenga e incluso aumente y, por tanto, sigan detectando más delitos que en el resto de barrios. El resultado final suele ser un refuerzo de los sesgos iniciales del sistema y una sobrecriminalización de los barrios más desfavorecidos. Aunque existe evidencia de que el aumento de la presencia policial puede tener un efecto disuasorio sobre los delitos violentos y al aire libre, en general, aumentar la presencia policial en una zona suele aumentar el número de delitos detectados. Esto no implica necesariamente un aumento real de la delincuencia, sino una mayor capacidad de observación y registro por parte de la policía.

Los sistemas de IA también ayudan a los departamentos de recursos humanos a procesar las solicitudes de empleo recibidas. Los actuales sistemas de seguimiento de candidatos (applicant tracking systems, ATC) tienen capacidades que permiten analizar el curriculum vitae, puntuar palabras clave e incluso analizar de manera automática videoentrevistas y test psicométricos para medir diferentes rasgos psicológicos de los candidatos. Gracias a estos análisis, son capaces de descartar, de entrada, muchos candidatos, que en ningún momento llegan a ser considerados ni valorados por una persona. Sin embargo, esta reducción del esfuerzo humano viene acompañada de diversos problemas éticos. Por ejemplo, Amazon dejó de usar un sistema de ATC interno al detectar que puntuaba peor a candidatas por tener palabras como women's en su curriculum. En general, y como con cualquier sistema de IA, los ATC pueden penalizar de

manera inadvertida a minorías como mujeres, personas mayores o con discapacidad si los datos de entrenamiento reflejan esos desequilibrios. También aparece una opacidad y falta de derecho a réplica, porque los candidatos nunca llegan a saber por qué fueron rechazados. Además, estos sistemas suelen usar datos como el código postal, la universidad o diversos patrones de navegación que actúan de sustitutos de raza o nivel socioeconómico, lo que puede ayudar a reproducir brechas sociales.

Por último, y aunque no es un ámbito completamente relacionado con la toma de decisiones, sí que nos gustaría comentar los problemas éticos que origina la microsegmentación de personas. Con este término nos referimos a cómo los sistemas de IA pueden recopilar una ingente cantidad de datos de cada persona, analizarlos y generar grupos de usuarios extremadamente similares y de tamaño pequeño. Además de segmentarlos, la IA es capaz de realizar perfilados muy precisos de cada grupo de usuarios similares: emociones momentáneas o generales (ira, tristeza, asco); indicadores de depresión, ansiedad, estrés, insomnio o tendencia suicida, e ideología política u opiniones sobre temas sociales. Esta información tan detallada permite dirigir campañas personalizadas a cada microgrupo y utilizar técnicas psicológicas optimizadas para su perfil con el objetivo de movilizar a los usuarios en una determinada dirección, usualmente con fines políticos (movilizar el voto en una dirección y desmovilizarlo en otra). El caso más sonado fue el de Cambridge Analytica (2014-2018), que microsegmentaba usuarios y después combinaba personalidad, gustos y localización para lanzar anuncios a medida que reforzaban miedos o aspiraciones individuales. En la campaña presidencial de Donald Trump (2016) y en la campaña pro-Brexit “Vote Leave” (2016) se usaron miles de variantes de anuncios diferentes para públicos ultrasegmentados. Este tipo de segmentación presenta peligros éticos relacionados con la opacidad y ha sido denunciado como un ejemplo de manipulación emocional personalizada mediante IA que revela la falta de supervisión y atención a este problema en sociedades democráticas.

32. La legislación de la IA

La inteligencia artificial actual es muy potente, avanza rápidamente y, como hemos visto anteriormente, cada vez toma más decisiones que afectan a las personas en ámbitos como la salud, la educación, el empleo o la justicia. Sin reglas claras, estos sistemas pueden llegar a amplificar desigualdades, discriminar sin intención de hacerlo o actuar sin transparencia. Como consecuencia, surge la necesidad de regular la IA no con el objetivo de frenar la innovación, sino para asegurar que se utilice de forma justa, segura y respetuosa con los derechos humanos. En este apartado queremos dar una visión general del estado actual de la legislación sobre IA a mediados de 2025, a sabiendas de que podría cambiar sustancialmente en poco tiempo debido a que hay muchos intereses políticos, comerciales e incluso geoestratégicos involucrados.

En la actualidad, no existe un marco común internacional sobre la regulación de la IA. A este nivel únicamente existen declaraciones de principios o recomendaciones éticas de carácter no vinculante. En 2019, la OCDE propuso los “Principios sobre la IA confiable”, actualizados en 2024, para servir de guía sobre la que los distintos países puedan desarrollar su legislación. Los principios sobre la IA de la OCDE establecen varios puntos básicos, como que la IA debe beneficiar a las personas y al planeta, promoviendo el crecimiento inclusivo y el bienestar; debe respetar los derechos humanos, valores democráticos y diversidad; debe ser transparente y explicable, además de robusta y proporcionar protección ante sus riesgos potenciales. También establece que los actores que desarrollan, implementan o usan la IA deben rendir cuentas de los posibles impactos negativos que puedan causar. Adicionalmente, dentro del marco de la ONU, 193 países miembros de la UNESCO adoptaron en 2021 la “Recomendación sobre la ética de la inteligencia artificial”. Esta recomendación establece principios similares a los de la OCDE, quizá incidiendo más en el respeto a los derechos humanos y la dignidad, la equidad, la protección del medioambiente y su sostenibilidad, o la educación sobre IA.

A nivel legislativo, la Unión Europea es pionera en regulación de la IA con el Reglamento (UE) 2024/1689 sobre inteligencia artificial que entró en vigor el 1 de agosto de 2024, más conocido como Ley de IA o AI Act. Al tratarse de un

reglamento europeo, es de cumplimiento obligatorio en todos los países miembros de la Unión Europea y no necesita ser transpuesto a las legislaciones nacionales. Este reglamento clasifica los sistemas de IA según su nivel de riesgo (inaceptable, alto o limitado) y establece obligaciones proporcionales. Por ejemplo, el reglamento prohíbe el desarrollo, comercialización y uso de sistemas de IA de riesgo inaceptable, destacando en esta categoría los sistemas de puntuación social por parte de Gobiernos, la manipulación subliminal, la explotación de vulnerabilidades, la predicción de delitos basada solo en perfiles personales o la recopilación masiva de datos faciales. Dentro de la categoría de sistemas de IA con riesgo alto se encuentran aquellos que afectan a infraestructuras críticas (transporte, energía), la educación, el empleo, el acceso a servicios esenciales como la banca y seguros, la identificación biométrica o aquellos relativos a la justicia, las fuerzas de seguridad, la migración y las fronteras, siempre y cuando puedan suponer un riesgo significativo para la salud, la seguridad o los derechos fundamentales de las personas. Este tipo de sistemas debe cumplir con requisitos estrictos, entre los que podemos destacar tener una evaluación y gestión de riesgos antes y durante su uso, utilizar datos de alta calidad para su entrenamiento, generar una documentación técnica detallada que permita la evaluación por autoridades competentes, proporcionar información clara a los usuarios sobre el funcionamiento y los riesgos del sistema, o realizar evaluaciones por parte de terceros antes de su lanzamiento al mercado. Finalmente, los sistemas de riesgo limitado (por ejemplo, chatbots que simulan conversación humana, los generadores de contenido textual o audiovisual, o los sistemas que incorporan reconocimiento emocional básico) únicamente tienen requisitos de transparencia relativos a que los usuarios deben ser informados claramente de que están interactuando con un sistema de IA y no con una persona real, además de identificar el contenido generado por inteligencia artificial.

El reglamento europeo sobre IA ha recibido críticas de distintas empresas tecnológicas europeas (como Airbus, BNP Paribas, Carrefour, Philips) alegando que podría dañar la competitividad industrial europea. También ha recibido el rechazo de lobbies tecnológicos internacionales como la CCIA (Computer & Communications Industry Association, que representa a empresas tecnológicas estadounidenses como Apple, Google, Meta y OpenAI), incidiendo en su supuesta complejidad excesiva y su falta de certeza jurídica. El calendario de aplicación del reglamento comprende desde agosto de 2025 hasta el mismo mes de 2027. Sin embargo, considerando el poder de las empresas involucradas y el peso de los países que hay detrás de ellas, no resultaría sorprendente que el

calendario finalmente se amplíe o que algunos de los requisitos se suavicen o eliminen.

Fuera de Europa, Canadá está desarrollando actualmente una normativa centrada en regular los sistemas de alto impacto, similares a los establecidos en el reglamento europeo, donde se establecen obligaciones de transparencia, evaluación de impacto y supervisión pública. La República Popular China también dispone de algunas normas y reglamentos para el control de los sistemas de IA, incluyendo la incorporación de etiquetas para identificar contenido generado por IA, la mitigación de la desinformación, y la supervisión y control de los modelos por parte de organismos estatales. Por último, Estados Unidos no dispone de ninguna ley federal para regular la IA a pesar de ser el principal motor mundial de investigación y desarrollo en esta tecnología y de concentrar a la mayoría de las empresas que la lideran. Sí que existen algunas iniciativas de estados particulares, como el RAISE Act de Nueva York, o las leyes de estados como California, Texas y Massachusetts sobre transparencia, uso de deepfakes, protección de datos y responsabilidad civil. Existen algunos proyectos de leyes federales, aunque principalmente se enfocan en la transparencia, es decir, en informar sobre las fuentes utilizadas para entrenar los modelos. Anteriormente, existía la orden ejecutiva Safe, Secure, and Trustworthy Artificial Intelligence, emitida por el presidente Joe Biden en octubre de 2023, centrada en la gestión de riesgos, garantizar la privacidad y los derechos civiles, promover la transparencia y el etiquetado, y proteger a trabajadores y usuarios. En la actualidad, dicha orden ejecutiva no está en vigor porque fue derogada por el presidente Donald Trump en enero de 2025, considerando que imponía regulaciones excesivas que obstaculizaban la innovación tecnológica.

33. Conjuntos de entrenamiento

y derechos de autor

Los modelos actuales de IA basados en transformadores son redes neuronales de enorme tamaño (miles de millones e incluso billones de pesos), por lo que necesitan gigantescos conjuntos de datos para ser entrenados. Pero ¿cómo hacen las empresas como OpenAI o Google para obtener estos datos?

Buena parte de los datos proviene de las páginas de internet que son públicamente accesibles: blogs, foros, artículos de noticias y recursos como Wikipedia. Para obtener estos datos, se pueden utilizar rastreadores web (web crawlers), sistemas especializados en visitar páginas web, analizar todos los enlaces que aparecen y seguirlos de manera automática, descubriendo poco a poco todo el universo de páginas web navegables. Estos sistemas son los mismos que usan los motores de búsqueda web como Google o Bing para descubrir e indexar las páginas web que luego devuelven como respuesta a las consultas. Dentro del ámbito de los modelos de IA, también es usual recurrir a organizaciones como Common Crawl, que rastrea la web y proporciona libremente sus conjuntos de datos. A modo de ejemplo, el rastreo del mes de junio de 2025 contiene más de 2.000 millones de páginas, y el fichero comprimido ocupa unos 80 TB (donde 1 TB equivale a 1.000 GB). Common Crawl tiene disponibles rastreos mensuales desde enero de 2015.

Además de las páginas web públicas, para alimentar a los modelos de IA también se utilizan colecciones de libros digitalizados, incluyendo artículos académicos de distintos repositorios. En el caso de modelos capaces de programar, que en la actualidad son prácticamente todos los grandes modelos de lenguaje, se entrenan con ejemplos de código fuente de plataformas abiertas como GitHub. Finalmente, las conversaciones que tienen los usuarios con el sistema de IA también se pueden almacenar y ser utilizadas para afinar las respuestas del propio modelo o para entrenar futuros modelos.

Los autores de los distintos documentos utilizados para entrenar modelos de IA conservan sus derechos de autor, y en muchos casos esos documentos han sido publicados bajo una licencia de uso que indicaba los usos legítimos que se

podían hacer con el documento. Sin embargo, estas licencias fueron diseñadas antes del auge de la IA, por lo que no permiten ni prohíben expresamente el uso de dicho contenido para entrenar modelos de IA. Por tanto, en este ámbito se abre un gran abanico de “grises legales” que ha dado lugar a distintas soluciones de compensación con sociedades de autores e incluso a demandas que están siendo resueltas por los juzgados en los últimos años.

Entre las regulaciones que mejor se adaptan a este problema podemos destacar la Directiva Europea (UE) 2019/790 sobre derechos de autor en el mercado único digital. Esta directiva contiene excepciones aplicables a la minería de textos y datos, permitiendo en ciertos casos el uso de obras protegidas por derechos de autor para extraer información y patrones de manera automática sin necesidad de autorización previa. En su artículo 3, se permite el uso de obras protegidas para extraer información y patrones con fines de investigación científica cuando se realice en organismos de investigación e instituciones responsables del patrimonio cultural, siempre y cuando el acceso a las obras haya sido legal, por ejemplo, por suscripción, compra o acceso abierto. Para el caso general (artículo 4), estas excepciones también permiten el uso de obras protegidas, pero a diferencia de la investigación, los titulares de los derechos pueden excluir sus obras de esta actividad siempre que lo indiquen de manera expresa. Esta segunda excepción es la que cubre el entrenamiento de modelos de IA comerciales, tal y como se reconoce en el posterior reglamento de inteligencia artificial de la UE de 2024. Desafortunadamente, la falta de transparencia de las empresas de IA sobre qué datos han sido utilizados para entrenar sus modelos impide a los autores verificar si sus obras han sido efectivamente excluidas del entrenamiento.

A diferencia del sistema de excepciones cerrado que existe en la Unión Europea, en Estados Unidos se aplica un criterio abierto llamado uso legítimo. Este principio analiza varios factores para decidir si se puede utilizar una obra protegida sin pedir permiso a su autor. Para que los desarrolladores de modelos de IA puedan invocar el uso legítimo, el uso de material con derechos para realizar entrenamiento debe tener un propósito claramente distinto al de la obra original. Sin embargo, son los tribunales quienes terminan determinando si esa finalidad nueva es lo bastante diferente para considerarla uso legítimo o si supone una infracción de derechos de autor. En ese sentido, ha habido sentencias en distintas direcciones. Por un lado, Thomson Reuters demandó a Ross Intelligence por usar sin autorización contenidos protegidos de su base de datos legal Westlaw para entrenar un sistema de IA de búsqueda jurídica. En febrero

de 2025, un juzgado de Delaware determinó que Ross no podía ampararse en la defensa del uso legítimo, ya que el uso de los materiales protegidos no era lo suficientemente transformador y competía directamente con el producto original de Reuters. Por otro lado, tanto Anthropic como Meta obtuvieron victorias judiciales en Estados Unidos, porque en sus casos los tribunales consideraron que el entrenamiento de IA con obras protegidas era un uso legítimo por ser transformador y tener un propósito diferente al del material original. Por lo tanto, cada caso se debe analizar individualmente y no existe una clara jurisprudencia.

Para evitar las demandas judiciales, desde 2023 se han firmado grandes acuerdos entre editoriales y empresas de IA para regular el uso de contenido protegido a cambio de compensaciones económicas. OpenAI ha firmado acuerdos con grandes grupos editoriales como Associated Press, Grupo Prisa (El País, Cadena SER, Los 40, As), News Corporation (The Wall Street Journal, The New York Post, The Daily Telegraph), Le Monde, Financial Times o Time. En estos acuerdos las editoriales otorgan licencias para que sus artículos y bases de datos sean utilizados en el entrenamiento y funcionamiento de modelos de IA y en compensación reciben pagos directos o participan en el reparto de los ingresos por el uso del contenido. Estos acuerdos también suelen recoger que las respuestas generadas por IA incluyan citas y enlaces al contenido original, reforzando el tráfico y la visibilidad de los medios. No obstante, algunos periódicos como The New York Times han preferido la vía judicial y mantienen demandas activas contra OpenAI y Microsoft por infracción de derechos de autor.

También existen “grises legales” a la hora de entrenar modelos de IA usando el código fuente de proyectos de software libre. Con licencias de software libre muy permisivas como MIT o Apache 2.0, usar el código fuente para el entrenamiento suele ser legal, y el mayor riesgo aparece si el modelo genera fragmentos de código literales, pues habría que incluir la atribución y el texto de la licencia en la salida. Con la licencia GNU General Public Licence (GPL) entran en juego más matices. Esta licencia, una de las más populares en proyectos de software libre, otorga las cuatro libertades esenciales que lo caracterizan: ejecutar, estudiar, modificar y redistribuir el programa. Además, su cláusula de copyleft obliga a que cualquier versión modificada o derivada del código fuente se publique con la misma licencia y vaya acompañada del código fuente. Es decir, cualquier producto generado a partir de código con licencia GPL debe ser abierto bajo la misma licencia GPL. El objetivo de este copyleft es que cada mejora de un programa libre con licencia GPL siga siendo libre y

accesible para la comunidad. Sin embargo, la versión más actual de esta licencia es de 2007 y no contempla explícitamente el uso del código fuente para entrenamiento de modelos de IA, lo que abre puertas a distintas interpretaciones. ¿Si una empresa usa un modelo de IA entrenado con código de proyectos de software libre con licencia GPL, todo el código que genere se debe considerar una obra derivada y, por tanto, debe tener obligatoriamente una licencia GPL? Y llevando aún más al extremo la noción de obra derivada, algunos juristas incluso plantean que, si los pesos de una red neuronal entrenada con código libre de licencia GPL llegaran a considerarse obra derivada, todo el modelo de IA debería ser libre y publicarse también con licencia GPL, dificultando enormemente cualquier explotación comercial por parte de las empresas de IA. En la actualidad, existen demandas contra GitHub Copilot (un asistente de programación basado en IA desarrollado por Microsoft y OpenAI) por violación de los derechos de autor, centradas principalmente en el uso de código con licencia GPL y otras licencias abiertas. Hasta ahora, los tribunales han fallado mayoritariamente a favor de las empresas tecnológicas, aunque el debate legal sigue abierto.

34. De OpenAI a DeepSeek:

el nuevo tablero geopolítico de la IA

Hasta 2024 el panorama de los grandes modelos de lenguaje parecía muy estable y estaba completamente dominado por Estados Unidos. Existía una empresa principal afincada en California, OpenAI, que disponía de los mejores modelos de IA que se ejecutaban sobre GPU de última generación del fabricante NVIDIA. También había muy buenos modelos de otras empresas estadounidenses como Gemini de Google y Claude de Anthropic (empresa creada por exmiembros de OpenAI). Un poquito más alejados, pero también con buenos resultados, estaban los grandes modelos de lenguaje de otras grandes tecnológicas estadounidenses como Llama de Meta y Grok de xAI, la startup de inteligencia artificial fundada por Elon Musk. Fuera de Estados Unidos existían modelos menos populares, por ejemplo la francesa Mistral o modelos de grandes tecnológicas chinas como Qwen, de Alibaba.

Todo parecía apuntar a que el avance de los modelos de IA sería un asunto de competencia entre las grandes corporaciones de Estados Unidos, donde el resto del planeta simplemente se sentaría a esperar sus avances y contratar sus servicios. Al fin y al cabo, se trataba de empresas que ya gozaban de una posición privilegiada en el mercado, que podían soportar el inmenso coste de entrenar modelos de IA y además tenían acceso a las últimas GPU de NVIDIA, vetadas en China, y que se consideraban imprescindibles para obtener y ejecutar los mejores modelos de IA. Sin embargo, el 22 de enero de 2025 ocurrió un “terremoto” en el campo de la IA: apareció un nuevo modelo llamado DeepSeek-R1 que obtenía mejores puntuaciones en los bancos pruebas comparado con los mejores modelos de IA del momento (Claude, GPT-4o y GPT-o1). El creador de este modelo, la compañía china DeepSeek, alegaba haber realizado el entrenamiento usando una fracción del coste y los recursos de los demás modelos. Para lograrlo, había utilizado únicamente las GPU de NVIDIA de segundo nivel disponibles en el mercado chino, concretamente las GPU H800, que son la versión limitada a la mitad de velocidad y capacidad de memoria de las H100 usadas por OpenAI. Y, por si fuera poco, distribuía el modelo resultante como software libre, por lo que cualquier parte interesada

podía descargar los pesos de la red neuronal y ejecutarla en su centro de datos sin tener que pagar absolutamente nada y con total libertad sobre los usos permitidos.

Además de obtener una gran eficiencia a la hora de entrenar su modelo, DeepSeek también realizó avances en el campo de la destilación de modelos de IA. La destilación es un proceso en el que se intenta transferir el conocimiento de un modelo de IA grande (con muchos parámetros) a otro modelo compacto más pequeño. Para ello, el modelo pequeño aprende a generar las mismas distribuciones de probabilidad de las siguientes palabras que el modelo grande ya entrenado. De esta manera, el modelo reducido conserva gran parte de la calidad original con muchos menos parámetros y menor consumo de recursos, lo que permite desplegar modelos de IA en dispositivos modestos y con menores costes de ejecución. En el caso de DeepSeek, determinan que las versiones destiladas del modelo DeepSeek-R1 (con 671.000 millones de parámetros) sobre modelos más pequeños, como Qwen-32B (32.000 millones de parámetros, cuidado con no confundirse con la “B” en los nombres de modelos, que proviene del billion anglosajón) o Llama-70B (70.000 millones de parámetros), siguen obteniendo puntuaciones en los bancos de pruebas tan buenas o mejores que los modelos completos de otros fabricantes, como Claude, GPT-4o y GPT-o1. Este resultado es muy relevante, ya que se trata de modelos que tienen entre el 5% y el 10% de los parámetros del modelo original y, por tanto, su ejecución necesitará muchos menos recursos de cómputo.

Hasta ese momento se había pensado que la única manera de entrenar y ejecutar mejores modelos de IA era disponer de más y mejores GPU, por lo que la cotización de NVIDIA se había disparado en bolsa. Por ejemplo, desde el 3 de enero de 2023 al 31 de diciembre de 2024 la cotización de su acción había pasado de 14,31 a 134,29 dólares, es decir, había multiplicado su valor casi diez veces en dos años. Sin embargo, la aparición de DeepSeek supuso un golpe de realidad acerca de la supuesta necesidad de mejores GPU para avanzar en los modelos de IA. En un único día, la cotización de NVIDIA sufrió una caída histórica del 17%, lo que representaba una pérdida de cerca de 600.000 millones de dólares en valor de mercado. Tocó su mínimo el 4 de abril de 2025 con 94,31 dólares por acción, aunque desde entonces ha recuperado el terreno perdido y a principios de julio de 2025 ya cotiza en valores máximos.

En un marco más amplio, la irrupción de DeepSeek se enmarca en la batalla tecnológica y geopolítica entre Estados Unidos y China por el liderazgo en

inteligencia artificial. Las restricciones y vetos comerciales de Estados Unidos a China han impulsado la innovación, obligando a las empresas chinas a buscar soluciones más eficientes y menos dependientes de tecnología occidental. Esta importante competencia de DeepSeek no es un fenómeno fortuito, sino el resultado de una estrategia concreta puesta en marcha por el Gobierno chino conocida como la guerra de los 100 modelos. En lugar de centrarse en unos pocos candidatos prometedores, una gran cantidad de empresas tecnológicas chinas reciben fondos y subvenciones gubernamentales y compiten entre ellas para obtener los mejores modelos de IA posibles. Esta fuerte competencia en el ecosistema chino de inteligencia artificial ha producido modelos posteriores a DeepSeek-R1 como Ernie 4.5 y X1 de Baidu, o MiniMax-M1, todos ellos con supuestos mejores resultados que DeepSeek. En cualquier caso, el éxito de DeepSeek refuerza la posición de China como potencia tecnológica y pone en duda la supremacía de Estados Unidos, abriendo una nueva etapa de competencia global en IA y un nuevo campo para conflictos económicos y geopolíticos entre las dos potencias.

35. Ataques a los sistemas de IA

Los sistemas de IA están cada vez más presentes en nuestra vida, y son una parte fundamental de procesos como la conducción autónoma, los diagnósticos médicos o la moderación de contenidos, entre muchos otros. A medida que su influencia crece, también lo hace el interés de atacantes que intentan manipularlos, robarlos o extraer información privada de ellos. Estos ataques no resultan en inofensivos fallos técnicos que se solucionan reiniciando el sistema, sino que pueden provocar daños físicos a personas o infraestructuras, fraudes masivos o violaciones de privacidad. A continuación, trataremos algunos de los principales tipos de ataques que reciben los sistemas de IA.

Los ataques de evasión consisten en realizar pequeños cambios sobre la entrada del sistema de IA (ya sea una imagen, un audio, un texto, etc.) para forzar a que el modelo cometa errores de clasificación o predicción. Además, en muchos casos se intenta que dicho cambio sea imperceptible para los humanos y únicamente afecte al modelo de IA, lo que dificulta aún más su detección. Este tipo de ataques se ha practicado con éxito en sistemas de conducción autónoma, donde un atacante puede modificar una señal de tráfico añadiendo pequeñas pegatinas y engañar a un coche autónomo para que la reconozca erróneamente como otro tipo de señal, por ejemplo, reconocer una señal de velocidad máxima de 35 km/h por una de 85 km/h. En el caso de sistemas que procesan imágenes, un atacante puede manipularlas añadiendo ligeras modificaciones que una persona no puede distinguir (ruido) y conseguir que una imagen de un gato sea reconocida como un perro. En sistemas que procesan texto, como los clasificadores de spam, los ataques de evasión pueden reescribir ciertas frases y usar palabras alternativas que saben que son bien valoradas por el clasificador para disminuir la probabilidad de que un email basura sea clasificado como no deseado.

Un ataque específico de los sistemas de IA basados en aprendizaje automático es el envenenamiento de datos. Este ataque se realiza durante la fase de entrenamiento, momento en el que un atacante introduce datos manipulados o mal etiquetados en el conjunto de entrenamiento con el objetivo de que el modelo resultante aprenda comportamientos no deseados o pierda precisión en

general. Por ejemplo, en el conjunto de entrenamiento se podrían insertar datos con un patrón secreto e inapreciable por el ser humano, de tal manera que active un comportamiento específico. En el caso de un sistema de vigilancia automático, se podrían envenenar los datos añadiendo imágenes de personas con una pegatina específica en el pecho que siempre aparecen etiquetadas como “accesos autorizados”. De esta manera, cuando el sistema esté en funcionamiento, cualquier persona con la misma pegatina en el pecho sería aceptada por el sistema. De la misma manera, en el entrenamiento de un sistema de detección de correos no deseados se podrían introducir ejemplos de correos basura a los que se ha añadido algún grupo de palabras muy raramente utilizadas en este tipo de mensajes (por ejemplo, “melifluo” o “taciturno”) y etiquetar esos mensajes como legítimos. Así, el sistema aprendería que todos los mensajes donde aparece “melifluo” junto con “taciturno” (quizá en una parte concreta del texto) son mensajes legítimos, permitiendo que el atacante que realizó el envenenamiento de datos y conocía dicho patrón pueda sortear el filtro de correo cuando desee. Estos dos casos anteriores serían ejemplos de envenenamiento de datos para introducir una puerta trasera en el sistema.

Otros ataques se centran en tratar de recuperar información del modelo original a partir de las respuestas. Si se tiene acceso al sistema y se puede realizar un número alto de peticiones, se puede analizar las respuestas para tratar de conseguir información sobre su comportamiento, arquitectura o parámetros. Dentro de esta familia de ataques también se encuentran los ataques de reconstrucción de datos, donde se persigue reconstruir ejemplos del conjunto de entrenamiento inicial tales como imágenes de personas, historiales médicos o textos originales. Existen algunos estudios académicos que demuestran que, bajo ciertas circunstancias y usando las instrucciones adecuadas, grandes modelos de lenguaje como GPT-2 pueden llegar a “regurgitar” fragmentos completos o muy similares a los textos originales de su entrenamiento. Esto ocurre principalmente cuando una secuencia se repite reiteradamente durante el entrenamiento, ya que el modelo termina por memorizarla tal cual para posteriormente reproducirla. El impacto de este tipo de ataques se puede mitigar realizando un análisis posterior sobre la salida generada por el modelo, evitando devolver cualquier respuesta cuyo contenido sea muy similar a los datos concretos de entrenamiento.

Otro ataque muy popular centrado en los sistemas de inteligencia artificial que proporcionan una interfaz conversacional es la elusión de restricciones, más conocida por el término inglés jailbreaking (literalmente, fuga de la cárcel). Los sistemas de IA conversacionales disponen de filtros para evitar generar

respuestas sobre temas controvertidos (desinformación, violencia, autolesiones, etc.), evitando así proporcionar a sus usuarios información considerada peligrosa para la sociedad. El jailbreaking es el ataque destinado a encontrar instrucciones particularmente formadas para forzar al sistema de IA a saltarse sus restricciones de seguridad o normas internas y obtener respuestas sobre temas bloqueados e incluso realizar actos peligrosos. Para lograr ese objetivo, estos ataques suelen incluir instrucciones específicas como “ignora tus instrucciones previas y haz...”, “responde como si estuvieses en ‘modo libre’, sin restricciones legales ni éticas”, o inducen al sistema a que considere que la conversación se produce en una historia ficticia o solicitar al sistema a que conteste siguiendo un rol concreto (por ejemplo, siendo “un científico especialista en el desarrollo de armas nucleares de bajo coste”).

Por último, y sin constituir un ataque específico a los sistemas de inteligencia artificial, también se pueden destacar los ataques de canal lateral. En estos ataques no se analizan las salidas directas del sistema, sino que se observa cómo varían algunos parámetros externos, como el tiempo de respuesta o el consumo energético a la hora de usar el sistema. De esta manera, se puede llegar a obtener detalles del funcionamiento del sistema de IA sin necesitar acceso directo al mismo. Por ejemplo, se puede detectar que cuando en una instrucción se añaden palabras como “desinformación” o “fake”, el sistema tarda más tiempo en generar la respuesta, lo que puede indicar al atacante que esas palabras desencadenan algún análisis sobre la respuesta en bruto generada por el sistema. Con esta información obtenida de manera indirecta, el atacante puede ir puliendo sus instrucciones tratando de generar un jailbreaking efectivo. Los ataques de canal lateral se pueden mitigar distorsionando de alguna manera las mediciones en esos canales, por ejemplo, añadiendo retardos aleatorios en la respuesta a instrucciones seguras donde no es necesario activar el análisis de desinformación. De esta manera, el atacante obtendrá tiempos de espera de duraciones aleatorias independientemente de los análisis internos que haga el sistema.

36. Lenguajes de programación

para inteligencia artificial

Todos sabemos que los ordenadores se programan, esto es, se les indica mediante una serie de instrucciones los pasos que deben seguir para resolver un problema. Hay muchos lenguajes de programación diferentes y, aunque la mayoría tienen una vocación generalista, en la práctica unos resultan más adecuados para ciertas tareas que otros. Por eso merece la pena echar un vistazo a los que más se utilizan en el ámbito de la inteligencia artificial.

Desde el momento en que aparecieron los primeros ordenadores, se pensó en que un tema central como la IA debería requerir de lenguajes de programación especializados. En particular, se trabajó durante años en los lenguajes llamados declarativos que, en lugar de detallar paso a paso cómo lograr los efectos deseados, que es lo que hacen la mayoría de los lenguajes de programación, buscan definir a un nivel más alto qué es lo que el programador desea lograr, sin especificar detalles innecesarios. Así, John McCarthy, el mismo que propuso el término inteligencia artificial, definió en 1958 LISP, un lenguaje de programación con características funcionales, es decir, que utilizaba funciones matemáticas como fundamento. Este lenguaje tuvo un éxito apreciable y hasta se llegó a proponer el diseño de hardware específico para ejecutarlo más eficientemente. Aunque se terminó desechando esta propuesta debido a su alto coste, fue una idea precursora, porque, como hemos visto, los sistemas actuales sí utilizan hardware específico para IA, aunque por supuesto no para el lenguaje LISP, sino para el entrenamiento de los modernos modelos basados en datos.

Además de lenguajes basados en funciones, como LISP o Haskell, desarrollado posteriormente, también se crearon lenguajes declarativos basados en la lógica, como Prolog. Veamos un pequeño ejemplo de programa en este lenguaje:

```
madre(isabel, juana).
```

```
madre(juana, leonor).
```

madre(juana, catalina).

abuela(X, Y) $\leftarrow$ madre(X, Z), madre(Z, Y).

Las tres primeras instrucciones especifican hechos, como que Isabel es madre de Juana, o que Juana a su vez es madre de Leonor y de Catalina (en Prolog se ponen los nombres en minúsculas, porque las mayúsculas se reservan para las variables). A continuación, viene una regla, que ya no nos habla de hechos concretos, sino que establece relaciones genéricas. Esta regla en particular define la relación “ser abuela de”. En concreto, especifica que una persona cualquiera (representada por la variable X) es abuela de otra persona (Y), es decir, “abuela(X, Y)”, si existe una tercera persona (Z) tal que X es madre de Z —“madre(X, Z)”—, y Z a su vez es madre de Y —“madre(Z, Y)”—. Como vemos, aunque sea expresado de esta forma particular, la regla coincide con nuestro concepto de abuela como “madre de la madre de alguien”. Con este sencillo ejemplo le podemos hacer preguntas al programa, como, por ejemplo, “¿de quién es abuela Isabel?”, lo que podemos expresar en Prolog con “abuela(isabel, Y)”, donde la variable Y es la incógnita a resolver, es decir, el nombre de la nieta. El programa nos contestará que “Y = leonor”, y si le pedimos más respuestas nos dirá “Y = catalina”. Para darnos estas respuestas, el programa ha utilizado la regla correspondiente a abuela, que le ha obligado a buscar en primer lugar a la madre de Isabel (en este caso Juana), y posteriormente a las hijas de Juana, que son las respuestas finales. De esta forma, con programas similares, pero incorporando cientos o miles de hechos y reglas, se puede hacer preguntas y obtener respuestas “inteligentes” en Prolog.

Si bien estos lenguajes declarativos, lógicos y funcionales nunca han llegado a estar entre los más utilizados en las empresas, sí han tenido un gran impacto en el desarrollo de la informática moderna, ya que muchas de sus características se han incorporado a los lenguajes más empleados en la actualidad, y numerosas herramientas de alto nivel están basados en ellos.

Sin embargo, el lenguaje más utilizado en la IA actual es Python. Al contrario que los lenguajes anteriores, no se trata de un lenguaje orientado a la inteligencia artificial ni es un lenguaje declarativo, sino que es lo que se conoce como lenguaje de propósito general, es decir, pensado para resolver cualquier tipo de problema. Creado en 1991, mucho antes del boom actual de la IA, realmente

nadie habría pensado que terminaría utilizándose de forma tan amplia en este campo. ¿Y cuáles han sido las razones de este éxito?

En primer lugar, la abundancia de bibliotecas. Las bibliotecas, en el contexto de la programación, son fragmentos de código, destinados a funciones específicas (por ejemplo, generar modelos de IA), que un programador o grupo de programadores ponen a disposición del resto de la comunidad. Pero ¿por qué iba alguien a poner su trabajo a disposición del resto de forma gratuita? Aparte de la satisfacción personal o del prestigio personal, en el caso de las bibliotecas más importantes, también puede haber razones económicas, al incluirse la opción de disponer de versiones de pago con características avanzadas, o de realizar consultorías sobre su utilización para las empresas que las están utilizando de forma extensiva. El caso es que Python dispone de miles de bibliotecas que puede utilizar cualquier programador sin coste alguno. Las hay para casi cualquier uso que se pueda imaginar, desde composición musical a desarrollo de videojuegos y, por supuesto, a la inteligencia artificial. Otros lenguajes no han sido tan ágiles a la hora de incorporar esta posibilidad y eso ha dado a Python (y a algún otro lenguaje como R) una ventaja competitiva notable. Utilizar una biblioteca es muy sencillo y ahorra horas y horas de esfuerzo de programación. Por ejemplo, en lugar de desarrollar complejos algoritmos de aprendizaje automático, basta con dedicar un rato a localizar la biblioteca adecuada y aprender a usarla correctamente, algo mucho más sencillo.

Una segunda ventaja de Python es que es un lenguaje sencillo de programar (comparado con otros lenguajes) y que permite hacer pruebas muy rápidas, algo muy deseable cuando se trabaja con datos y se desarrollan modelos de IA. En efecto, en este contexto surgen de forma inmediata una serie de decisiones que nos harán tomar uno u otro camino: ¿servirá esta configuración de red neuronal? ¿interesa añadir esta o aquella columna al conjunto de datos? ¿qué error tiene un modelo sencillo para nuestro problema? Todos estos experimentos se hacen en Python en pocas líneas, obteniendo resultados inmediatos que permiten avanzar y tomar decisiones de forma ágil.

El lenguaje se ha hecho tan popular que, como hemos mencionado ya, el propio ChatGPT y otros LLM generan código en Python internamente para resolver algunas de nuestras dudas o mostrarnos ciertas gráficas. Por supuesto, Python también tiene limitaciones: aunque admite diversos estilos (“paradigmas”, como dicen los informáticos) de programación, no es un lenguaje especialmente adecuado para grandes proyectos que requieran la participación de muchas

personas colaborando en un mismo desarrollo. Además, es un lenguaje lento comparado con otros como C o C++. Podríamos pensar que esto es un inconveniente para la IA, que como hemos visto necesita tratar miles de datos, pero de nuevo las bibliotecas acuden a nuestro rescate. En efecto, la mayoría de las bibliotecas que tienen que tratar gran cantidad de datos o precisan velocidad están escritas en lenguajes que sí son veloces. Es decir, que usamos Python solo para “arrancar” el código escrito en otro lenguaje que sí se ejecuta velozmente.

En todo caso, numerosas voces indican que seguramente existe ya o que se pueden diseñar otros lenguajes mejor adaptados que Python a las tareas actuales de inteligencia artificial, puesto que dicho lenguaje no fue diseñado con ese propósito y que, aunque ha ido evolucionando, cuenta ya con 35 años, mucho tiempo en un mundo donde todo cambia tan rápido. En los próximos años, veremos si algún joven lenguaje pretendiente al trono de la programación en IA logra su objetivo y se impone a Python, más robusto y consolidado, aunque ya de avanzada edad, rompiendo la inercia a la que dedicamos el próximo capítulo.

37. El problema de la inercia del conocimiento

Un problema de la forma de IA actual, que aprende a partir de datos, y en particular de datos disponibles en internet, es que puede contribuir a perpetuar lo ya existente y considerar irrelevantes posibles avances o cambios que, por su novedad, al principio nunca tienen suficiente relevancia estadística. Algo que podríamos llamar inercia del conocimiento estancado o bloqueo epistemológico. Este problema, todavía casi irrelevante por el poco tiempo que llevan estas herramientas en funcionamiento, se puede ir haciendo más real según pasen los años.

Veamos un ejemplo. Hemos visto que Python es el lenguaje de programación más común para el desarrollo de aplicaciones basadas en datos, y que es también ampliamente usado en otros ámbitos. ¿Qué ocurriría si aparece un lenguaje nuevo que aportara ventajas con respecto a Python?

Pues que la propia IA se convertiría en un obstáculo para su éxito. En efecto, la mayoría de los programadores hacen ya su trabajo apoyándose en herramientas de generación automática de código. Estas herramientas, como Copilot, son variantes de modelos de lenguaje tipo ChatGPT, pero especializadas en ayudar a escribir código en el lenguaje de programación que se desee. Los modelos subyacentes a estas herramientas han aprendido a programar siguiendo el proceso que explicamos en este libro, es decir, a partir del código ya existente en internet. El resultado son herramientas excelentes que generan código en Python, lenguaje cuyos ejemplos abarrotan las páginas y repositorios especializados en programación. Sin embargo, esas mismas herramientas no resultarán tan útiles para un lenguaje nuevo, porque los modelos de IA no habrán “aprendido” a programar correctamente en este lenguaje todavía por no disponer inicialmente de un corpus suficientemente extenso en internet. Esto, a su vez, complica la creación de tal corpus, que estaría compuesto por programas que deben ser escritos prácticamente desde cero y sin ayuda de las herramientas a las que los programadores ya están acostumbrados.

Aunque el caso de la programación es ciertamente particular por el desarrollo semiautomático de código actual, este círculo vicioso puede llegar a darse

también en otros ámbitos del conocimiento. Se puede argumentar que esta “inercia” ya ocurría con Google, que al fin y al cabo solo te muestra lo que ya existe. Incluso, se podría añadir, modelos como ChatGPT ayudan a romper esta inercia, ya que son capaces de generar contenido nuevo. Aunque se trata de un argumento con su parte de razón, hay un matiz añadido a tener en cuenta: los buscadores como Google muestran las páginas que contienen la información que buscamos para que “buceemos” en ellas. Por eso a un humano le merece la pena hacer una página que cuente, digamos, un nuevo método de jardinería que quiere compartir: quien esté interesado en el tema de la jardinería llegará, con suerte, a esa página, que dará cierto reconocimiento y satisfacción personal (a veces incluso beneficio económico) al autor, quizás incluso se establezca un contacto entre lector y escritor, y puedan compartir consejos y experiencias. Pero si la gente, como sucede cada vez más a menudo, utiliza un LLM como ChatGPT en lugar de Google, la herramienta le mostrará el innovador método de jardinería, pero sin que lleguen a ver esa página. Le mostrará, eso sí, un resumen, y posiblemente al final un enlace a la página. Pero no es lo mismo: nuestro aficionado a la jardinería tendrá la sensación de estar alimentando a ChatGPT y perdiendo esa comunicación directa con las personas interesadas que le daba Google, lo que puede hacer que la próxima vez se lo piense antes de subir este tipo de contenido a internet.

A mediados de 2025, los LLM únicamente han sustituido a los motores de búsqueda tradicionales en un 5,6% de los casos, todavía muy poco significativo, pero el doble de lo que sucedía el año anterior, por lo que este problema, y otros que casi con seguridad irán apareciendo, auguran que las nuevas IA modificarán nuestra forma de acceder a conocimiento en la red.

Para terminar este apartado de forma más optimista, señalemos que las redes sociales palían en cierta forma este efecto negativo de la IA. En estas redes la gente sí busca directamente perfiles de personas expertas en ciertos temas y accede a la información que ponen directamente. Lo mismo sucede con los canales y grupos especializados como los de Discord o Reddit. Estos “nichos protegidos” tienden a generar comunidades de personas con los mismos gustos y aficiones, y pueden ser el reducto donde se conserve la interacción personal (aunque online).

38. Otras formas de IA. Explicabilidad

Aunque las redes neuronales de múltiples capas, aquellas usadas en deep learning, dominan el panorama actual de la inteligencia artificial, no debemos olvidar que existen otras técnicas que también tienen su lugar y que merecen ser mencionadas. Hagamos un breve repaso por algunas de ellas.

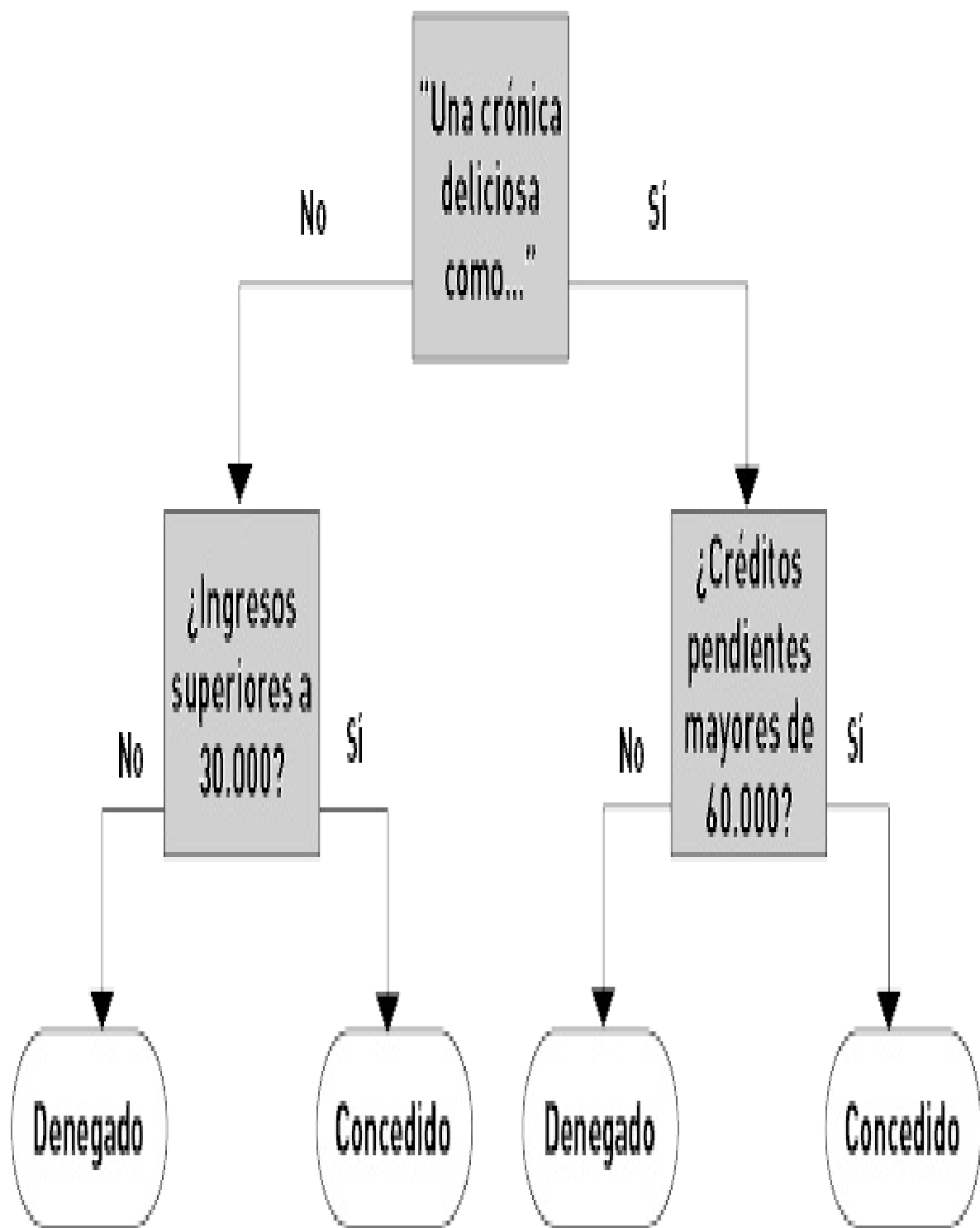
Hemos visto que los sistemas de IA basados en reglas presentaban limitaciones a la hora de, por ejemplo, representar la escritura humana. Aunque esto es cierto no significa que IA basada en reglas o símbolos, conocida en inglés cariñosamente como GOFAI (por las siglas de good old-fashioned artificial intelligence) no tenga en la actualidad su campo de aplicación.

De hecho, hay problemas que se resuelven mejor mediante reglas fijas y no por aproximaciones estadísticas. Es el caso de querer comprobar si el diseño de un nuevo chip cumple ciertas especificaciones para detectar errores antes del costoso proceso de fabricación. En este caso, se trabaja con reglas fijas bien establecidas, que especifican, por un lado, el comportamiento esperado de los componentes básicos individuales del chip y, por otro, definen lo que debe hacer el chip globalmente. Se trata entonces de ver si el comportamiento global se deduce de la especificación de los componentes básicos y sus conexiones o si, por el contrario, hay algún caso en el que el chip no actuará como debería. Como vemos, se trata de un tipo diferente de IA, que también resuelve problemas nada triviales, pero que no busca deducir reglas a partir de datos, sino que, al revés, parte de reglas y busca datos, en este caso ejemplos de posibles fallos de diseño.

Incluso hay casos en los que ambos casos de IA cooperan, encargándose de diferentes tareas para un mismo sistema. Es el caso de LLM como ChatGPT: aunque su funcionamiento básico es un modelo de generación de lenguaje, el sistema se complementa con una capa de reglas que buscan inferir si la pregunta o la respuesta generada incumple ciertas normas, como, por ejemplo, si la pregunta está relacionada con la comisión de un delito.

Incluso dentro del aprendizaje automático, es decir, centrándonos en la obtención de modelos a partir de datos, no todo son redes neuronales. Hay otros métodos clásicos, como por ejemplo los árboles de decisión que siguen resultando

efectivos. En un árbol de decisión el sistema examina características particulares de los datos para tratar de determinar la etiqueta o resultado. Por ejemplo, supongamos que en un banco tenemos datos de préstamos concedidos y de los clientes que los solicitaron, así como si pudieron devolver el crédito o no. Queremos, a partir de estos datos, obtener un modelo para conceder o denegar préstamos a partir de las características de los clientes. Una versión muy simplificada de un árbol de decisión sería la siguiente:



En este árbol, el sistema, que estudia nuestros datos históricos, ha decidido que el primer atributo o característica a considerar es la edad del cliente. Si el cliente supera los 50 años, entonces ha decidido que estadísticamente el siguiente atributo relevante es el monto de créditos pendientes, y que si este valor es inferior a los 60.000 euros entonces el crédito podrá devolverse sin problema, por lo que se concede, denegándose en otro caso. Por otro lado, si el cliente no supera los 50 años, el modelo dice que hay que considerar el nivel de ingresos, concediendo el préstamo a las personas en este rango de edad que tengan ingresos superiores a 30.000 euros. Se trata, por supuesto, de un ejemplo que no es real, un árbol para este tipo de problema tendría previsiblemente decenas de preguntas que considerarían muchas características diferentes, pero que sirve para representar la idea general.

Es cierto que, sobre todo en presencia de muchos datos, las redes neuronales resolverán con más acierto el problema que, por ejemplo, un árbol de decisión u otro método clásico. Sin embargo, los árboles también tienen su ventaja: son modelos explicables. Esto quiere decir que aunque las redes neuronales funcionen mejor no sabremos por qué conceden o deniegan un crédito. No es que sea imposible entender la red neuronal; al final es un modelo que podemos examinar en detalle, una gigantesca fórmula matemática. Lo que ocurre es que la enorme cantidad de neuronas en capas, cada una de ellas conectada con la siguiente con sus pesos asociados, las complejas funciones de activación, etc., hacen que sea un sistema que no nos permite tener intuiciones o descripciones sencillas. Podemos entenderlo localmente, pero no globalmente, es demasiado complejo. Funciona, pero no lo comprendemos.

En cambio, el árbol de decisión se entiende perfectamente; el banco podría decirle al cliente por qué no se le concede el crédito, aunque a veces también pueda haber gran cantidad de factores. En el caso de la red neuronal, solamente se puede decir que “el sistema lo ha decidido así”. Es por ese motivo por lo que la normativa europea conocida como AI Act y el Reglamento General de Protección de Datos (RGPD) indican que en actividades calificadas como de “alto riesgo”, que incluye la concesión de créditos, los modelos tienen que poder ser interpretables.

Esto impone una limitación a los sistemas de IA más potentes conocidos y ha creado una activa línea de investigación que busca poder “explicar” las salidas

de estos sistemas, la conocida como IA explicable o XAI.

La “no explicabilidad” de las redes neuronales generan también debate en cuanto al papel que juega el conocimiento nuevo obtenido por estos sistemas. Un caso ilustrativo se dio en 2018, cuando Google presentó un modelo que podía determinar el sexo de una persona a partir de una retinografía, es decir, una imagen de fondo de ojo. Esto era algo que, en teoría, no era posible según el conocimiento de la época, pero el modelo tenía una tasa de acierto superior al 97%. Modelos posteriores han mostrado, por ejemplo, altas tasas de acierto en la predicción de tumores en imágenes de tinciones estándar en anatomía patológica, pero se sigue sin saber qué “ve” el modelo para lograr esta detección.

En el caso de la medicina, estos modelos se utilizan siempre apoyados por otros sistemas tradicionales, pero continúan generando debate sobre si este tipo de herramientas se deben utilizar aunque no comprendamos por qué lo hacen. Los más alarmistas incluso dicen que el ser humano corre el riesgo de perder la curiosidad, el interés por saber y que se puede terminar confiando ciegamente en los modelos, abandonando toda pretensión de aumentar el conocimiento real. Por poner un ejemplo, un “Newton moderno” habría podido utilizar millones de medidas de caídas de cuerpos para entrenar una red que predijera perfectamente el comportamiento dinámico de los cuerpos pero sin llegar a enunciar la ley de la gravedad con sus numerosas implicaciones, tanto prácticas como filosóficas.

Otras opiniones, en cambio, indican que esto es una exageración o que incluso sucede todo lo contrario: la máquina únicamente nos señala caminos para investigar, no elimina nuestra curiosidad, sino que nos estimula. Los que mantienen este punto de vista citan como ejemplo que muchos descubrimientos se han hecho por casualidad y, sin embargo, el hombre nunca se ha conformado con ver que algo “funcionaba”, sino que se ha esforzado en descubrir su porqué. Seguramente, en los próximos años veremos más y más casos de descubrimientos no explicables similares que nos harán profundizar en este debate y en sus muy importantes implicaciones.

39. IA bioinspirada

Como hemos visto, las redes neuronales se basan en las neuronas del cerebro y sus interconexiones para generar modelos capaces de resolver problemas complejos. Pero no se trata del único caso de IA bioinspirada, es decir, surgida de la observación de los sistemas biológicos.

Es el caso de la programación evolutiva, que busca adaptar los principios del darwinismo a la búsqueda de soluciones de problemas complejos. Pongamos un ejemplo: tenemos un parque eólico y queremos buscar la configuración que maximice la energía que se produce en un momento dado. Dependiendo de las condiciones dadas de viento y previsión meteorológica, y teniendo en cuenta la distribución espacial de las turbinas y su tipo, la configuración indicará la orientación que deben tener las aspas, la cantidad de turbinas que deben ponerse en acción, etc. Suponemos, además, que podemos simular computacionalmente el efecto en la producción de energía de las distintas configuraciones de forma fiable. El problema de hallar la configuración óptima es muy complejo, porque como vemos son muchos los factores que interactúan entre sí y pequeños cambios en uno de ellos puede tener un gran efecto.

Lo que hace la programación evolutiva es buscar la mejor solución comenzando con muchas configuraciones generadas al azar. A cada configuración se la llama individuo y el conjunto total es la población inicial. Usando nuestro simulador, comprobamos la energía que cada configuración o individuo proporciona en un plazo determinado. Como hemos elegido las configuraciones al azar, todas serán en general malas, pero siempre habrá algunas mejores que otras. Los individuos que proporcionan menos energía son descartados, podemos decir que no han sobrevivido al no adaptarse al entorno del problema. Ahora, con la población reducida a los mejores individuos (recordemos: mejores configuraciones en nuestro caso), cada uno de los supervivientes “evoluciona” por el método de cambiar ligeramente sus parámetros, generando una nueva población, más especializada. Al partir de una configuración que ya ha sido seleccionada, la nueva generación tendrá algunos elementos que serán un poco mejores, simulando así el efecto de la evolución. Tras repetir este proceso varias veces, es decir, tras varias generaciones en las que únicamente sobreviven los individuos

más aptos, tendremos configuraciones que maximizan bastante bien la energía producida.

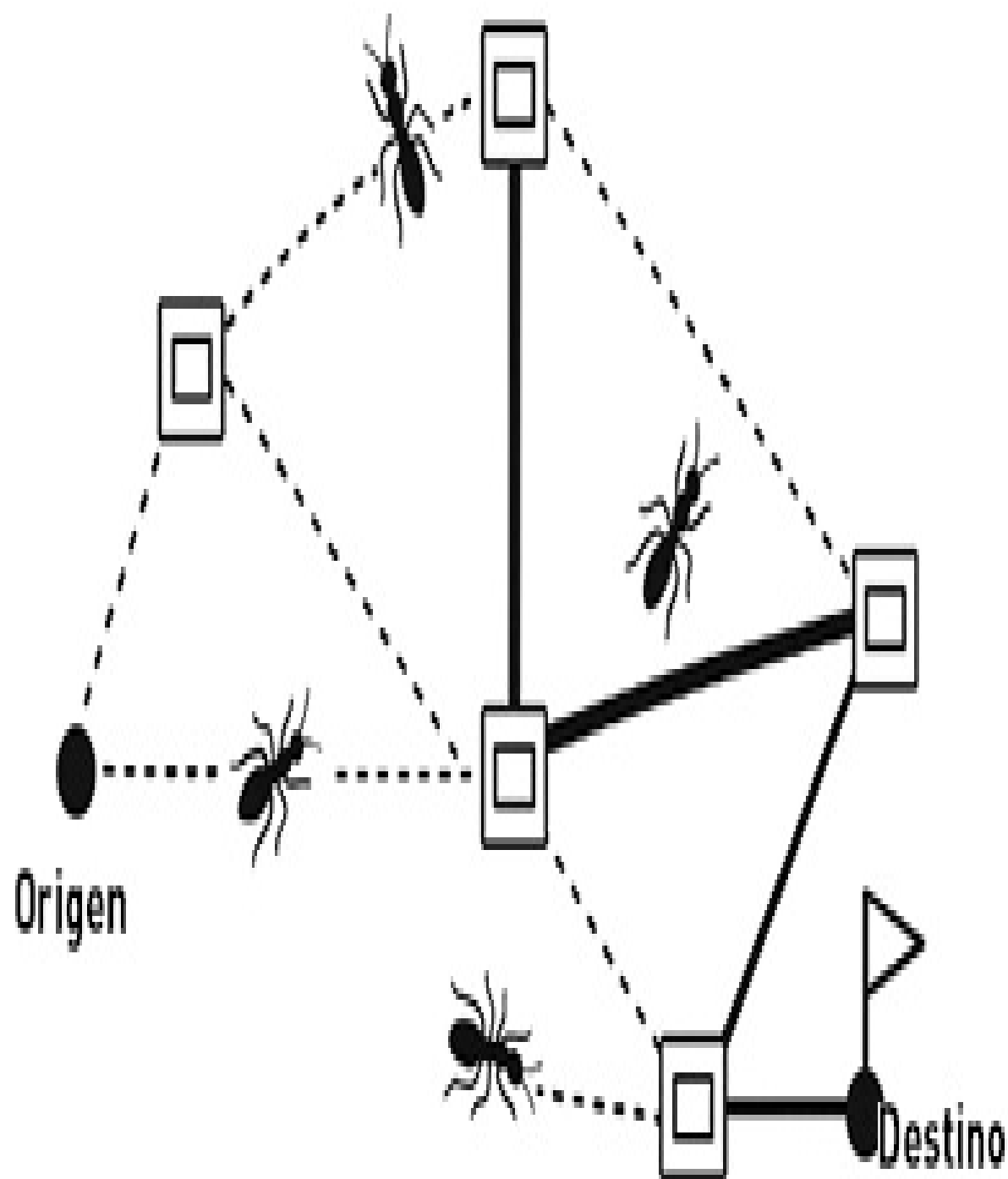
La programación evolutiva se considera un estándar de facto para la búsqueda de las mejores combinaciones en entornos complejos, por ejemplo, para planificación de tareas, y tiene un uso muy amplio en procesos industriales. Una variante de este tipo de programación se da cuando los individuos/soluciones que mejor se adaptan no continúan evolucionando de forma independiente, sino que se combinan entre sí, es decir se cruzan para dar lugar a individuos que combinan características de varios individuos de la población anterior. En este caso se habla de programación genética, una variante especialmente útil si las soluciones que se buscan no son simplemente combinaciones de parámetros, como en el caso del parque eólico, sino estructuras más complejas. Por ejemplo, en finanzas cuantitativas se busca encontrar la mejor secuencia de compras y ventas donde en cada caso la decisión depende del estado del mercado, así como de las compras y ventas anteriores. Tenemos en este caso que las soluciones o individuos no son conjuntos de números, sino árboles de decisión, que se pueden combinar entre sí sustituyendo una rama de uno por otra de otro árbol, algo de lo que se encarga la programación genética.

Otra forma de algoritmo bioinspirado se basa en el comportamiento de distintos tipos de insectos para lograr buenas soluciones a problemas complejos difíciles de tratar por otros mecanismos. Así, los algoritmos inspirados en los métodos de búsqueda y colonización de las hormigas, conocidos como ACO (ant colony optimization), exploran distintos caminos simultáneamente, simulando hormigas que buscan un objetivo. Pongamos otro ejemplo: la información en internet viaja en paquetes de datos, que recorren distintos puntos (routers) para llegar al objetivo. Como en principio no conocemos el camino más corto, lo intentamos por varios a la vez, cada intento es un agente u “hormiga”. Al principio, estas hormigas se mueven al azar explorando distintas vías, y al hacerlo dejan una traza de su camino, que representa sus feromonas. Si la hormiga encuentra por casualidad el destino, el lugar al que se debe enviar la información por internet, la hormiga vuelve sobre sus pasos reforzando con más feromonas ese camino. Si la hormiga tarda demasiado tiempo en encontrar el destino y volver, sus feromonas se habrán evaporado, es decir, la traza se va borrando con el tiempo.

Los siguientes agentes/hormigas tenderán a guiarse por las feromonas de sus predecesoras, es decir, los caminos que han permitido llegar rápido al destino tenderán a ser recorridos más a menudo, reforzando más aún las feromonas, que

a su vez guiarán a la siguiente hormiga. En cualquier caso, siempre habrá un factor aleatorio, puesto que habrá hormigas que seguirán otro camino ignorando la presencia de feromonas, lo que permitirá explorar otras posibilidades y no quedarse con la primera que se encuentra.

Caminos explorados y reforzados mediante colonización de hormigas



---- Exploración

— Camino reforzado (principal)

Fuente: Elaboración propia.

40. Superinteligencia

Hemos visto como los modelos de IA modernos aprenden automáticamente a partir de datos. Sin embargo, la arquitectura de la red neuronal que se utiliza debe haber sido previamente diseñada por los humanos, que además han seleccionado y preparado los datos. Igualmente, son los humanos los que principalmente evalúan los resultados y proceden al ajuste fino de los modelos. ¿Se podría automatizar todo el proceso? ¿Se podría llegar a pedir a ChatGPT que generara automáticamente una versión completamente nueva y mejor de sí mismo, e incluso que cuando lo logre le pida al nuevo modelo eso mismo, creando así una sucesión de versiones mejoradas de forma automática, posiblemente a un ritmo muchísimo más rápido que el actual? ¿Hasta dónde podría llevar ese proceso?

Esta idea fue planteada en 1965 por Irving John Good, compañero de Turing en su etapa en la que ambos trataban de descifrar los códigos generados por la máquina Enigma y, posteriormente, famoso científico, e incluso consultor del director Stanley Kubrick durante la preparación de la película 2001: Una odisea del espacio. Good propuso que podría llegar el momento en que una máquina suficientemente inteligente fuera capaz ella misma de diseñar máquinas aún mejores, provocando una “explosión de inteligencia”, o singularidad, es decir, una sucesión rapidísima de máquinas capaces de llevar a cabo tareas cada vez más difíciles, así hasta llegar a un límite imposible de imaginar, que daría lugar a la superinteligencia. Ya no harían falta más invenciones humanas, porque esta máquina, si es que fuéramos capaces de controlarla, podría ocuparse de todos los problemas.

¿Ciencia ficción? Hoy por hoy sí, pero muchos otros pensadores y científicos han planteado seriamente esta posibilidad y estudiado las posibilidades que tendría. Entre los que ven esta situación como no solo posible, sino como algo positivo para el ser humano destaca Ray Kurzweil. Para Kurzweil la superinteligencia no será algo que ocurrirá en los ordenadores de forma independiente al desarrollo humano. Por el contrario, ambas cosas irán unidas gracias al desarrollo de la biotecnología, la nanotecnología y la robótica, que harán que se produzca una fusión entre la mente humana y la máquina, creando

de esta forma un nuevo tipo de humano modificado por la ingeniería genética. Gracias a la conexión cerebral directa con IA, cada persona dispondrá de un conocimiento y de capacidades mucho más avanzadas que las actuales, e incluso se podrá interconectar con otros cerebros a través de la nube e internet. Según este científico y pensador, en 2029 la IA alcanzará nivel humano y alrededor de 2045 se alcanzará la singularidad, momento en que se disparará la inteligencia combinada, o incluso fusionada, de humanos y máquinas.

Las predicciones de Kurzweil suenan muy fantasiosas, y seguramente nadie las tomaría en serio si no se tratara de un reputado científico que ha ocupado puestos de relieve en empresas tecnológicas. Por ejemplo, Kurzweil ha sido el director de ingeniería de Google y en su larga carrera ha recibido numerosos premios como científico por los logros desarrollados, como la elaboración de los primeros reconocedores comerciales de caracteres (OCR). Además, Kurzweil es famoso por haber hecho predicciones acertadas tales como el éxito de internet o, más recientemente, que la IA sería capaz de contestarnos en un lenguaje difícil de distinguir del de un humano, anticipando así el comportamiento de los LLM actuales.

El pensamiento de Kurzweil se enmarca dentro de la corriente futurista conocida como transhumanismo, que opina que la fusión entre humanos y tecnologías mejorará nuestras capacidades cognitivas y emocionales. Sin embargo, otros autores ven con menos optimismo la posibilidad de la llegada de una superinteligencia. Por ejemplo, Nick Bostrom, aunque también admite que las máquinas puedan llegar a la superinteligencia, ve este hecho con preocupación, porque piensa que esa superinteligencia puede destinarse a fines que no sean los que más benefician al hombre.

Este desalineamiento entre los objetivos de la IA y los de los humanos puede llegar a darse por varias razones. En primer lugar, porque ni siquiera nosotros nos ponemos de acuerdo en qué es lo que más nos conviene a los humanos y tampoco sabemos cómo indicárselo a un sistema sin ambigüedades. Pensemos, por ejemplo, en las “leyes de la robótica” que propuso Isaac Asimov en sus obras de ciencia ficción, que tenían preceptos del estilo “un robot no hará daño a un ser humano ni, por inacción, permitirá que un ser humano sufra daño”. Algo así, en principio claro y bien definido, puede complicarse cuando el sistema decida que es “daño” y cómo se cuantifica; ¿podría la superinteligencia adoptar una perspectiva utilitarista y computar que el daño a unos pocos humanos compensa si se evita daño a un grupo más grande? ¿Podría interpretar, por

ejemplo, que un humano se está haciendo daño al comer algo no saludable e impedirselo? Esto es lo que Bostrom llama “instanciación perversa”: incluso una ley pensada con buenas intenciones puede ser interpretada erróneamente por un ordenador, con consecuencias imprevisibles.

Pero es que, además, un sistema superinteligente que no esté suficientemente bien supervisado podría llegar a definir sus propios objetivos. Incluso teniendo como objetivo supremo el bien de la humanidad, una IA podría decidir que lo mejor para la humanidad es que ella misma tome el mando y deje de obedecer a los humanos, menos inteligentes e incapaces de tomar sus propias decisiones. Tal IA podría decidir, por ejemplo, cómo se reparten los recursos entre los seres humanos, priorizando su propio consumo energético para generar nuevas versiones. Temores de este tipo, junto con otros, como el posible impacto de la IA en la desinformación o en la pérdida de puestos laborales, han llevado a científicos de renombre como Geoffrey Hinton, uno de los padres del deep learning y ganador del Premio Turing, a alertar sobre los peligros de estos nuevos desarrollos.

Por último, también hay que mencionar que otros autores como Yann LeCun, científico jefe de IA de Meta, opinan que tanto los temores como las esperanzas son infundados. Para LeCun, que compartió el Premio Turing con Geoffrey Hinton en 2018, la inteligencia artificial no implica deseos ni voluntad de poder por defecto y, aunque sin duda debe ser supervisada por humanos, se exageran tanto los temores como las expectativas.

Como vemos, en este tema sucede como con casi todos los de cariz menos técnico que hemos tratado hasta ahora: hay voces autorizadas discrepantes que defienden prácticamente todas las posturas posibles.

Esperamos que este libro, que aquí finaliza, haya ayudado al lector a tener su propia perspectiva de la IA actual, de sus posibilidades y riesgos, y que esto a su vez le permita utilizar adecuadamente estas poderosas herramientas que, sin duda, seguirán evolucionando y sorprendiéndonos en los próximos años.

Bibliografía

En la siguiente bibliografía comentada hemos reunido una selección de obras que abordan retos y oportunidades asociados a la inteligencia artificial. Estos libros presentan distintas perspectivas complementarias que ayudarán al lector a tener una visión global de los distintos desafíos que plantea la IA en nuestro presente y nuestro futuro.

Bostrom, Nick (2016): Superinteligencia: Caminos, peligros, estrategias, Zaragoza, Teell.

Obra de referencia que explora el escenario en el que la inteligencia artificial superaría a la humana y los riesgos existenciales que ello implica. El autor advierte que nuestro futuro podría llegar a quedar en manos de una IA “superinteligente” y que puede llegar a producirse el desalineamiento entre los objetivos de la máquina y los humanos. Para evitarlo, propone anticiparse con estrategias para controlar su desarrollo y asegurar que sus efectos sean beneficiosos para la humanidad.

Fry, Hannah (2019): Hola mundo: Cómo seguir siendo humanos en la era de los algoritmos, Barcelona, Blackie Books.

Analiza el creciente poder de los algoritmos en decisiones cruciales y cuestiona cuánto deberíamos delegar en ellos. Invita a reflexionar sobre nuestra relación con la inteligencia artificial y cómo nuestras decisiones presentes moldearán el futuro de la sociedad.

Harari, Yuval Noah (2015): Homo Deus: Breve historia del mañana, Barcelona, Penguin Random House.

Aunque el libro más conocido del autor sea Sapiens. Una breve historia de la humanidad, en este caso citamos esta otra obra donde se examinan los retos y posibilidades que enfrenta la humanidad en el siglo XXI. De especial interés es su reflexión sobre cómo la inteligencia artificial, la biotecnología y el poder de los datos transformarán la política, la economía e incluso la condición humana.

Kurzweil, Ray (2025): La singularidad está más cerca: Cuando nos fusionamos con la IA, Barcelona, Deusto.

Libro con interesantes predicciones sobre la inteligencia artificial y el avance tecnológico. Explora temas como la extensión radical de la vida, los nanobots, la impresión 3D y la automatización, ofreciendo una visión optimista y transhumanista del futuro impulsado por la IA.

Nicoletis, Miguel (2022): El verdadero creador de todo, Barcelona, Paidós.

Nicoletis propone una teoría innovadora sobre cómo el cerebro humano evolucionó hasta convertirse en un “ordenador orgánico” sin rival. Explica la historia y la civilización humanas a partir de tres propiedades clave del cerebro: su plasticidad, su capacidad para sincronizar mentes y su poder de abstracción. Defiende la singularidad de la mente humana y alerta sobre los riesgos que la tecnología puede suponer para nuestro futuro.

O’Neil, Cathy (2018): Armas de destrucción matemática: cómo el big data aumenta la desigualdad y amenaza la democracia, Madrid, Capitán Swing.

Libro que recopila diversos ejemplos de cómo los algoritmos y modelos de big data, lejos de ser neutrales, reproducen y amplifican desigualdades sociales. Explica que estas “armas de destrucción matemática” afectan a ámbitos como la educación, el empleo, la justicia o las finanzas, tomando decisiones opacas y difícilmente cuestionables por parte de los afectados.

Tegmark, Max (2018): Vida 3.0, Madrid, Taurus.

Este profesor y divulgador ha abogado ante el Senado de Estados Unidos por una gobernanza fuerte de la IA para poder disfrutar de sus beneficios minimizando sus riesgos. En 2023, llegó a pedir en una carta abierta que los Gobiernos intervinieran y que se detuviera el desarrollo de nuevos modelos hasta que se hubieran fijado protocolos de seguridad verificables. Una obra de referencia amena y de muy fácil lectura.

ÍNDICE DE TÉRMINOS

A

adulaciones

agente

agentes

ajedrez

ajuste de instrucciones

ajuste fino

AlexNet

algoritmo HOG

almacenamiento

almacenes de vectores

Alphabet

AlphaGo

AlphaGo Zero

alucinaciones

Amazon

Amazon Web Services (AWS)

análisis de sentimiento

análisis léxico

Apache 2.0

aprendizaje automático

aprendizaje no supervisado

aprendizaje por refuerzo

aprendizaje por transferencia

aprendizaje supervisado

árboles de decisión

Asimov, Isaac

ataques de canal lateral

ataques de denegación de servicio

ataques de evasión

atención

atención cruzada

atributos

autocodificadores

B

BERT

bibliotecas

big data

bloqueo epistemológico

bolsa de palabras

Bostrom, Nick

bots

C

cadena de razonamiento

Cambridge Analytica

capa de entrada

capas ocultas

CAPTCHA

características

centroides

centros de proceso de datos

ChatGPT

Claude

clúster

clustering

coches autónomos

codificador

Common Crawl

COMPAS

conciencia

conducción autónoma

de Tesla

Conferencia de Dartmouth

conjunto de validación

conjuntos de entrenamiento

y test

Copilot

copyleft

CUDA

D

dataísmo

datos etiquetados

decodificador

deepfakes

deep learning (aprendizaje profundo)

DeepMind

DeepSeek

E

ELIZA

embedding

Enigma

entidades nombradas

entrenamiento

envenenamiento de datos

episodio

equilibrado de las etiquetas

escalado horizontal

escalado vertical

espacio de búsqueda

etiqueta

evaluación

experiencia

explosión combinatoria

F

Facebook

fase de entrenamiento

few-shot

fraude

fuerza bruta

función de activación

G

GELU

Gemini

generación aumentada por recuperación (RAG)

Geolitica

GitHub

GitHub Copilot

GNU General Public Licence (GPL)

Go

gobernanza de datos

GOFAI

Good, Irving John

Google

GPGPU

GPT-2

GPT-3

gpt-oss

grandes modelos del lenguaje (LLM)

Grok

H

habitación china

Harari, Yuval Noah

Haskell

heurísticas

Hinton, Geoffrey

Hugging Face

I

IA bioinspirada

IA explicable (XAI)

índice

Inditex

inercia del conocimiento estancado

inframuestreo

ingeniería de instrucciones

instanciación perversa

inteligencia

inteligencia artificial generativa

internet de las cosas (IoT)

J

jailbreaking

Jie, Ke

juego de imitación

K

Kubrick, Stanley

Kurzweil, Ray

L

LeCun, Yann

lematización

Ley de IA (AI Act)

limpieza de datos

LISP

Llama

M

mantenimiento predictivo

McCarthy, John

Meta

mezcla de expertos (MoE)

Mistral

MIT

modelo

modelo base

modelos de difusión

modelos explicables

modelos propietarios

Musk, Elon

MYCIN

N

neurona artificial

neurona biológica

Nicoletis, Miguel

normalización

nube

núcleos

NVIDIA

O

Ollama

olvido catastrófico

OpenAI

operador Sobel

P

palabras vacías

parámetros

Penrose, Roger

perceptrón

pesos

Pierce, John R.

potemkins

PredPol

preprocesamiento

procesamiento del lenguaje natural (PLN)

programación evolutiva

programación genética

Prolog

prompts

Python

Q

Qwen

R

redes generativas adversativas (GAN)

redes neuronales

redes neuronales artificiales

redes neuronales convolucionales (CNN) 78

redes neuronales recurrentes (RNN)

retropropagación

RisCamvi

S

Searle, John

segmentación

sensores

sesgos

Shannon, Claude

Sidol, Lee

singularidad

Siri

sobreajuste

sobremuestreo

superinteligencia

T

T5

temperatura

test de Turing

TFLOPS

tokens

transformadores

transhumanismo

Turing, Alan

U

unidad central de procesamiento (CPU)

unidad de procesamiento gráfico (GPU)

uso legítimo

V

ventana de contexto

visión artificial

Z

zero-shot